# Optimization for Data Science

Convexity, gradient methods, duality, constrained algorithms

Francesco Secoli • Academic year 2024/2025

**Disclaimer.** Derived from the *Optimization for Data Science* course materials (Academic Year 2024/2025), MSc in Data Science & Business Informatics, University of Pisa.

These notes are open educational content created by a student. They are **not an academic source** and may contain inaccuracies. You may freely share, modify, and reuse this material for educational and non-commercial purposes with appropriate attribution. The content is my personal interpretation of the professor's course materials and should not replace official teaching resources. I assume no responsibility for any errors or misinterpretations.

These notes were produced with an **AI-in-the-middle** workflow: a first human pass, then **Claude Code** to support formulation, understanding, and rewriting, followed by a final human review.

If you find errors, have suggestions, or spot unintentionally included copyrighted material (which I will promptly remove on notification), contact me at `sclfnc@proton.me`.

# Contents

# 1 Foundations

Every optimization problem asks the same question: among all available choices, which one is best? We start from the simplest possible version of this question and progressively discover why it gets hard. The goal is not just to set up definitions: it is to understand *where the difficulty lives*, so that the tools developed in later sections are motivated by a genuine need.

## 1.1 Optimization Problems

**Definition 1.1 (Univariate function).** A function $f\colon \mathbb{R} \to \mathbb{R}$ maps an input $x$ to an output $y = f(x)$. Four sets capture its geometry:

- $\mathrm{Gr}(f) = \{(x, f(x)) \mid x \in \mathbb{R}\} \subset \mathbb{R}^2$ *[Graph]*
- $\mathrm{im}(f) = \{y \mid \exists\, x : y = f(x)\} \subset \mathbb{R}$ *[Image]*
- $L(f, v) = \{x \mid f(x) = v\} \subset \mathbb{R}$ *[Level set at value $v$]*
- $L(f, 0) = \{x \mid f(x) = 0\} \subset \mathbb{R}$ *[Roots]*

**Definition 1.2 (Optimization problem).** Given a function $f$ (the *objective*) and a set $X \subseteq \mathbb{R}^n$ (the *feasible region*), the optimization problem is

$$(P) \quad f^* = \inf\{f(x) : x \in X\}. \tag{1.1}$$

When the infimum is attained, we write min instead of inf and denote any minimizer by $x^* \in \arg\min\{f(x) : x \in X\}$. When $X = \mathbb{R}^n$, the problem is *unconstrained*; otherwise it is *constrained*. The optimal value $f^* = \nu(P)$ is unique, but the minimizer $x^*$ need not be. All minimizers share the same objective value, so they are equivalent "in the eyes of $f$." We write $X^* = L(f, f^*) \cap X$ for the *optimal set*, and call any $x \in X$ a *feasible solution* and any $x \notin X$ an *infeasible* one.

**Definition 1.3 (Types of constraints).** Constraints describing $X$ fall into three families:

- $g(x) = v$ *[Equality: a level set]*
- $g(x) \leq v$ *[Inequality: a sublevel set $S(g, v) = \{x : g(x) \leq v\}$]*
- $g_1(x) \leq 0 \ \wedge \ \cdots \ \wedge \ g_k(x) \leq 0$ *[Multiple: an intersection of sublevel sets]*

A common special case: *box constraints* $x_- \leq x \leq x_+$ with $x_-, x_+ \in \mathbb{R}^n$ and $x_- \leq x_+$ componentwise. If one wants $g(x) \geq 0$ instead, simply rewrite as $-g(x) \leq 0$.

These definitions apply to any $f$ and any $X$. But having a well-posed problem on paper does not mean we can solve it. Before tackling difficulty, we note a few transformations that are always free.

## 1.2 Simple Reformulations

Several transformations change $f^*$ in a predictable way without affecting $X^*$. These are *reformulations*: equivalent problems with identical optimal sets.

- **Min–max duality.** $\min\{f(x) : x \in X\} = -\max\{-f(x) : x \in X\}$, so $\arg\min f = \arg\max(-f)$. Minimization and maximization are interchangeable (but $\min f \neq \max f$ in general, often very different problems).
- **Additive constant.** $\min\{f(x) + c\} = c + \min\{f(x)\}$ for any $c \in \mathbb{R}$, preserving $X^*$.
- **Positive scaling.** $\min\{c\, f(x)\} = c \min\{f(x)\}$ for $c > 0$, again preserving $X^*$.

We use these freely. Throughout the text, "minimize" is assumed without loss of generality.

### 1.3 When the Optimum Does Not Exist

Solving $(P)$ actually involves (at least) two distinct tasks: finding $x^*$ and proving it is optimal, or constructively showing that $f$ is unbounded below. Both can fail in subtle ways.

**Definition 1.4 (Function unbounded below).** A function $f$ is *unbounded below* if for every $M > 0$ there exists some $x$ with $f(x) < -M$. No finite minimum exists; we write $f^* = -\infty$ as shorthand for "there is no finite lower bound on $\mathrm{im}(f)$." This rarely happens in data science (loss functions are typically nonnegative), but it is nontrivial and important in optimization theory, tied to duality and nonemptiness of feasible sets.

Even when $f^*$ is finite, a minimizer may fail to exist. Consider $f(x) = e^x$: the infimum is 0, but no $x$ achieves it: $\mathrm{im}(f) = (0, +\infty)$ is bounded below yet has no minimum. Another example: $f(x) = |x|$ for $x \neq 0$ and $f(0) = 1$; the infimum is 0, but $f$ never reaches it. In both cases $\inf\{f(x)\}$ exists, but $\min\{f(x)\}$ does not.

Why does this happen? Because $\mathrm{im}(f)$ is an *open* set that does not contain its boundary. The fix is conceptually simple: work with *closed* sets and *closed* intervals $[x_-, x_+]$ rather than open ones $(x_-, x_+)$. Could we use open boxes instead? Hardly: if $x_-$ and $x_+$ "can't be touched," just use $[x_- + \varepsilon_-,\, x_+ - \varepsilon_+]$ for appropriately small margins. On a computer, infinite precision is impossible anyway. We will generalize this to "just use closed sets and be done with it."

### 1.4 Extended Reals, Infima, and Suprema

To handle the pathological cases cleanly, we need a framework where infima always exist.

**Definition 1.5 (Infimum and supremum).** $\mathbb{R}$ is totally ordered: for any $x, y \in \mathbb{R}$, at least one of $x \leq y$ or $y \leq x$ holds. For $S \subseteq \mathbb{R}$: $s = \inf S$ if $s \leq t$ for all $t \in S$ and no larger number has this property; $s = \sup S$ is defined symmetrically. If $\inf S \in S$ then $\inf S = \min S$; analogously for sup.

Two issues remain: the infimum may not belong to $S$ (so min does not exist), and the infimum may not be finite. The *extended reals* $\overline{\mathbb{R}} = \{-\infty\} \cup \mathbb{R} \cup \{+\infty\}$ resolve the second issue: every subset of $\overline{\mathbb{R}}$ has an infimum and a supremum in $\overline{\mathbb{R}}$. We adopt the conventions $\inf \emptyset = +\infty$ and $\sup \emptyset = -\infty$.

With this, $f^*$ in Equation (1.1) always exists in $\overline{\mathbb{R}}$. Three outcomes are possible: $f^* = -\infty$ (unbounded below), $f^* \in \mathbb{R}$ but no $x^*$ attains it (infimum not achieved), or $f^* = f(x^*)$ for some $x^* \in X$ (minimum exists). Writing min instead of inf implicitly asserts the third case.

On computers, $x \in \mathbb{R}$ is really a rational number with at most 16 digits of precision, and data science is pushing toward even lower precision (float16, FP8). Finding the exact $x^*$ is impossible in general [1], but approximation errors are unavoidable anyway. What matters is finding *approximate* solutions.

## 1.5 Approximate Solutions and Optimality Gaps

**Definition 1.6 (Approximate solution).** Fix a tolerance $\varepsilon > 0$. A point $x_\varepsilon$ is an *$\varepsilon$-optimal solution* of $(P)$ if

$$f^* \leq f(x_\varepsilon) \leq f^* + \varepsilon. \tag{1.2}$$

How hard it is to find $x_\varepsilon$ depends on $\varepsilon$, on $f$, and on $X$.

Two standard measures quantify the gap between a candidate $x$ and the true optimum:

- $A(x) = f(x) - f^*$ *[Absolute gap]*
- $R(x) = A(x)/|f^*|$ *[Relative gap]*

The absolute gap is not scale-invariant. Replacing $(P)$ by $\min\{\alpha f(x)\}$ for $\alpha > 0$ gives $\nu(P_\alpha) = \alpha f^*$, so $R(x)$ stays the same while $A(x)$ scales by $\alpha$. The relative gap is ill-defined when $f^* = 0$; a common fix replaces $|f^*|$ with $\max\{|f^*|, 1\}$.

Both measures require knowledge of $f^*$, which is typically unknown. This is *the* central issue in optimization: computing (or estimating) $f^*$. Sometimes $f^*$ is approximately known: in neural network training, $f^* \approx 0$ is a reasonable guess, but in SVM training it is not. And $f^*$ is only needed if a global optimum is required, which is not always the case.

Much of optimization theory is devoted to bounding these gaps *without* knowing $f^*$: through duality, descent lemmas, or convergence-rate analysis.

## 1.6 Why Optimization is Hard

Even approximate optimization can be impossible. The reason is fundamental: isolated minima can hide anywhere [1]. Does restricting to a bounded interval $X = [x_-, x_+]$ help? No: there are still uncountably many points to examine. Is it because $f$ "jumps"? Not quite: even continuous functions can have downward spikes of arbitrarily small width, scattered anywhere in the domain. This is true even on a bounded interval.

The lesson: to make optimization *possible* (even approximately) we must impose regularity conditions on $f$. The function cannot be allowed to change too fast. The weaker the conditions, the harder the problem; the stronger the conditions, the faster our algorithms. This trade-off is the organizing principle for everything that follows:

- $f$ linear or quadratic with known coefficients $\Longrightarrow$ closed-form solution, $\mathcal{O}(1)$ (Section 2).
- $f$ Lipschitz continuous $\Longrightarrow$ solvable, but only with $\mathcal{O}(1/\varepsilon)$ evaluations (Section 3).
- $f$ quadratic, strongly convex $\Longrightarrow$ gradient methods with linear convergence, $\mathcal{O}(\log(1/\varepsilon))$ (Section 5).
- $f$ smooth ($C^1$ or $C^2$), convex $\Longrightarrow$ gradient-based methods, $\mathcal{O}(1/\varepsilon)$ (Section 6).

- $f$ nonsmooth but convex $\Longrightarrow$ subgradient and bundle methods, $\mathcal{O}(1/\varepsilon^2)$ (Section 7).

We begin with the nicest possible functions (the ones where optimization is essentially trivial) and work our way up.

## 1.7 Multivariate Problems

**Definition 1.7 (Multivariate optimization).** When $f\colon \mathbb{R}^n \to \mathbb{R}$ with $n > 1$, the feasible region can take many forms. A simple but common one is the *box* (hyperrectangle):

$$X = \{x \in \mathbb{R}^n : x_- \le x \le x_+\}, \qquad x_-, x_+ \in \mathbb{R}^n,\ x_- \le x_+, \tag{1.3}$$

where the inequalities hold component-wise.

The jump from $\mathbb{R}$ to $\mathbb{R}^n$ is dramatic. $\mathbb{R}^n$ is the Cartesian product of $\mathbb{R}$ with itself $n$ times: "exponentially larger" in the sense that even the simplest discrete subset grows fast. Restrict to integer points in the unit box $\{0,1\}^n$: you still have $2^n$ candidates. With $n = 30$, that is already over a billion ($2^{30} \approx 1.1 \times 10^9$); with $n = 100$, about $10^{30}$, far beyond any feasible enumeration. Continuous optimization sidesteps this combinatorial explosion by exploiting the structure of $f$, but only when $f$ is "nice enough."

The dimension $n$ can range from small (2, 3, 100) to large ($10^4$, $10^5$) to enormous ($10^9$, $10^{11}$). Even visualizing $f$ is harder: $\mathrm{Gr}(f) \subset \mathbb{R}^{n+1}$ is unpicturable for $n > 2$. Level sets $L(f, v) \subset \mathbb{R}^n$ help, but only for $n \le 3$. A more general tool (the *tomography*) slices the function along a chosen direction, reducing the multivariate problem to a univariate one. We develop this idea in Section 2.

## 1.8 Multi-objective Optimization

So far we assumed the value of any $x \in X$ can be expressed as a single number, which means that given $x'$ and $x''$, we can always tell which one we prefer. This relies on $\mathbb{R}$ having a total order. Often, however, there is more than one objective:

$$(P) \quad \min\{[f_1(x),\ f_2(x),\ \ldots,\ f_k(x)] : x \in X\}, \tag{1.4}$$

where $f_1, f_2, \ldots$ may be contrasting or measured in incomparable units. Car cost versus flashiness versus fuel efficiency; loss function versus regularizer in machine learning; expected return versus risk in portfolio selection.

**Definition 1.8 (Pareto optimality).** Since $\mathbb{R}^k$ for $k > 1$ lacks a total order, there is no single "best" solution. A feasible point $x^*$ is *Pareto-optimal* (or *non-dominated*) if no other $x \in X$ satisfies $f_i(x) \le f_i(x^*)$ for all $i$ with strict inequality for at least one $i$. The set of all Pareto-optimal points is the *Pareto frontier*.

The Pareto frontier is typically a curve (when $k = 2$) or surface (when $k > 2$), and a decision-maker must choose a point on it. Two practical reductions to a single-objective problem:

- **Scalarization:** $\min\{f_1(x) + \alpha\, f_2(x) : x \in X\}$. Different $\alpha > 0$ trace different points on the frontier.
- **Budgeting:** $\min\{f_1(x) : f_2(x) \le \beta,\ x \in X\}$, or symmetrically $\min\{f_2(x) : f_1(x) \ge \beta_1,\ x \in X\}$. The parameter $\beta$ controls the trade-off.

Choosing $\alpha$ or $\beta$ is inherently fuzzy: it is the nature of multi-objective problems. In machine learning, this is typically handled at the modelling stage: regularization hyperparameters are tuned via grid search, cross-validation, or similar techniques.

We assume from now on that the problem has been reduced to a single objective $f\colon \mathbb{R}^n \to \mathbb{R}$. The next section develops the algebraic tools needed to analyze the simplest nontrivial classes of objectives (linear and quadratic functions) and shows that even these "easy" cases require nontrivial machinery once $n > 1$.

## 2 Linear Algebra and Quadratic Forms

The nicest objectives are linear and quadratic, and even these require nontrivial algebra once the dimension grows. This section climbs a ladder of complexity: linear functions (trivial), univariate quadratics (trivial via a trick), separable multivariate quadratics (still easy), and finally general quadratics (where variables interact and the real work begins). At the top of the ladder sits spectral theory, which reveals that eigenvalues control everything about a quadratic: curvature, level-set geometry, solvability, and computational cost.

### 2.1 Scalar Products, Norms, and Distances

Before analyzing any function on $\mathbb{R}^n$, we need the geometric primitives: how to measure angles, lengths, and proximity.

**Definition 2.1 (Scalar product).** The *(Euclidean) scalar product* of $x, z \in \mathbb{R}^n$ is $\langle x, z\rangle = \sum_{i=1}^n x_i z_i$. It satisfies four defining properties: symmetry ($\langle x, z\rangle = \langle z, x\rangle$), positive definiteness ($\langle x, x\rangle \ge 0$ with equality iff $x = 0$), homogeneity ($\langle \alpha x, z\rangle = \alpha\langle x, z\rangle$), and linearity ($\langle x + w, z\rangle = \langle x, z\rangle + \langle w, z\rangle$). Other scalar products exist and make sense in other spaces (matrices, integrable functions, random variables); cf. the kernel trick in SVMs (Section 8).

The scalar product encodes direction: $\langle x, z\rangle = \|x\| \cdot \|z\| \cdot \cos(\theta)$, where $\theta$ is the angle between $x$ and $z$. Positive means "same direction," zero means orthogonal ($x \perp z$), negative means "opposite direction." The *Cauchy–Schwarz inequality* $|\langle x, z\rangle| \le \|x\|\|z\|$ is an immediate consequence.

**Definition 2.2 (Norm and distance).** The *(Euclidean) norm* of $x \in \mathbb{R}^n$ is $\|x\| = \sqrt{\langle x, x\rangle} = \sqrt{x_1^2 + \cdots + x_n^2}$. It satisfies: $\|x\| \ge 0$ with equality iff $x = 0$; $\|\alpha x\| = |\alpha|\|x\|$; and the *triangle inequality* $\|x + z\| \le \|x\| + \|z\|$. Two related identities: $\|x + z\|^2 = \|x\|^2 + \|z\|^2 + 2\langle x, z\rangle$ and the *parallelogram law* $2\|x\|^2 + 2\|z\|^2 = \|x + z\|^2 + \|x - z\|^2$. The *distance* between $x$ and $z$ is $d(x, z) = \|x - z\|$.

**Definition 2.3 (Ball).** The closed ball centered at $x \in \mathbb{R}^n$ with radius $r > 0$ is $\mathscr{B}(x, r) = \{z \in \mathbb{R}^n : \|z - x\| \le r\}$. Balls replace intervals as the basic notion of "neighborhood" in $\mathbb{R}^n$. The

norm determines the topology of the space (what is "close" to what) and this will matter when we discuss convergence and continuity.

The Euclidean norm is just one member of a large family. The *p-norm* is $\|x\|_p = (\sum_{i=1}^n |x_i|^p)^{1/p}$ for $p \geq 1$. Important special cases: $\|x\|_2$ (Euclidean), $\|x\|_1 = \sum_i |x_i|$ (used in Lasso regularization, Section 7.2), $\|x\|_\infty = \max_i |x_i|$ (Chebyshev), and $\|x\|_0 = \#\{i : |x_i| > 0\}$ (counting "norm", not actually a norm, since it violates homogeneity). Different $p$ give different ball shapes: $\mathscr{B}_1$ is a diamond, $\mathscr{B}_2$ a sphere, $\mathscr{B}_\infty$ a hypercube. In finite dimension all norms are *equivalent*: for any two norms $\|\cdot\|$ and $\|\cdot\|'$, there exist $0 < \alpha \leq \beta$ such that $\alpha\|x\| \leq \|x\|' \leq \beta\|x\|$ for all $x$. This means convergence in one norm implies convergence in every other: the topology of $\mathbb{R}^n$ doesn't depend on the choice of norm.

## 2.2 Matrices

A matrix $A \in \mathbb{R}^{m\times n}$ has $m$ rows $A_i \in \mathbb{R}^n$ and $n$ columns $A^j \in \mathbb{R}^m$. The *transpose* $A^T \in \mathbb{R}^{n\times m}$ exchanges rows with columns: $(A^T)_{ij} = A_{ji}$. A square matrix $A \in \mathbb{R}^{n\times n}$ is *symmetric* if $A = A^T$, equivalently $A_{ij} = A_{ji}$ for all $i, j$. A scalar $a \in \mathbb{R}$ can be viewed as a $1 \times 1$ symmetric matrix.

$\mathbb{R}^n$ is a finite-dimensional vector space of dimension $n$. The *canonical basis* $\{u^1, \ldots, u^n\}$ consists of the $n$ unit vectors: $u^i_j = 1$ if $j = i$ and 0 otherwise. Every $x \in \mathbb{R}^n$ is a linear combination $x = \sum_i x_i u^i$.

The matrix–vector product $Ax \in \mathbb{R}^m$ (for $x \in \mathbb{R}^n$) has $i$-th component $\langle A_i, x\rangle$; the product $x^T A$ (for $x \in \mathbb{R}^m$) gives a row vector with $j$-th component $\langle A^j, x\rangle$. The scalar product can be written $\langle x, z\rangle = x^T z = z^T x$. The matrix–matrix product $AB$ (for $A \in \mathbb{R}^{m\times k}$, $B \in \mathbb{R}^{k\times n}$) has entry $(AB)_{ij} = \langle A_i, B^j\rangle$ and costs $\mathscr{O}(nmk)$: expensive unless $A$ or $B$ is sparse.

The *spectral norm* (or *operator norm*) of $A$ is $\|A\| = \max_{\|x\|=1}\|Ax\| = \sigma_{\max}(A) = \sqrt{\lambda_{\max}(A^T A)}$, where $\sigma_{\max}$ is the largest singular value. For symmetric $A$, $\|A\| = \max_i |\lambda_i|$. The spectral norm is submultiplicative ($\|AB\| \leq \|A\|\|B\|$) and controls how much a matrix can stretch a vector. The *spectral radius* $\rho(A) = \max_i |\lambda_i(A)|$ satisfies $\rho(A) \leq \|A\|$, with equality when $A$ is symmetric.

## 2.3 Linear Functions

The simplest possible objective is linear. We start univariate, then generalize.

### Univariate case

A *linear univariate function* is $f(x) = bx$ for a fixed $b \in \mathbb{R}$. The parameter $b$ is the *slope*: if $b > 0$, $f$ is increasing; if $b < 0$, decreasing; if $b = 0$, constant. Unconstrained optimization is trivial: $\min f = -\infty$ and $\max f = +\infty$ unless $b = 0$, in which case both equal 0.

Box-constrained optimization $\min\{bx : x \in [x_-, x_+]\}$ is equally immediate. When $b > 0$, the minimum sits at $x_-$ and the maximum at $x_+$; when $b < 0$ the roles reverse; when $b = 0$ every point is optimal. This "works" even if $x_- = -\infty$ or $x_+ = +\infty$, since $b \cdot (\pm\infty) = \pm\infty$ by convention. A closed-form $\mathscr{O}(1)$ solution: don't get used to it.

### Multivariate case

A *linear multivariate function* is $f(x) = \langle b, x\rangle = \sum_{i=1}^{n} b_i x_i$ for a fixed $b \in \mathbb{R}^n$. Linearity means $f(\gamma x) = \gamma f(x)$ and $f(x+z) = f(x) + f(z)$. The graph of $f$ is a hyperplane in $\mathbb{R}^{n+1}$; level sets are parallel hyperplanes in $\mathbb{R}^n$, all perpendicular to $b$.

For box-constrained optimization on a hyperrectangle $X = \{x : x_- \leq x \leq x_+\}$, the problem separates into $n$ independent univariate problems (one per coordinate), each solvable in $\mathcal{O}(1)$. Total cost: $\mathcal{O}(n)$. This is the last time optimization will be this easy.

## 2.4 Quadratic Univariate Functions

The next step up: $f(x) = ax^2 + bx$ for fixed $a, b \in \mathbb{R}$. A homogeneous quadratic ($b = 0$) plus a linear ($a = 0$) term.

### Homogeneous case ($b = 0$)

$f(x) = ax^2$ is symmetric around the origin. If $a > 0$, the parabola opens upward: $f$ decreases for $x < 0$, increases for $x > 0$, and the minimum is at $x = 0$. If $a < 0$, it opens downward: the maximum is at $x = 0$, and $f$ is unbounded below. If $a = 0$, $f$ is constant.

Box-constrained optimization on $[x_-, x_+]$ requires a case analysis. When $a > 0$: if $0 \in [x_-, x_+]$ then $\arg\min = 0$ and $\arg\max = \arg\max\{f(x_-), f(x_+)\}$; if $x_- > 0$ then $\arg\min = x_-$; if $x_+ < 0$ then $\arg\min = x_+$. Still $\mathcal{O}(1)$.

### Non-homogeneous case ($a \neq 0$, $b \neq 0$)

The linear term $bx$ breaks the symmetry around the origin. A powerful trick restores it: define $\bar{x} = -b/(2a)$ and substitute $z = x - \bar{x}$. Expanding $f(x) = a(z + \bar{x})^2 + b(z + \bar{x})$ and using $2a\bar{x} + b = 0$, the cross-terms cancel:

$$f(x) = az^2 + f(\bar{x}). \tag{2.1}$$

The problem reduces to a homogeneous quadratic in $z$ plus a constant offset. The point $\bar{x}$ is the *center* of the function: the vertex of the parabola. If $a > 0$, the unique minimum is at $\bar{x}$; if $a < 0$, the unique maximum. Still $\mathcal{O}(1)$.

**Remark 2.4.** This "completing the square" trick will reappear in the multivariate setting (Section 2.12.2). The idea is always the same: shift the origin to the center, where the function becomes homogeneous and therefore easier to analyze.

## 2.5 Separable Quadratic Functions

A *separable (non-homogeneous) quadratic* is

$$f(x) = \sum_{i=1}^{n} (a_i x_i^2 + b_i x_i), \qquad (a, b) \in \mathbb{R}^{2n}. \tag{2.2}$$

This is a sum of $n$ univariate quadratics (one per coordinate) with no cross-terms. Each can be minimized independently by completing the square: the center is $\bar{x}_i = -b_i/(2a_i)$ when $a_i \neq 0$. Total cost: $\mathcal{O}(n)$.

An important special case is $f(x) = \|x\|^2 = \sum_i x_i^2$ (all $a_i = 1$, $b_i = 0$), whose level sets are circles (when $n = 2$) or spheres. When $a_1 \neq a_2$, the level sets become axis-aligned ellipses: the larger $a_i/a_j$, the more elongated. Non-zero $b_i$ simply shifts the center from the origin to $\bar{x}$.

This is the last time we get closed formulas. The separable structure means variables don't interact: each $x_i$ can be chosen without knowing the others. In a general quadratic, this independence breaks.

## 2.6 Tomographies

Before tackling the general multivariate case, we need a tool to visualize and analyze functions of $n$ variables. The graph $\mathrm{Gr}(f) \subset \mathbb{R}^{n+1}$ is unpicturable for $n > 2$. Level sets help but live in $\mathbb{R}^n$, still hard for $n > 3$. The solution: "slice" the function along a chosen direction and study the resulting one-dimensional function.

**Definition 2.5 (Tomography).** Given $f\colon \mathbb{R}^n \to \mathbb{R}$, an origin $x \in \mathbb{R}^n$, and a direction $d \in \mathbb{R}^n$, the *tomography* of $f$ along $d$ from $x$ is

$$\varphi_{x,d}(\alpha) = f(x + \alpha d)\colon \mathbb{R} \to \mathbb{R}. \tag{2.3}$$

When $x$ and $d$ are clear from context, we write simply $\varphi(\alpha)$.

The norm of $d$ only changes the horizontal scale: $\varphi_{x,\beta d}(\alpha) = \varphi_{x,d}(\beta\alpha)$, so it is often convenient to use a normalized direction $\|d\| = 1$. Restricting to a coordinate direction $u^i$ (the $i$-th canonical basis vector) gives the partial restriction $f_x^i(\alpha) = f(x_1, \ldots, x_{i-1}, \alpha, x_{i+1}, \ldots, x_n)$.

### Tomography of a linear function

For $f(x) = \langle b, x\rangle$ with origin $x = 0$ and $\|d\| = 1$:

$$\varphi(\alpha) = \alpha\langle b, d\rangle = \alpha\|b\|\cos(\theta), \tag{2.4}$$

where $\theta$ is the angle between $b$ and $d$. The tomography is a linear function of $\alpha$. It is steepest when $d$ is collinear with $b$ (either same direction or opposite); flat when $d \perp b$. The direction $-b/\|b\|$ gives the steepest decrease: a foreshadowing of the gradient method.

Line search methods exploit tomographies directly: pick a descent direction $d$, then minimize $\varphi_{x,d}$ over $\alpha \geq 0$. We will see this idea at work in every gradient-based algorithm.

## 2.7 The General Quadratic Function

Now the variables interact. The general (homogeneous) quadratic on $\mathbb{R}^n$ is

$$f(x) = \frac{1}{2}\, x^T Q x = \frac{1}{2}\Big[\sum_{i=1}^{n} Q_{ii} x_i^2 + \sum_{\substack{i,j=1 \\ j\neq i}}^{n} Q_{ij} x_i x_j\Big], \tag{2.5}$$

for a fixed $Q \in \mathbb{R}^{n\times n}$. The off-diagonal terms $Q_{ij}x_ix_j$ couple the variables: changing $x_i$ affects the optimal choice for $x_j$. This coupling is what makes general quadratics fundamentally harder than separable ones.

An important observation: even if $Q$ is not symmetric, we can always replace it with $(Q+Q^T)/2$ without changing $f$. Indeed, $x^TQx = x^T[(Q+Q^T)/2]\,x$. So there is no loss of generality in assuming $Q$ symmetric from the start.

The tomography along a direction $d$ from the origin gives

$$\varphi(\alpha) = f(\alpha d) = \frac{1}{2}\,\alpha^2(d^TQd). \tag{2.6}$$

This is a univariate parabola whose sign and steepness depend entirely on $d^TQd$. Different directions yield different curvatures. Understanding this dependence requires knowing the spectrum of $Q$.

## 2.8 Eigenvalues and Spectral Decomposition

**Definition 2.6 (Eigenvalues and eigenvectors).** Given a matrix $Q \in \mathbb{R}^{n\times n}$, a scalar $\lambda \in \mathbb{R}$ is an *eigenvalue* and a nonzero vector $v \in \mathbb{R}^n$ is the corresponding *eigenvector* if $Qv = \lambda v$. An eigenvector is a direction along which $Q$ acts as a simple rescaling.

Eigenvalues can be complex in general, and eigenvectors need not be orthogonal. Symmetric matrices are the exception, and the one that matters for optimization.

**Theorem 2.7 (Spectral theorem for symmetric matrices).** *If $Q \in \mathbb{R}^{n\times n}$ is symmetric, then $Q$ has $n$ real eigenvalues (counted with multiplicity) and $n$ eigenvectors that can be chosen orthonormal.*

We denote these orthonormal eigenvectors $H_1, \dots, H_n \in \mathbb{R}^n$ with corresponding eigenvalues $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n$ (ordered from largest to smallest), so that $QH_i = \lambda_i H_i$ for all $i$.

**Definition 2.8 (Eigenvector matrix and spectral decomposition).** Collecting the eigenvectors into $H = [H_1 \mid \cdots \mid H_n] \in \mathbb{R}^{n\times n}$, orthonormality gives $H^TH = I$ (so $H$ is orthogonal). The *spectral decomposition* is

$$Q = H\Lambda H^T = \sum_{i=1}^{n} \lambda_i H_i H_i^T, \qquad \Lambda = \operatorname{diag}(\lambda_1, \dots, \lambda_n). \tag{2.7}$$

Each term $\lambda_i H_i H_i^T$ is a rank-one matrix. The decomposition expresses $Q$ as a weighted sum of these terms, one per eigenvalue.

Why does this matter? The spectral decomposition turns matrix operations into scalar ones. If we change coordinates via $y = H^Tx$, the quadratic $x^TQx$ becomes $\sum_i \lambda_i y_i^2$: a sum of independent squared terms, each scaled by an eigenvalue. Large eigenvalues stretch, small ones compress. The ratio between the largest and smallest controls how "distorted" the quadratic is, and (as we will see) how fast gradient methods converge.

**Theorem 2.9 (Variational characterization of eigenvalues).** *For a symmetric $Q$, the extreme eigenvalues satisfy*

$$\lambda_1 = \max\left\{\frac{d^T Q d}{d^T d} : d \neq 0\right\}, \qquad \lambda_n = \min\left\{\frac{d^T Q d}{d^T d} : d \neq 0\right\}. \tag{2.8}$$

*The ratio $d^T Q d / d^T d$ is the Rayleigh quotient. The maximum is attained at $d = H_1$, the minimum at $d = H_n$.*

### Determinant and inverse

**Definition 2.10 (Determinant).** The determinant of $Q$ equals the product of its eigenvalues: $\det(Q) = \prod_{i=1}^{n} \lambda_i$. If $\det(Q) \neq 0$, then $Q$ is nonsingular, every eigenvalue is nonzero, and $Q^{-1}$ exists. The eigenvectors of $Q^{-1}$ are the same as those of $Q$, with reciprocal eigenvalues: $Qv = \lambda v$ implies $Q^{-1}v = (1/\lambda)\, v$. It follows that $\det(Q^{-1}) = \det(Q)^{-1}$.

Computing eigenvalues by finding roots of the characteristic polynomial $\det(Q - \lambda I) = 0$ is practical only for $n = 2$ (quadratic formula) or $n = 3$ (cubic formula). For $n = 2$: $\det(Q) = Q_{11}Q_{22} - Q_{12}^2$ (using symmetry). For larger $n$, iterative algorithms (QR, divide-and-conquer) are used instead [2]. Diagonal and triangular matrices are the exception: their eigenvalues are the diagonal entries.

## 2.9 Matrix Definiteness

The sign pattern of the eigenvalues determines the shape of the associated quadratic, and therefore the nature of its critical points.

**Definition 2.11 (Matrix definiteness).** A symmetric matrix $Q$ is:

- *Positive definite* ($Q \succ 0$): all $\lambda_i > 0$. The quadratic $\frac{1}{2}x^T Q x$ has a unique global minimum at the origin.
- *Negative definite* ($Q \prec 0$): all $\lambda_i < 0$. Unique global maximum at the origin.
- *Positive semidefinite* ($Q \succeq 0$): all $\lambda_i \geq 0$. The quadratic is convex, but the minimum need not be unique: flat directions correspond to zero eigenvalues.
- *Negative semidefinite* ($Q \preceq 0$): all $\lambda_i \leq 0$. Concave, with a possibly non-unique maximum.
- *Indefinite*: eigenvalues of both signs. The origin is a saddle point.

## 2.10 Tomographies of Quadratic Functions

The spectral decomposition makes tomographies along eigenvectors completely transparent.

$$\varphi_{0,H_i}(\alpha) = f(\alpha H_i) = \frac{1}{2}\,\alpha^2 \lambda_i. \tag{2.9}$$

Along eigenvector $H_i$, the quadratic reduces to a univariate parabola scaled by $\lambda_i$. The steepest upward curvature is along $H_1$ ($\lambda_1$, the largest eigenvalue); the steepest downward curvature

(or least upward) is along $H_n$ ($\lambda_n$, the smallest). Intermediate directions give intermediate curvatures, always in $[\lambda_n, \lambda_1]$.

**Example 2.12 (Tomographies under different definiteness).** Consider $n = 2$ with $H = \frac{\sqrt{2}}{2}\left[\begin{smallmatrix} -1 & 1 \\ 1 & 1 \end{smallmatrix}\right]$ (fixed eigenvectors, rotated 45°).

- $Q \succ 0$ with $\lambda = (8, 4)$: $d^T Q d > 0$ for all $d \neq 0$. Every tomography is an upward parabola. Steepest along $H_1$ ($\lambda_1 = 8$); least steep along $H_2$ ($\lambda_2 = 4$).
- $Q \succeq 0$ with $\lambda = (8, 0)$: $d^T Q d \geq 0$, but the tomography along $H_2$ is completely flat. The quadratic has a "valley" in the $H_2$-direction.
- $Q$ indefinite with $\lambda = (8, -2)$: $d^T Q d$ can be positive or negative depending on $d$. Along $H_1$ the parabola opens upward; along $H_2$ it opens downward. The origin is a saddle point.
- $Q \prec 0$ with $\lambda = (-4, -8)$: $d^T Q d < 0$ for all $d \neq 0$. Every tomography is a downward parabola. Steepest negative curvature along $H_2$ ($\lambda_2 = -8$).

## 2.11 Level Sets and Geometry

The spectral decomposition also governs the shape of level sets. For a homogeneous quadratic $f(x) = \frac{1}{2}x^T Q x$, all level sets are centered at the origin (since $f$ is symmetric: $f(x) = f(-x)$).

The level set $L(f, v)$ intersects each eigendirection $H_i$ at distance $\sqrt{2v/\lambda_i}$ from the origin (provided $\lambda_i > 0$ and $v > 0$). So the eigenvectors are the axes of the level-set ellipsoid, and the axis lengths are $\sqrt{2v/\lambda_i}$. Smaller $\lambda_i$ means a longer axis: the function is flatter in that direction, so it takes more distance to reach a given function value.

Four geometric regimes correspond to the four definiteness classes:

- All $\lambda_i > 0$ (positive definite): level sets are ellipsoids. When $\lambda_1 = \lambda_n$ (identity), they are spheres.
- Some $\lambda_i = 0$ (positive semidefinite): level sets are "degenerate" ellipsoids: cylinders extending to infinity along the null-space directions.
- Eigenvalues of both signs (indefinite): level sets are hyperboloids. The positive-eigenvalue directions give the "bowl" part; the negative-eigenvalue directions give the "saddle" part.
- All $\lambda_i < 0$ (negative definite): level sets are ellipsoids again (the function is a downward paraboloid).

## 2.12 Optimizing Quadratic Functions

Definiteness of $Q$ and the presence of a linear term jointly determine whether (and how) a quadratic can be optimized.

### *2.12.1 Homogeneous Multivariate Quadratics*

For $f(x) = \frac{1}{2}x^T Q x$ (no linear term), optimality depends entirely on the eigenvalue signs:

- $Q \succeq 0$ and $Q \preceq 0$ simultaneously (i.e., $Q = 0$): $f \equiv 0$, min = max = 0.
- $Q \succeq 0$: min = 0 at $x = 0$, but max = $+\infty$.
- $Q \preceq 0$: max = 0 at $x = 0$, but min = $-\infty$.

- $Q$ indefinite: both min $= -\infty$ and max $= +\infty$.

What about box-constrained optimization $\min\{f(x) : x \in [x_-, x_+]\}$? Unlike the univariate case, this is *not* easy in general. Optimizing a quadratic over a box is $\mathscr{NP}$-hard for indefinite $Q$ [3]. Even when $Q \succeq 0$, the problem is polynomial but nontrivial: the algorithms we develop in Section 5 are needed. Minimization and maximization behave very differently here.

#### 2.12.2 Non-homogeneous, Nonsingular Case

The full quadratic is

$$f(x) = \frac{1}{2}\, x^T Q x + \langle q, x\rangle + c, \qquad Q \in \mathbb{R}^{n\times n},\ q \in \mathbb{R}^n,\ c \in \mathbb{R}. \tag{2.10}$$

When $Q$ is nonsingular ($\det(Q) \neq 0$, equivalently $\lambda_i \neq 0$ for all $i$), we can repeat the univariate trick. Setting $\nabla f(x) = Qx + q = 0$ yields the critical point $\bar{x} = -Q^{-1}q$. Think of $\bar{x}$ as the center of the function. Substituting $z = x - \bar{x}$ and using $Q\bar{x} = -q$, all linear terms cancel:

$$f(x) = \frac{1}{2}\, z^T Q z + f(\bar{x}). \tag{2.11}$$

The problem reduces to a homogeneous quadratic in $z$ plus a constant offset. If $Q \succ 0$, the unique minimum sits at $z = 0$, i.e., $x = \bar{x}$. If $Q$ is indefinite, $\bar{x}$ is a saddle point. This is the exact multivariate analogue of the completing-the-square trick.

#### 2.12.3 Non-homogeneous, Singular Case

When $Q$ is singular, $Q^{-1}$ doesn't exist and the trick above fails. The analysis uses the kernel–image decomposition of $Q$.

Let $I^0 = \{i : \lambda_i = 0\}$ and $I^+ = \{i : \lambda_i \neq 0\}$, with $k = |I^0| > 0$. The kernel $\ker(Q) = \{v : Qv = 0\}$ is the span of $\{H_i\}_{i\in I^0}$, and the image $\mathrm{im}(Q)$ is the span of $\{H_i\}_{i\in I^+}$. Since the eigenvectors form an orthonormal basis, $\mathbb{R}^n = \mathrm{im}(Q) \oplus \ker(Q)$ with $\mathrm{im}(Q) \perp \ker(Q)$.

Decompose $q = q^+ + q^0$ with $q^+ \in \mathrm{im}(Q)$ and $q^0 \in \ker(Q)$. The component $q^+$ lives in the range of $Q$, so we can absorb it via a partial change of variables (finding $\bar{x}$ with $Q\bar{x} = -q^+$). With $z = x - \bar{x}$:

$$f(x) = \frac{1}{2}\, z^T Q z + \langle q^0, z\rangle + f(\bar{x}). \tag{2.12}$$

The quadratic term $\frac{1}{2} z^T Q z$ kills the kernel directions (because $Qv = 0$ for $v \in \ker(Q)$), but the linear term $\langle q^0, z\rangle$ may not vanish along those directions. What matters is $q^0$:

- If $q^0 = 0$ (equivalently $q \in \mathrm{im}(Q)$, or $Q\bar{x} = -q$ has a solution): a minimum exists (assuming $Q \succeq 0$), but it is not unique. The entire affine subspace $\bar{x} + \ker(Q)$ consists of minimizers: the function is flat along null-space directions, and all these points share the same objective value $f(\bar{x})$.
- If $q^0 \neq 0$ (equivalently $q \notin \mathrm{im}(Q)$): no minimum exists. Moving along the null-space direction $q^0$, the quadratic term stays constant but the linear term $\langle q^0, z\rangle$ keeps decreasing. The function is unbounded below.

In either case, the key computational task is the same: solve $Q\bar{x} = -q$ (or prove it has no solution). This is a linear system: $\mathscr{O}(n^3)$ at worst, doable for moderate $n$, but prohibitive for $n \approx 10^9$. Iterative methods that avoid forming $Q^{-1}$ become essential, and they are the subject of Section 5.

# 3 Univariate Optimization

Linear and quadratic functions have closed-form optima. For anything more complex (a transcendental objective, a polynomial of high degree, a black-box simulation) we need iterative algorithms. This section develops the key univariate methods, from slow and reliable to fast and fragile. At each step, we trade a stronger assumption on $f$ for a faster algorithm. The section ends with the question: what if the local optimum we find is not the global one?

## 3.1 Lipschitz Continuity: Making Optimization Possible

Section 1.6 showed that optimization is impossible for arbitrary functions: isolated minima can hide in arbitrarily narrow spikes. The cure: impose a "speed limit" on how fast $f$ can change.

**Definition 3.1 (Lipschitz continuous function).** A function $f$ is *Lipschitz continuous* with constant $L > 0$ on a set $X$ if

$$|f(x) - f(z)| \leq L\,|x - z| \qquad \forall\, x, z \in X. \tag{3.1}$$

When $X = \mathbb{R}$, $f$ is *globally* $L$-Lipschitz. When $X = [x - \varepsilon,\, x + \varepsilon]$ for some $\varepsilon > 0$, $f$ is *locally* $L$-Lipschitz at $x$. The constant $L$ depends on $X$: locally Lipschitz everywhere does not imply globally Lipschitz ($f(x) = x^2$ is the standard counterexample).

**Definition 3.2 (Continuity).** A function $f$ is *continuous* at $x$ if

$$\forall\, \varepsilon > 0,\ \exists\, \delta > 0 : z \in [x - \delta,\, x + \delta] \ \Rightarrow\ |f(z) - f(x)| < \varepsilon. \tag{3.2}$$

If $f$ is continuous at every $x \in \mathbb{R}$, we write $f \in C^0$. Many simple functions are continuous, and continuity is preserved under sums, products, composition, max, and min.

**Theorem 3.3 (Lipschitz implies continuity).** *If $f$ is locally Lipschitz at $x$, then $f$ is continuous at $x$.*

*Proof.* Given $\varepsilon > 0$, choose $\delta = \varepsilon / L$. Then $|z - x| < \delta$ implies $|f(z) - f(x)| \leq L|z - x| < L\delta = \varepsilon$. □

The converse fails: $f(x) = \sqrt{|x|}$ is continuous everywhere but not Lipschitz on any interval containing the origin (it becomes "infinitely steep" as $x \to 0$).

### Complexity of Lipschitz optimization

Lipschitz continuity is the weakest useful regularity condition. With it, optimization becomes possible, but expensive.

**Theorem 3.4 (Complexity for Lipschitz functions).** *Let $f$ be $L$-Lipschitz on a bounded interval $[x_-, x_+]$ with diameter $D = x_+ - x_-$. A uniform sampling strategy (evaluating $f$ at points spaced at most $2\varepsilon/L$ apart) finds an $\varepsilon$-optimal solution in*

$$\mathcal{O}\left(\frac{L \cdot D}{\varepsilon}\right) \tag{3.3}$$

*function evaluations. No algorithm can guarantee a better worst-case rate for the class of all L-Lipschitz functions [1].*

The bound scales as $1/\varepsilon$: each extra digit of accuracy costs ten times more evaluations. Beyond $\varepsilon \approx 10^{-3}$ or $10^{-4}$, this becomes impractical. And in higher dimensions, the situation is dramatically worse (we will see why in Section 6).

Two practical complications make things harder still. First, $L$ is typically unknown and hard to estimate, yet algorithms need it. Second, a method that works well for one class of functions may fail on another: the *no free lunch theorem* [4] says that, averaged over all possible objectives, every algorithm performs equally badly. These limitations motivate the gradient-based methods of the next sections, which exploit differentiability to achieve rates like $\mathcal{O}(\log(1/\varepsilon))$.

### 3.2 Local Optimality

Even if we stumble onto $x^*$, how do we *recognize* it? Verifying global optimality requires knowing $f^*$, which is the very thing we are trying to compute. A weaker but tractable condition: local optimality.

**Definition 3.5 (Local minimum).** A point $x^*$ is a *local minimum* of $f$ if there exists $\varepsilon > 0$ such that $f(x^*) \leq f(x)$ for all $x \in [x^* - \varepsilon,\, x^* + \varepsilon]$. A *strict* local minimum additionally requires $f(x^*) < f(z)$ for all $z$ in that neighborhood other than $x^*$ itself.

**Definition 3.6 (Strictly unimodal function).** A function is *strictly unimodal* on an interval $[x_-, x_+]$ if it has a unique minimum $x_*$ and is strictly decreasing on $[x_-, x_*]$ and strictly increasing on $[x_*, x_+]$.

Most functions are not unimodal globally. But near any local minimum $x^*$, a typical $C^1$ function *is* unimodal within some neighborhood: the *attraction basin* of $x^*$. Every local optimum has its own basin, and once we land inside one, finding the minimum is comparatively easy. The trouble is that all basins "look the same" from the inside: a local method cannot tell whether the local minimum it finds is the global one. Finding the right basin is an entirely different (and much harder) problem, which we defer to Section 3.8.

For now, we focus on finding *some* local minimum efficiently. If $f$ is strictly unimodal on $[x_-, x_+]$, placing two test points $x'_-$ and $x'_+$ inside the interval tells us which half contains the minimum: if $f(x'_-) \geq f(x'_+)$, then $x^* \in [x'_-, x_+]$; if $f(x'_-) \leq f(x'_+)$, then $x^* \in [x_-, x'_+]$ [5]. By iterating, we shrink the interval to any desired width $\delta$. The question is: where should we place $x'_-$ and $x'_+$?

### 3.3 Golden Ratio Search

The strategy is to optimize worst-case behavior: shrink the interval as quickly as possible, regardless of which half gets discarded. Each iteration drops either $[x_-, x'_-]$ or $[x'_+, x_+]$; since we don't know which, these two segments should be equal. This means choosing a ratio $r \in (1/2, 1)$ and placing $x'_- = x_- + (1-r)D$ and $x'_+ = x_- + rD$.

Smaller $r$ gives faster shrinkage, but there is a catch: if $r$ is close to $1/2$, the two test points nearly coincide, and in the next iteration both must be recomputed from scratch. To save one

evaluation per step, we need the surviving test point to become one of the two new test points. This imposes $r : 1 = (1 - r) : r$, i.e., $r^2 = 1 - r$, giving

$$r = \frac{\sqrt{5} - 1}{2} \approx 0.618, \tag{3.4}$$

the reciprocal of the golden ratio $g = (1 + \sqrt{5})/2 \approx 1.618$.

**Algorithm 1** Golden Ratio Search (GRS)

**Require:** Function $f$, interval $[x_-, x_+]$, precision $\delta$
**Ensure:** Approximate minimizer of $f$ in $[x_-, x_+]$

```
procedure GRS(f, x_-, x_+, δ)
    (x'_-, x'_+) ← (x_- + (1 − r)(x_+ − x_-), x_- + r(x_+ − x_-))
    while x_+ − x_- > δ do
        if f(x'_-) > f(x'_+) then
            (x_-, x'_-, x'_+) ← (x'_-, x'_+, x_- + r(x_+ − x_-))
        else
            (x_+, x'_+, x'_-) ← (x'_+, x'_-, x_- + (1 − r)(x_+ − x_-))
        end if
    end while
end procedure
```

After $k$ iterations the interval has width $Dr^k$, so the method stops when $k \approx 4.78 \log(D/\delta)$: exponentially fast in $\delta$, a huge improvement over the $\mathcal{O}(1/\varepsilon)$ of Lipschitz sampling. With naive bisection ($r = 0.5$) and two evaluations per step, the constant would be $\approx 6.64$ instead of 4.78. Golden ratio search is asymptotically optimal among methods using only function values [5]; a slightly better finite-$k$ variant uses Fibonacci numbers.

Note that $\delta$ and $\varepsilon$ are different: $\delta$ controls the interval width, not the gap $A(x)$. If $f$ is $L$-Lipschitz, then $A(x^k) \le L\delta$, so reaching $A \le \varepsilon$ requires $k \approx 4.78 \log(LD/\varepsilon)$.

### 3.4 Derivatives and First-Order Models

Golden ratio search uses only function values. Can we do better if we also know *in which direction* $f$ is decreasing? For a linear function $f(x) = bx$, the sign of $b$ immediately tells us: left if $b > 0$, right if $b < 0$. For a general nonlinear $f$, the derivative plays the same role locally.

**Definition 3.7 (Derivative and first-order model).** The *derivative* of $f$ at $x$ is $f'(x) = \lim_{t \to 0}[f(x + t) - f(x)]/t$, provided the limit exists and is finite. When it does, the *first-order model* $L_x(z) = f(x) + f'(x)(z - x)$ is the best linear approximation of $f$ near $x$. The sign of $f'(x)$ tells the local direction of decrease: $f'(x) < 0$ means $f$ is locally decreasing, $f'(x) > 0$ means increasing.

If the left limit $f'_-(x) = \lim_{t \to 0^-}$ and right limit $f'_+(x) = \lim_{t \to 0^+}$ both exist but differ, $f$ is not differentiable at $x$. The standard example: $f(x) = |x| = \max\{x, -x\}$, where $f'_-(0) = -1$ and $f'_+(0) = +1$. Nondifferentiable functions arise naturally in practice ($\ell_1$-regularization, hinge loss); we deal with them in Section 7.

Differentiability implies continuity (but not vice versa). Derivatives of standard functions are known: $[x^k]' = kx^{k-1}$, $[e^x]' = e^x$, $[\ln x]' = 1/x$, $[\sin x]' = \cos x$, $[\cos x]' = -\sin x$. The standard rules preserve differentiability: $[\alpha f + \beta g]' = \alpha f' + \beta g'$ (linearity), $[f \cdot g]' = f'g + fg'$ (product), $[f/g]' = (f'g - fg')/g^2$ (quotient), $[f(g(\cdot))]' = f'(g(\cdot)) \cdot g'$ (chain rule). Two important operations that do *not* preserve differentiability: $\max\{f, g\}$ and $\min\{f, g\}$. In practice, automatic differentiation [2] computes derivatives efficiently, so computing them is not our bottleneck.

#### Regularity classes

We write $f \in C^k$ when $f$ has $k$ continuous derivatives. The hierarchy matters for optimization: $f \in C^1$ means $f'$ is continuous, which implies $f$ is Lipschitz on every bounded interval (by the extreme value theorem applied to $|f'|$). $f \in C^2$ means $f''$ exists and is continuous, giving us curvature information. The best case for univariate optimization is $f \in C^3$: both $f$ and $f'$ are Lipschitz, and we get quadratic convergence from Newton's method.

#### Local optimality and derivatives

The connection is immediate. If $f'(x) \neq 0$, then $f$ is either increasing or decreasing at $x$, so $x$ cannot be a local minimum. Therefore $f'(x^*) = 0$ at every local minimum, a *necessary* condition. But $f'(x) = 0$ also holds at maxima and saddle points (inflection points with zero slope). To distinguish them, we need the second derivative: $f''(x^*) > 0$ confirms a local minimum, $f''(x^*) < 0$ a local maximum, and $f''(x^*) = 0$ is inconclusive. Finding local minima therefore reduces to finding roots of $f'$, and checking the sign of $f''$.

### 3.5 Dichotomic Search

When derivative information is available, we can find a root of $f'$ by bisection on its sign. The intermediate value theorem guarantees: if $f' \in C^0$ with $f'(x_-) < 0$ and $f'(x_+) > 0$, then $\exists\, x \in [x_-, x_+]$ with $f'(x) = 0$.

**Algorithm 2** Dichotomic Search (DS)

```
1:  procedure DS(f, x_-, x_+, ε)
2:      repeat
3:          x ← (x_- + x_+)/2
4:          if |f'(x)| ≤ ε then return x
5:          end if
6:          if f'(x) < 0 then
7:              x_- ← x                          ▷ Minimum is to the right
8:          else
9:              x_+ ← x                          ▷ Minimum is to the left
10:         end if
11:     until x_+ − x_- ≤ δ
12: end procedure
```

Each step halves the interval, giving linear convergence with $r = 0.5$. The complexity is $k \approx 3.32 \log(D/\delta)$: fewer iterations than golden ratio search (4.78), though each iteration now

requires a derivative evaluation instead of a function evaluation. If $f'$ is $L$-Lipschitz (i.e., $f$ is $L$-smooth), the stopping criterion $|f'(x)| \le \varepsilon$ relates to the interval width via $k \approx 3.32 \log(LD/(2\varepsilon))$.

### Finding the initial interval

What if we don't start with $f'(x_-) < 0 < f'(x_+)$? A simple strategy: *exponential expansion*. Starting from a point with $f'(x_+) < 0$, double the step until the sign flips.

**Algorithm 3** Exponential Interval Expansion

```
Δx ← 1
while f'(x_+) ≤ −ε do
    x_+ ← x_+ + Δx
    Δx ← 2 Δx                          ▷ Double the step
end while
```

If $f$ is unbounded below, the expansion never terminates, a useful diagnostic. If $f$ is coercive ($f(x) \to \infty$ as $|x| \to \infty$), it always finds a valid bracket.

## 3.6 Quadratic Interpolation

Plain bisection picks the midpoint, ignoring the *magnitude* of $f'$ at the endpoints. A better guess: fit a quadratic model $ax^2 + bx + c$ to the derivative information at $x_-$ and $x_+$, and use its minimum as the next iterate.

Matching $f'(x_-)$ and $f'(x_+)$ gives

$$a = \frac{f'(x_+) - f'(x_-)}{2(x_+ - x_-)}, \qquad b = \frac{x_+ f'(x_-) - x_- f'(x_+)}{x_+ - x_-}. \tag{3.5}$$

Setting $2ax + b = 0$ yields the interpolated estimate (the "secant formula"):

$$x = \frac{x_- f'(x_+) - x_+ f'(x_-)}{f'(x_+) - f'(x_-)}. \tag{3.6}$$

This always falls between $x_-$ and $x_+$, but it can land dangerously close to an endpoint when one derivative is much larger than the other: a case where the quadratic model is "very skewed."

The general lesson: *never blindly trust a model*. A safeguard clamps the estimate away from the boundaries:

$$x \leftarrow \max\{x_- + \sigma(x_+ - x_-),\ \min(x_+ - \sigma(x_+ - x_-),\ x)\}, \qquad \sigma \in (0, 0.5]. \tag{3.7}$$

In the worst case (model completely wrong), this degrades to linear convergence with $r = 1 - \sigma$. When the model is good, convergence is superlinear.

**Theorem 3.8 (Superlinear convergence of quadratic interpolation).** *If $f \in C^3$ with $f'(x^*) = 0$ and $f''(x^*) \neq 0$, then safeguarded quadratic interpolation converges with order $p = (1 + \sqrt{5})/2 \approx 1.618$ (the golden ratio again) provided the initial guess is close enough to $x^*$ [6].*

Cubic interpolation, which uses all four pieces of information ($f$ and $f'$ at both endpoints), achieves quadratic convergence ($p = 2$) under the same hypotheses [6, 2]. It is more complex to implement but often worthwhile in practice.

## 3.7 Newton's Method

Newton's method pushes the approximation idea to its limit: instead of modelling $f$ linearly, it uses a *quadratic* model, exploiting the second derivative. Two equivalent views lead to the same update.

**View 1:** build a first-order model of $f'$ at $x^k$ and solve for its root.

$$L_k'(x) = f'(x^k) + f''(x^k)(x - x^k) = 0 \qquad \Longrightarrow \qquad x^{k+1} = x^k - \frac{f'(x^k)}{f''(x^k)}. \tag{3.8}$$

**View 2:** build the second-order Taylor model of $f$ at $x^k$ and minimize it.

$$Q_k(x) = f(x^k) + f'(x^k)(x - x^k) + \frac{1}{2} f''(x^k)(x - x^k)^2. \tag{3.9}$$

Setting $Q_k'(x) = 0$ yields the same update.

**Algorithm 4** Newton's Method (NM)

```
procedure NM(f, x, ε)
    while |f'(x)| > ε do
        x ← x − f'(x)/f''(x)
    end while
end procedure
```

**Theorem 3.9 (Quadratic convergence of Newton's method).** *If $f \in C^3$ near $x^*$ with $f'(x^*) = 0$ and $f''(x^*) \neq 0$, and $x^0$ is sufficiently close to $x^*$, then Newton's method converges quadratically. By Taylor's theorem, there exists $\xi$ between $x^k$ and $x^*$ such that*

$$x^{k+1} - x^* = \frac{f'''(\xi)}{2\, f''(x^k)} (x^k - x^*)^2. \tag{3.10}$$

*Quadratic convergence means the number of correct digits roughly doubles at each step: spectacular speed, but only when the initial guess is good enough.*

*Proof.* Write $x^{k+1} - x^* = (x^k - x^*) - f'(x^k)/f''(x^k)$. Substituting $f'(x^*) = 0$ and applying Taylor's formula to $f'$ around $x^k$: $f'(x^*) - f'(x^k) - f''(x^k)(x^* - x^k) = \frac{1}{2} f'''(\xi)(x^* - x^k)^2$ for some $\xi$ between $x^k$ and $x^*$. Dividing by $f''(x^k)$ gives the result. Since $f \in C^3$ and $f''(x^*) \neq 0$, there exists a neighborhood where $|f''(x)| \geq k_2 > 0$ and $|f'''(x)| \leq k_1 < \infty$. If $k_1 |x^k - x^*|/(2k_2) < 1$, then $|x^{k+1} - x^*| < |x^k - x^*|$, and convergence follows by induction. □

Newton's method converges fast only locally. Without a good starting point, it can diverge, oscillate, or converge to a maximum. Global convergence requires safeguards (line search, trust region): topics we develop in the multivariate setting (Section 6). And $f''(x^k) = 0$ is trouble: the method is undefined, signaling a flat second-order model.

## 3.8 Global Optimization

Local methods find local minima efficiently. But what if the problem has many local minima and we need the best one? Unless the objective has special structure, this is a fundamentally harder question.

### Convexity: when local equals global

**Theorem 3.10 (Convexity and global optimality).** *If $f$ is convex (meaning $f''(x) \geq 0$ everywhere, or more generally $f(\alpha x + (1-\alpha)z) \leq \alpha f(x) + (1-\alpha)f(z)$ for all $x, z$ and $\alpha \in [0,1]$) then every local minimum is automatically a global minimum. Convexity ensures that $f'$ is nondecreasing, ruling out multiple stationary points (except along a flat segment).*

Convexity is the single most important structural assumption in optimization. When it holds, local methods are already global: we get the best of both worlds. Many models are purposely constructed to be convex (SVM, ridge regression) precisely so that optimization is tractable. "If you have the choice, choose convex" [7]. We study convexity thoroughly in Section 4.

### Spatial branch-and-bound

When convexity doesn't hold and the global optimum is genuinely needed, the standard approach is *spatial branch-and-bound*. The idea: construct a convex lower approximation $\underline{f}(x) \leq f(x)$ on a bounded domain $X = [x_-, x_+]$, and solve $\min_x \underline{f}(x)$ to get a lower bound on $f^*$.

When the gap between the lower bound $\underline{f}(\bar{x})$ and the best known function value is too large, the domain is split into subregions. Each subregion gets its own convex relaxation. Subregions whose lower bound exceeds the best known value are *pruned*: they cannot contain the global minimum.

The method's efficiency hinges on the quality of $\underline{f}$. A tight relaxation means aggressive pruning and fast convergence; a loose one means exploring many subregions. Worst-case complexity is exponential (keep dicing the domain until the pieces are tiny) but practical performance depends on the degree of nonconvexity and the quality of the relaxation.

For Mixed-Integer Linear Programs, the branch-and-bound framework is especially effective: everything is "trivial" once the integer variables are fixed or relaxed. Mixed-Integer Nonlinear Convex Programs are harder but still tractable. General nonlinear nonconvex problems require additional machinery (function transformations, convexification techniques) typically implemented in specialized solvers. These methods are immensely more efficient than blind search, yet immensely less efficient than local optimization.

## 3.9 Summary: Regularity and Speed

The univariate methods form a hierarchy, each trading a stronger assumption for a faster rate:

These ideas (models, line search, trust regions, regularity-dependent rates) will reappear in every multivariate method. But first, we need the multivariate calculus: gradients, Hessians, and the optimality conditions that generalize $f'(x^*) = 0$ and $f''(x^*) \geq 0$ to $\mathbb{R}^n$.

# 4 Unconstrained Optimality and Convexity

Moving from one dimension to many changes everything. Directions multiply, the landscape acquires curvature in many independent ways, and the derivative (a scalar in $\mathbb{R}$) becomes a vector

| Method | Information used | Assumption | Complexity |
|---|---|---|---|
| Uniform sampling | $f(x)$ only | $L$-Lipschitz | $\mathcal{O}(LD/\varepsilon)$ |
| Golden ratio | $f(x)$ only | unimodal | $\mathcal{O}(\log(D/\delta))$ |
| Dichotomic | $f(x), f'(x)$ | $f \in C^1$, unimodal | $\mathcal{O}(\log(D/\delta))$ (faster constant) |
| Quad. interpolation | $f'(x_\pm)$ | $f \in C^3$ | superlinear ($p \approx 1.618$) |
| Newton | $f'(x), f''(x)$ | $f \in C^3$ | quadratic ($p = 2$) |

**Table 3.1.** Univariate optimization methods: assumptions vs. speed.

in $\mathbb{R}^n$. This section builds the calculus we need: gradients, Hessians, optimality conditions, and the hierarchy of convexity notions that determine how hard a problem is to solve.

The practical motivation is concrete. An *artificial neural network* (ANN) is an $L$-layered directed graph: layer $l = 1$ is the input ($n(1) = h$ neurons), layer $l = L$ is the output ($n(L) = k$ neurons), and layers $2, \dots, L-1$ are hidden. Each neuron $i$ in layer $l+1$ computes $o_i^{l+1}(W^{l+1}; s) = \sigma_i^{l+1}(\sum_{j \in N(l)} w_{ji}^l \, o_j^l(W^l; s) + w_i^l)$, where $w_{ji}^l$ are the arc weights and $\sigma_i^l(\cdot)$ is the *activation function*. Common choices (with their derivatives): identity $\sigma(z) = z$ ($\sigma' = 1$), sigmoid $\sigma(z) = 1/(1+e^{-z})$ ($\sigma' = \sigma(1-\sigma)$), hyperbolic tangent ($\sigma' = 1-\sigma^2$), and ReLU $\sigma(z) = \max\{z, 0\}$ ($\sigma' = 1$ if $z > 0$, 0 otherwise, nondifferentiable at $z = 0$).

The total parameter count is $n = n(2)(n(1)+1) + n(3)(n(2)+1) + \cdots + n(L)(n(L-1)+1)$: easily in the millions or billions. Training amounts to minimizing a loss $f(w) = \sum_{s \in S} \mathscr{L}(y^s, o^L(w; s))$, where $\mathscr{L}$ is typically the MSE $\mathscr{L}(z) = \frac{1}{2}\|z\|^2$ or the cross-entropy. The objective is highly nonlinear, very-large-scale, and nonconvex; even computing $f(w)$ is expensive when $|S|$ is large. Regularization ($\lambda\|w\|^2$ or $\lambda\|w\|_1$) is standard, and the hyperparameter $\lambda > 0$ is fixed via grid search: itself an exponential problem (Section 4.1).

One practical observation matters most: global optima are *not* required (and may even be harmful, overfitting). Local optima are typically of very good quality because the loss landscape is "not adversarial": gradient-based methods find good solutions reliably. This is the setting where the theory of this section pays off.

### 4.1 The Curse of Dimensionality

In one dimension, a Lipschitz function on $[x_-, x_+]$ can be $\varepsilon$-optimized in $\mathcal{O}(L \cdot D/\varepsilon)$ evaluations (Result 3.4). What happens in $\mathbb{R}^n$? A uniform grid with spacing $2\varepsilon/L$ in each coordinate yields $(L \cdot D/\varepsilon)^n$ grid points: exponential in the dimension.

**Theorem 4.1 (Multivariate Lipschitz lower bound).** *For the class of $L$-Lipschitz functions on a box of diameter $D$ in $\mathbb{R}^n$, no algorithm can guarantee an $\varepsilon$-optimal solution in fewer than $\Omega((L \cdot D/\varepsilon)^n)$ evaluations [1].*

This is the *curse of dimensionality*: for $n = 3$ or 5, a grid is feasible (and is, in fact, the standard approach to hyperparameter optimization); for $n = 100$, it is absurd. If $f$ is analytic, clever spatial branch-and-bound (Section 3.8) can find the global optimum; if $f$ is a black box (no

derivatives), effective heuristics exist but offer no optimality guarantee [4]. In all cases, complexity grows "fast" with $n$.

Even when iterative methods break the exponential barrier, they are not free of $n$-dependence. Most convergence results are *dimension-independent* (the iteration count doesn't explicitly depend on $n$), but the cost per iteration does: computing $f$ or $\nabla f$ takes at least $\mathscr{O}(n)$, and for many problems $\mathscr{O}(n^2)$. For $n \approx 10^9$, even $\mathscr{O}(n^2)$ is prohibitive; some hidden dependency on $n$ may also lurk in the $\mathscr{O}(\cdot)$ constants.

The way out: exploit structure. If $f$ is smooth, gradient-based methods break the exponential barrier entirely. If $f$ is convex, local equals global, and the problem becomes tractable. Both properties require the multivariate calculus we develop next.

## 4.2 Neighborhoods and Convergence

**Definition 4.2 (Convergence of a sequence).** A sequence $\{x_i\}$ in $\mathbb{R}^n$ converges to $x$ if

$$\forall\, \varepsilon > 0,\ \exists\, h : \|x_i - x\| \le \varepsilon\ \ \forall\, i \ge h, \tag{4.1}$$

equivalently $x_i \in \mathscr{B}(x, \varepsilon)$ for all large $i$, equivalently $\lim_{i\to\infty} \|x_i - x\| = 0$. We write $\{x_i\} \to x$.

The ball $\mathscr{B}(x, r)$ from Result 2.3 replaces the interval $[x - r, x + r]$ as the basic neighborhood. Convergence in $\mathbb{R}^n$ reduces to convergence of each component, but the norm-based definition is more natural for optimization: it measures distance from the target without privileging any coordinate.

## 4.3 Gradients, Jacobians, and Hessians

In one dimension, the derivative $f'(x)$ tells us the direction of steepest increase. In $\mathbb{R}^n$, the same role is played by the gradient, but now we must also handle vector-valued functions and second-order information.

**Definition 4.3 (Directional derivative).** For $f\colon \mathbb{R}^n \to \mathbb{R}$, the *directional derivative* at $x$ along $d \in \mathbb{R}^n$ is

$$\frac{\partial f}{\partial d}(x) = \lim_{t\to 0} \frac{f(x + td) - f(x)}{t} = \varphi'_{x,d}(0), \tag{4.2}$$

where $\varphi_{x,d}$ is the tomography from Result 2.5. The directional derivative measures the rate of change of $f$ along $d$, evaluated at $\alpha = 0$.

**Definition 4.4 (Gradient).** The *gradient* of $f\colon \mathbb{R}^n \to \mathbb{R}$ at $x$ is the vector of partial derivatives:

$$\nabla f(x) = \left[\frac{\partial f}{\partial x_1},\ \frac{\partial f}{\partial x_2},\ \ldots,\ \frac{\partial f}{\partial x_n}\right]^T \in \mathbb{R}^n. \tag{4.3}$$

It points in the direction of steepest increase of $f$, and its norm equals the maximum directional derivative: $\max_{\|d\|=1} \frac{\partial f}{\partial d}(x) = \|\nabla f(x)\|$, attained at $d = \nabla f(x)/\|\nabla f(x)\|$.

The connection to tomographies is direct. If $f$ is differentiable, the directional derivative along any $d$ is $\frac{\partial f}{\partial d}(x) = \langle \nabla f(x), d\rangle$. When $d = -\nabla f(x)$, this equals $-\|\nabla f(x)\|^2 < 0$: the negative

gradient is always a descent direction. This observation is the engine of every gradient-based algorithm.

**Example 4.5 (Gradient of a quadratic).** For $f(x) = \frac{1}{2}x^T Q x + q^T x$ we get $\nabla f(x) = Qx + q$. The gradient is affine in $x$: this is what makes quadratic optimization tractable.

**Definition 4.6 (Differentiability).** A function $f$ is *differentiable* at $x$ if there exists a vector $b \in \mathbb{R}^n$ such that

$$\lim_{\|h\| \to 0} \frac{|f(x+h) - f(x) - \langle b, h \rangle|}{\|h\|} = 0. \tag{4.4}$$

When this holds, $b = \nabla f(x)$ and the first-order model $L_x(z) = f(x) + \langle \nabla f(x), z - x \rangle$ approximates $f$ to first order near $x$. If $f$ is differentiable everywhere with continuous gradient, then $f \in C^1$.

**Theorem 4.7 (Continuity and differentiability).** *Continuous partial derivatives imply differentiability, which implies continuity:*

$$\textit{continuous partials in } \mathscr{B}(x,\delta) \;\Rightarrow\; f \textit{ differentiable at } x \;\Rightarrow\; f \textit{ continuous at } x. \tag{4.5}$$

*Neither implication reverses in general.*

**Definition 4.8 (Jacobian).** For a vector-valued function $f \colon \mathbb{R}^n \to \mathbb{R}^m$, the *Jacobian* at $x$ is the $m \times n$ matrix of first partial derivatives:

$$J_f(x) = \begin{bmatrix} \partial f_1/\partial x_1 & \cdots & \partial f_1/\partial x_n \\ \vdots & \ddots & \vdots \\ \partial f_m/\partial x_1 & \cdots & \partial f_m/\partial x_n \end{bmatrix}. \tag{4.6}$$

When $m = 1$, the Jacobian reduces to the gradient (as a row vector).

**Definition 4.9 (Hessian).** For $f \colon \mathbb{R}^n \to \mathbb{R}$ with $f \in C^2$, the *Hessian* is the $n \times n$ matrix of second partial derivatives:

$$\nabla^2 f(x) = J_{\nabla f}(x), \qquad [\nabla^2 f(x)]_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}. \tag{4.7}$$

The Hessian is symmetric whenever the second partial derivatives are continuous.

**Example 4.10 (Hessians of simple functions).** For a linear function $f(x) = \langle b, x \rangle$: $\nabla^2 f(x) = 0$. For a quadratic $f(x) = \frac{1}{2}x^T Q x + q^T x$: $\nabla^2 f(x) = Q$ (constant).

**Remark 4.11 (Regularity classes).** $f \in C^2$ means the Hessian is symmetric and continuous everywhere, so the gradient is $C^1$. While $C^2$ (or $C^3$) regularity is convenient for theory, many practical objectives have lower regularity: piecewise smooth functions, for instance. Section 7 addresses such cases.

## 4.4 Optimality Conditions

These are the multivariate analogues of $f'(x^*) = 0$ and $f''(x^*) \geq 0$. The logic is the same as in one dimension: the gradient condition eliminates non-stationary points, and the Hessian condition distinguishes minima from maxima and saddle points.

**Theorem 4.12 (Optimality conditions).** *Let $f\colon \mathbb{R}^n \to \mathbb{R}$ be sufficiently smooth.*

- *First-order necessary: if $x$ is a local minimum, then $\nabla f(x) = 0$.*
- *Second-order necessary: if $\nabla f(x) = 0$ and $x$ is a local minimum, then $\nabla^2 f(x) \succeq 0$.*
- *Second-order sufficient: if $\nabla f(x) = 0$ and $\nabla^2 f(x) \succ 0$, then $x$ is a strict local minimum.*

The gap between necessary and sufficient sits at $\nabla^2 f(x) \succeq 0$ but $\nabla^2 f(x) \not\succ 0$, i.e., the Hessian is positive semidefinite with at least one zero eigenvalue. Higher-order information is needed to decide.

Two practical consequences. First, local optimization reduces (approximately) to solving the nonlinear system $\nabla f(x) = 0$: a nontrivial task when $f$ is not quadratic. Second, when $f$ is "simple" (quadratic), $\nabla f(x) = 0$ becomes a *linear* system $Qx + q = 0$: quadratic models are going to be useful as building blocks. But stationary points are not always local minima (they can be maxima or saddle points) so the Hessian check matters.

## 4.5 Convexity

The optimality conditions above are necessary but not sufficient (except in the strict case). Convexity fills this gap: for convex functions, every local minimum is automatically global, and $\nabla f(x) = 0$ is both necessary and sufficient for global optimality.

**Definition 4.13 (Convex function).** A function $f$ is *convex* if, for all $x, z \in \mathbb{R}^n$ and $\alpha \in [0, 1]$,

$$f(\alpha x + (1 - \alpha)z) \leq \alpha f(x) + (1 - \alpha) f(z). \tag{4.8}$$

The function $-f$ is convex iff $f$ is concave. A function that is both convex and concave is affine (linear plus a constant).

Convexity does not require differentiability ($\|\cdot\|_1$ is convex but not $C^1$). When derivatives exist, equivalent characterizations emerge at each regularity level:

- $f \in C^1$ convex $\Leftrightarrow$ $\nabla f$ is monotone: $\langle \nabla f(z) - \nabla f(x), z - x \rangle \geq 0$ for all $x, z$.
- $f \in C^1$ convex $\Leftrightarrow$ $f(z) \geq f(x) + \langle \nabla f(x), z - x \rangle$ for all $z$. Geometrically: the first-order model is a global underestimator, and the epigraph of $L_x$ contains the epigraph of $f$. A direct consequence: $\nabla f(x) = 0 \Leftrightarrow x$ is a global minimum.
- $f \in C^2$ convex $\Leftrightarrow$ $\nabla^2 f(x) \succeq 0$ for all $x$. For quadratics, $f$ convex iff $Q \succeq 0$.
- $f \in C^2$ with $\nabla^2 f(x) \succeq \tau I$ ($\tau > 0$): the best case for optimization.

The first-order characterization is especially powerful. It says that a convex function always lies above its tangent planes. When $\nabla f(x) = 0$, the tangent plane is horizontal, so $f(z) \geq f(x)$ for all $z$: global optimality from a purely local condition.

Sometimes the best way to prove a function is convex is *by construction*: build it from known convex pieces using operations that preserve convexity.

**Convexity-preserving operations**

1. *Non-negative combination:* $f, g$ convex, $\delta, \beta \geq 0 \Rightarrow \delta f + \beta g$ convex.
2. *Pointwise supremum:* $\{f_i\}_{i \in I}$ convex (possibly infinitely many) $\Rightarrow \sup_i f_i$ convex.
3. *Affine substitution:* $f$ convex $\Rightarrow f(Ax + b)$ convex for any matrix $A$ and vector $b$.
4. *Composition with increasing convex function:* $f$ convex, $g\colon \mathbb{R} \to \mathbb{R}$ convex and nondecreasing $\Rightarrow g \circ f$ convex.
5. *Infimal convolution:* $f_1, f_2$ convex $\Rightarrow f(x) = \inf\{f_1(x_1) + f_2(x_2) : x_1 + x_2 = x\}$ convex.
6. *Image under linear map:* $g$ convex $\Rightarrow f(x) = \inf\{g(z) : Az = x\}$ convex.
7. *Partial minimization:* $g(x, z)$ convex $\Rightarrow f(x) = \inf_z g(x, z)$ convex.
8. *Perspective:* $f(x)$ convex $\Rightarrow p(x, u) = u\, f(x/u)$ convex for $u > 0$.

## 4.6 Quasi-convexity

In one dimension, unimodality sufficed for local methods to work. The multivariate generalization is quasi-convexity.

**Definition 4.14 (Quasi-convex function).** A function $f$ is *quasi-convex* if for all $x, z$ and $\alpha \in [0, 1]$,

$$f(\alpha x + (1 - \alpha)z) \leq \max\{f(x),\, f(z)\}. \tag{4.9}$$

Equivalently, all sublevel sets $S(f, \ell) = \{x : f(x) \leq \ell\}$ are convex. In one dimension, quasi-convexity is the same as unimodality.

Every convex function is quasi-convex (the sublevel sets of a convex function are convex), but not vice versa. The key consequence is the same: a quasi-convex function cannot have local minima that are not global.

The algebra of quasi-convex functions is weaker than convex. Positive scaling preserves quasi-convexity ($f$ quasi-convex and $\delta \geq 0 \Rightarrow \delta f$ quasi-convex), but the sum of two quasi-convex functions is *not* necessarily quasi-convex, unlike the convex case. There is no "disciplined quasi-convex programming" toolkit analogous to the convex one [7].

And yet, quasi-convexity arises more often than one might expect. Neural network loss landscapes, while nonconvex, often exhibit approximate quasi-convexity: a single basin dominates, and local methods find it reliably.

## 4.7 Strong Convexity and Smoothness

Plain convexity says the function curves upward, but not *how much*. Strong convexity quantifies a minimum curvature; smoothness quantifies a maximum one. Together they bracket the Hessian and determine convergence rates.

**Definition 4.15 (Strong convexity).** A function $f$ is *$\tau$-strongly convex* (with modulus $\tau > 0$) if $f(x) - \frac{\tau}{2}\|x\|^2$ is convex. Equivalent characterizations:

- $C^0$: $\alpha f(x) + (1-\alpha)f(z) \geq f(\alpha x + (1-\alpha)z) + \frac{\tau}{2}\alpha(1-\alpha)\|z-x\|^2$.
- $C^1$: $f(z) \geq f(x) + \langle \nabla f(x), z - x\rangle + \frac{\tau}{2}\|z-x\|^2$.
- $C^2$: $\nabla^2 f(x) \succeq \tau I$ for all $x$.

Strong convexity guarantees a unique global minimum and provides a quadratic lower bound on $f$ around any point. The larger $\tau$, the steeper the "bowl" and the easier the optimization.

**Definition 4.16 ($L$-smoothness).** A differentiable function $f$ is *$L$-smooth* if its gradient is Lipschitz continuous:

$$\|\nabla f(x) - \nabla f(z)\| \leq L\|x - z\| \qquad \forall\, x, z. \tag{4.10}$$

For $f \in C^2$, this is equivalent to $\nabla^2 f(x) \preceq LI$ for all $x$ (and $\nabla^2 f(x) \succeq -LI$ in the non-convex case).

$L$-smoothness provides a quadratic upper bound: the function cannot curve too sharply. Algorithmically, it guarantees that a step of size $1/L$ in the negative gradient direction always decreases $f$: the foundation of fixed-stepsize gradient methods.

**Theorem 4.17 (Joint characterization).** *If $f \in C^2$ is both $L$-smooth and $\tau$-strongly convex, the eigenvalues of the Hessian are uniformly bounded:*

$$\tau I \preceq \nabla^2 f(x) \preceq LI \qquad \Longleftrightarrow \qquad 0 < \tau \leq \lambda_n(x) \leq \lambda_1(x) \leq L. \tag{4.11}$$

*The ratio $\kappa = L/\tau$ is the* condition number *of the problem. It plays the same role for general smooth objectives as $\kappa(Q) = \lambda_1/\lambda_n$ does for quadratics: gradient methods converge at a rate that depends on $\kappa$.*

When $\kappa = 1$, the Hessian is a multiple of the identity everywhere: the function is "isotropic" and gradient descent converges in one step. When $\kappa \gg 1$, the function is ill-conditioned: some directions are much steeper than others, and gradient methods zig-zag. The condition number is the single most important quantity governing the speed of first-order optimization.

We now have all the theory needed. The next sections put it to work: Section 5 develops gradient and conjugate gradient methods for quadratic objectives, where the theory is cleanest; Section 6 extends everything to general smooth functions.

## 5 Gradient Methods for Quadratic Functions

Quadratic functions are the simplest nontrivial objectives, yet they already expose the core tension of iterative optimization: how fast can we converge, and what controls the speed? This section answers both questions. We start with steepest descent, analyze its convergence in terms of the spectrum of $Q$, and then build the conjugate gradient method as a principled fix for steepest descent's weaknesses. At the end, we address the direct alternative: factoring $Q$ and solving a linear system.

The practical setting: for moderate $n$ (up to $\approx 10^4$), direct methods costing $\mathcal{O}(n^3)$ are viable. For large $n$ ($10^5$ and beyond), we need iterative methods whose cost per step is $\mathcal{O}(n^2)$, or less, if $Q$ is sparse. We can afford "many" iterations before hitting the $\mathcal{O}(n^3)$ budget.

## 5.1 Iterative Methods

**Definition 5.1 (Iterative procedure).** Starting from an initial point $x^0$, an *iterative procedure* generates a sequence $\{x^k\}$ whose objective values $\{f(x^k)\}$ should converge to $f^*$. Convergence is not always guaranteed: it requires monotonicity or similar structural conditions on the sequence. If $f(x^k) \to -\infty$, the problem is unbounded.

A sequence $\{f^k\}$ may not have a limit at all (oscillation). But any *monotone* sequence has a limit (possibly $\pm\infty$): monotone algorithms are well-behaved in this sense. In practice, we produce a *minimizing sequence* $\{x^k\}$ such that $f(x^k) \to f^*$.

For a quadratic $f(x) = \frac{1}{2}x^TQx + q^Tx$ with $Q$ symmetric, the gradient at iterate $x^k$ is

$$g^k = \nabla f(x^k) = Qx^k + q, \tag{5.1}$$

and the optimality condition is $g^* = 0$.

## 5.2 Steepest Descent

The gradient method moves in the direction $-g^k$ with a stepsize chosen to minimize $f$ along that ray. The key observation: the tomography $\varphi_{x^k,-g^k}(\alpha) = f(x^k - \alpha g^k) - f(x^k)$ is a univariate quadratic in $\alpha$:

$$\varphi(\alpha) = \frac{1}{2}\,\alpha^2 (g^k)^T Q g^k - \alpha \|g^k\|^2. \tag{5.2}$$

Since $(g^k)^TQg^k \geq 0$ (for $Q \succeq 0$) and $-\|g^k\|^2 < 0$ (when $g^k \neq 0$), we have $\varphi(\alpha) < 0$ for small $\alpha > 0$: the negative gradient is indeed a descent direction. The same information that says "you can't stop" simultaneously says "you can improve over there."

Minimizing $\varphi$ gives the exact stepsize $\alpha^k = \|g^k\|^2/((g^k)^TQg^k)$. The variational characterization of eigenvalues (Result 2.9) bounds it: $1/\lambda_1 \leq \alpha^k \leq 1/\lambda_n$.

**Algorithm 5** Steepest Descent for Quadratic Functions (SDQ)

**Require:** $Q \in \mathbb{R}^{n\times n}$, $q \in \mathbb{R}^n$, initial $x \in \mathbb{R}^n$, tolerance $\varepsilon > 0$
**Ensure:** Approximate minimizer of $\frac{1}{2}x^TQx + q^Tx$

```
procedure SDQ(Q, q, x, ε)
    while ‖Qx + q‖ > ε do
        g ← Qx + q
        α ← ‖g‖²/(gᵀQg)                ▷ Exact line search
        x ← x − αg
    end while
end procedure
```

Each iteration costs $\mathcal{O}(n^2)$ (one matrix–vector product dominates). With a small implementation trick, the gradient at the next iterate can be updated as $g^{k+1} = g^k - \alpha^k Q g^k$, reusing the product $Qg^k$ already computed for the stepsize.

The exact stepsize makes consecutive gradients orthogonal: $(g^{k+1})^T g^k = 0$. This is good locally, but causes a global zig-zag pattern: the method keeps revisiting directions it has already explored.

## 5.3 Convergence for $Q \succ 0$

To measure progress, we use the *Q-norm error*:

$$A(x) = \frac{1}{2}(x - x_*)^T Q (x - x_*), \tag{5.3}$$

which equals $f(x) - f(x_*)$ (the absolute gap).

**Theorem 5.2 (Contraction of steepest descent).** *For $Q \succ 0$, steepest descent with exact line search satisfies*

$$A(x^{k+1}) = \left(1 - \frac{\|g^k\|^4}{((g^k)^T Q g^k)((g^k)^T Q^{-1} g^k)}\right) A(x^k). \tag{5.4}$$

The contraction factor depends on how well $g^k$ aligns with the eigenvectors of $Q$. To bound it uniformly, we need the condition number.

**Definition 5.3 (Condition number).** For $Q \succ 0$ with eigenvalues $\lambda_1 \geq \cdots \geq \lambda_n > 0$, the *condition number* is $\kappa = \kappa(Q) = \lambda_1/\lambda_n \geq 1$.

A direct bound on the Rayleigh-quotient ratio gives $r \leq (\kappa - 1)/\kappa < 1$, so $A(x^k) \leq r^k A(x^0) \to 0$ exponentially. The Kantorovich inequality tightens this considerably.

**Theorem 5.4 (Kantorovich inequality).** *For any nonzero $x$ and $Q \succ 0$,*

$$\frac{\|x\|^4}{(x^T Q x)(x^T Q^{-1} x)} \geq \frac{4\,\lambda_1 \lambda_n}{(\lambda_1 + \lambda_n)^2}. \tag{5.5}$$

*This yields the tighter contraction bound*

$$r \leq \left(\frac{\kappa - 1}{\kappa + 1}\right)^2. \tag{5.6}$$

When $\kappa = 1$ (all eigenvalues equal, i.e., $Q \propto I$), $r = 0$: the method converges in one step. When $\kappa \gg 1$, $r \approx 1 - 4/\kappa$: convergence is painfully slow. Example: $\kappa = 1000$ gives $r \approx 0.996$, so reaching $A(x^k) \leq 10^{-6}$ from $A(x^0) = 1$ requires $k \geq 3450$ iterations. That is a lot, but also dimension-independent, valid for $n = 2$ and $n = 10^8$ alike.

## 5.4 Complexity and Convergence Rates

**Definition 5.5 (Complexity).** The *complexity* of an iterative method is the number of iterations $k$ needed to reach accuracy $\varepsilon$. Three common criteria: $\|x^k - x_*\| \leq \varepsilon$ (distance),

$A(x^k) \le \varepsilon$ (absolute gap), or $R(x^k) \le \varepsilon$ (relative gap). These are not interchangeable: $A(x^k) \le \varepsilon$ implies $\|x^k - x_*\| \le \sqrt{\varepsilon/\lambda_n}$, and the converse involves $\lambda_1$.

With a linear contraction rate $r < 1$, the error satisfies $v^k \le r^k v^0 \le \varepsilon$ whenever

$$k \ge \frac{\log(v^0/\varepsilon)}{\log(1/r)}. \tag{5.7}$$

This is $\mathcal{O}(\log(1/\varepsilon))$ (good) but the constant $1/\log(1/r)$ blows up as $r \to 1$. When $r \approx 1$, the denominator $\log(1/r) \approx 1 - r$ is small, giving $k \in \mathcal{O}(r/(1-r) \cdot \log(v^0/\varepsilon))$.

**Definition 5.6 (Rate of convergence).** The *rate* (or *order*) of convergence is defined by

$$\lim_{i \to \infty} \frac{f(x^{i+1}) - f^*}{(f(x^i) - f^*)^p} = r, \tag{5.8}$$

where $p$ is the order and $r$ the rate constant. For $p > 1$, each step gains more accuracy than the previous one.

| Rate | $p$ | $r$ | Complexity |
|---|---|---|---|
| Sublinear | 1 | 1 | $\mathcal{O}(1/\varepsilon)$ or worse |
| Linear | 1 | $< 1$ | $\mathcal{O}(\log(1/\varepsilon))$ |
| Superlinear | $(1, 2)$ | 0 | between linear and quadratic |
| Quadratic | 2 | $> 0$ | $\mathcal{O}(\log\log(1/\varepsilon))$ |

**Table 5.1.** Classification of convergence rates. Quadratic convergence roughly doubles the number of correct digits at each step: effectively $\mathcal{O}(1)$ in practice.

Steepest descent achieves linear convergence at best, with rate governed by $\kappa$.

## 5.5 The Stopping Criterion

The criterion one *wants* is $A(x^k) \le \varepsilon$ or $R(x^k) \le \varepsilon$. But both require $f^*$, which is unknown. The practical proxy is $\|g^k\| \le \varepsilon'$ for some tolerance $\varepsilon'$.

How do $\|g^k\|$ and $A(x^k)$ relate? Since $g^k = Q(x^k - x_*)$, we have $\|g^k\| \le \lambda_1 \|x^k - x_*\|$, but this is the wrong direction (large $\|g\|$ does not imply large error, but small $\|g\|$ does not directly imply small error either). The useful bound goes through the inner product:

$$A(x^k) = \frac{1}{2}\langle x^k - x_*, g^k \rangle \le \frac{1}{2}\|g^k\|\|x^k - x_*\|. \tag{5.9}$$

If we knew an upper bound $\delta \ge \|x^k - x_*\|$ (which we don't), then $\|g^k\| \le 2\varepsilon/\delta$ would guarantee $A(x^k) \le \varepsilon$. If we knew $\lambda_n > 0$, then $\|g^k\| \le \sqrt{2\lambda_n \varepsilon}$ would suffice.

In practice, exact control on the final gap is nontrivial. But for many applications, we don't need it: running until $\|g^k\|$ is "small enough" works well, and the precise $\varepsilon$ mapping is rarely critical.

## 5.6 Hidden Dependence on Dimension and the $\lambda_n = 0$ Case

The condition number is formally dimension-free, but the eigenvalues $\lambda_1, \dots, \lambda_n$ themselves can depend on $n$. If $\kappa$ grows with $n$, so does the iteration count: the "dimension-independence" is illusory.

Worse: $\lambda_n = 0$ means $\kappa = \infty$, and the linear-rate analysis breaks down entirely. Does that mean the method fails? Not necessarily: it just can't be proven convergent in the same way.

**Theorem 5.7 (Sublinear convergence for $\lambda_n = 0$).** *If $Q \succeq 0$ (possibly singular) and $f^*$ is finite, the gradient method with fixed stepsize $\alpha = 1/\lambda_1$ satisfies [8]*

$$f(x^k) - f^* \leq \frac{2\lambda_1 \|x^0 - x_*\|^2}{k-1}. \tag{5.10}$$

This is $\mathcal{O}(1/k)$, i.e., $\mathcal{O}(1/\varepsilon)$ iterations to reach accuracy $\varepsilon$: sublinear convergence. Each extra digit of accuracy costs ten times more iterations. Beyond $10^{-3}$ or $10^{-4}$, this is typically impractical. The bound cannot be improved in general (for the class of convex $L$-smooth functions), and it can be even worse for non-smooth problems, but that is a story for Section 7.

Can we do better than steepest descent when $\lambda_n > 0$? Yes: dramatically.

## 5.7 Conjugate Gradient Method

Steepest descent zig-zags because consecutive gradients are orthogonal but not *globally* independent: the method revisits directions it has already explored. Can we build search directions that avoid this? The conjugate gradient method does exactly this.

### Geometric motivation

If $Q = I$ (perfectly conditioned), steepest descent converges in one step. The idea: change coordinates so that $Q$ *becomes* the identity.

**Theorem 5.8 (Square root decomposition).** *If $Q \succeq 0$, there exists a matrix $R = Q^{1/2}$ such that $Q = R^2$. A symmetric choice is $R = H\sqrt{\Lambda}\,H^T$. If $Q$ is nonsingular, so is $R$.*

Setting $z = Rx$ transforms the quadratic into $h(z) = \frac{1}{2} z^T I\, z + q^T R^{-1} z$: perfect conditioning. In $z$-space, sequential optimization over coordinate directions $u^1, \dots, u^n$ reaches $z_*$ in $n$ steps. Translating back to $x$-space: we need directions that are orthogonal *after* the $R$-transformation.

**Definition 5.9 ($Q$-conjugate directions).** Directions $p^0, \dots, p^{n-1}$ are *$Q$-conjugate* if $(p^i)^T Q\, p^j = 0$ for all $i \neq j$. This is equivalent to $\langle Rp^i, Rp^j \rangle = 0$: orthogonality in $z$-space. Any set of $Q$-conjugate directions is linearly independent.

**Theorem 5.10 (Finite termination).** *If $p^0, \dots, p^{n-1}$ are $Q$-conjugate, the method $x^{i+1} = x^i + \alpha^i p^i$ with exact line search terminates in at most $n$ steps. In floating-point arithmetic, finite termination breaks down, but the early iterates still converge fast.*

How do we generate $Q$-conjugate directions cheaply, without computing the eigenvectors?

**Definition 5.11 (Fletcher–Reeves formula).** The conjugate gradient method builds directions via

$$p^k = -g^k + \beta^k p^{k-1}, \qquad \beta^k = \frac{(g^k)^T Q\, p^{k-1}}{(p^{k-1})^T Q\, p^{k-1}}, \qquad \alpha^k = \frac{-(g^k)^T p^k}{(p^k)^T Q\, p^k}. \tag{5.11}$$

An equivalent (cheaper) formulation for quadratics: $\alpha^k = \|g^k\|^2/((p^k)^T Q p^k)$ and $\beta^k = \|g^k\|^2/\|g^{k-1}\|^2$, which avoids the product $Qp^{k-1}$ in the $\beta$ computation.

**Algorithm 6** Conjugate Gradient for Quadratic Functions (CGQ)

**Require:** $Q \in \mathbb{R}^{n\times n}$, $q \in \mathbb{R}^n$, initial $x \in \mathbb{R}^n$, tolerance $\varepsilon > 0$
**Ensure:** Approximate minimizer of $\frac{1}{2}x^T Q x + q^T x$

```
procedure CGQ(Q, q, x, ε)
    p ← 0
    while ‖Qx + q‖ > ε do
        g ← Qx + q
        if p = 0 then
            p ← −g                          ▷ First direction: steepest descent
        else
            β ← (gᵀQ p)/(pᵀQ p)
            p ← −g + β p                    ▷ Q-conjugate correction
        end if
        α ← −(gᵀp)/(pᵀQ p)
        x ← x + α p
    end while
end procedure
```

After streamlining, each iteration costs about the same as steepest descent: one matrix–vector product plus a few $\mathcal{O}(n)$ operations. The method is worth using only if it converges in substantially fewer iterations, and it does.

Like steepest descent, the conjugate gradient generates gradients satisfying $g^{k+1} \perp g^k$. But unlike steepest descent, $g^k$ is also orthogonal to *all* previous gradients: the zig-zag disappears.

### Convergence rate

**Theorem 5.12 (Convergence of conjugate gradient).** *The conjugate gradient method satisfies*

$$A(x^{k+1}) \le 2\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^k A(x^0). \tag{5.12}$$

*The rate depends on $\sqrt{\kappa}$ instead of $\kappa$: a dramatic improvement over steepest descent.*

For $\kappa = 1000$: steepest descent gives $r \approx 0.996$; conjugate gradient gives $r \approx 0.938$. The difference is enormous over thousands of iterations.

A finer bound exploits the eigenvalue distribution directly.

**Theorem 5.13 (Eigenvalue-dependent bound).** *After $k$ steps,*

$$A(x^{k+1}) \leq \left(\frac{\lambda_k - \lambda_n}{\lambda_k + \lambda_n}\right)^k A(x^0), \tag{5.13}$$

*where $\lambda_k$ is the $k$-th largest eigenvalue. Two consequences:*

- *If $Q$ has only $k$ distinct eigenvalues, the method terminates in $k$ iterations, regardless of $n$.*
- *If $k$ eigenvalues are clustered near a single value, convergence behaves as if the effective dimension were $n - k$.*

*The more clustered the spectrum, the faster the method.*

## 5.8 Preconditioning

If we can't change $Q$, we can at least change how we look at it.

**Definition 5.14 (Preconditioned system).** Given a nonsingular matrix $P$, the substitution $z = Px$ transforms the quadratic into

$$h(z) = \frac{1}{2} z^T (P^{-T} Q\, P^{-1})\, z + q^T P^{-1} z. \tag{5.14}$$

The transformed Hessian is $\tilde{Q} = P^{-T} Q\, P^{-1}$, and convergence depends on $\kappa(\tilde{Q})$.

The ideal preconditioner has $P = Q^{1/2}$, giving $\tilde{Q} = I$ and one-step convergence. But computing $Q^{1/2}$ costs $\mathcal{O}(n^3)$ and defeats the purpose. A good preconditioner approximates $Q$ well enough to reduce $\kappa$ substantially while being cheap to "invert" (i.e., to solve $Pw = z$).

Common choices:

- *Diagonal:* $P = \sqrt{\operatorname{diag}(Q)}$: trivial to invert, but only helps when off-diagonal entries are small. When $Q \not\succ 0$, a heuristic fix replaces zero or negative diagonal entries with 1.
- *Incomplete Cholesky:* a sparse approximation of the full Cholesky factor [2]. More expensive to compute but often dramatically reduces $\kappa$.

The art of preconditioning is problem-specific: the best preconditioner exploits the structure of *your* particular $Q$.

## 5.9 Cholesky Factorization

For moderate-sized problems, solving $Qx + q = 0$ directly is often the best strategy. No iterative guesswork, no convergence rate to worry about: just algebra.

**Definition 5.15 (Cholesky decomposition).** For $Q \succeq 0$, there exists a lower triangular matrix $L$ such that $Q = LL^T$. The entries of $L$ are computed recursively:

$$L_{ii} = \sqrt{Q_{ii} - \sum_{k=1}^{i-1} L_{ik}^2}, \qquad L_{ji} = \frac{Q_{ji} - \sum_{k=1}^{i-1} L_{jk} L_{ik}}{L_{ii}} \quad (j > i). \tag{5.15}$$

The factorization costs $\mathcal{O}(n^3)$. Once $L$ is known, we rewrite $Qx = -q$ as two triangular systems:

$$Lw = -q, \qquad L^T x = w. \tag{5.16}$$

Each triangular solve (*backsolve*) costs $\mathcal{O}(n^2)$: at step $k$, compute $x_k = (b_k - \sum_{i<k} L_{ki} x_i)/L_{kk}$. Only sums and products, plus $n$ divisions (numerically delicate when $L_{kk} \approx 0$).

The factorization also serves as a definiteness test. If $Q_{kk} - \sum_{i<k} L_{ik}^2 = 0$ at some step, the matrix is singular ($Q \not\succ 0$); the factorization produces a lower trapezoidal matrix with a zero diagonal block. If the quantity is negative, $Q \not\succeq 0$: the matrix is indefinite.

Why Cholesky rather than the spectral decomposition $R = H\sqrt{\Lambda}\,H^T$? Both cost $\mathcal{O}(n^3)$, but $L$ is triangular: solving $Lw = z$ is a single forward substitution, whereas using $R$ requires a full matrix–vector product. Cholesky has a smaller constant and better numerical properties for this task.

The choice between iterative (CG) and direct (Cholesky) methods depends on $n$, sparsity, and the number of right-hand sides. For a single solve with dense $Q$ and moderate $n$, Cholesky wins. For very large $n$, sparse $Q$, or when only an approximate solution is needed, CG is the tool of choice.

## 6 Smooth Unconstrained Optimization

The quadratic case gave us clean formulas and exact stepsizes. Real objectives are messier: no finite termination, no closed-form line search, and convergence rates that depend on properties we may not know. This section extends the gradient method to general smooth functions, develops the line search machinery needed to make it work, and then introduces second-order and quasi-Newton methods that converge faster, at a price. The unifying view: each algorithm constructs a *model* of $f$ (first-order, second-order, or data-driven) and uses the model to choose the next iterate. The model is always wrong to some extent, and managing that mismatch is the core challenge.

### 6.1 The Gradient Method

The algorithm is the same as for quadratics, but now the tomography $\varphi_{x^i,d^i}(\alpha) = f(x^i + \alpha d^i)$ is no longer a parabola: its shape depends on $f$.

**Algorithm 7** Gradient Method with Steepest Descent

**Require:** $f$, initial $x$, tolerance $\varepsilon > 0$
**Ensure:** Approximate stationary point of $f$

```
procedure SD(f, x, ε)
    while ‖∇f(x)‖ > ε do
        d ← −∇f(x)
        α ← stepsize(f, x, d)
        x ← x + αd
    end while
end procedure
```

At each iterate $x^i$, the first-order model $L^i(x) = f(x^i) + \langle \nabla f(x^i), x - x^i \rangle$ is a good approximation only in a small ball around $x^i$. The update $x^{i+1} = x^i + \alpha^i d^i$ is meaningful only when $f(x^{i+1}) < f(x^i)$.

The stepsize $\alpha^i$ is the critical design choice. Too large and the method overshoots: $f(x^{i+1}) > f(x^i)$. Too small and the method stagnates, decreasing $f$ only infinitesimally per step. Scylla and Charybdis, and finding a path between them is the subject of the next subsections.

Using the unnormalized gradient $d^i = -\nabla f(x^i)$ (rather than $d^i = -\nabla f(x^i)/\|\nabla f(x^i)\|$) ties the step length to the gradient magnitude: far from the optimum the steps are large, near it they shrink automatically.

## 6.2 Fixed Stepsize

A fixed stepsize $\alpha^i = \bar{\alpha}$ is simple and cheap, but it needs a guarantee that $f$ decreases at every step. Such a guarantee requires controlling how fast $\nabla f$ can change, i.e., $L$-smoothness (Result 4.16).

The tomography along the gradient direction gives $\varphi'_{x,-\nabla f(x)}(0) = -\|\nabla f(x)\|^2 < 0$: the initial descent is always present. $L$-smoothness ensures this descent doesn't vanish too quickly. If $f$ is $L$-smooth, $\varphi$ inherits $[L\|d\|^2]$-smoothness, giving the upper bound

$$\varphi(\alpha) \leq \varphi(0) + \|\nabla f(x)\|^2 \left[\frac{L\alpha^2}{2} - \alpha\right]. \tag{6.1}$$

The worst-case quadratic bound says: the function can curve *up* at rate $L$, but we know the initial slope is $-\|\nabla f(x)\|^2$.

Choosing $\bar{\alpha} = 1/L$ yields the descent bound

$$f(x^{i+1}) - f(x^i) \leq -\frac{\|\nabla f(x^i)\|^2}{2L}. \tag{6.2}$$

The decrease is *additive*, not multiplicative, sublinear convergence:

$$f(x^k) - f^* \leq \frac{2L\|x^0 - x_*\|^2}{k-1} \qquad \Longrightarrow \qquad k \in \mathscr{O}\left(\frac{LD^2}{\varepsilon}\right). \tag{6.3}$$

This matches the quadratic case when $\lambda_n = 0$ (Result 5.7).

The $\mathscr{O}(1/\varepsilon)$ bound is not optimal for the class of $L$-smooth convex functions: $\mathscr{O}(1/\sqrt{\varepsilon})$ is achievable (see Section 6.12). But a lower bound shows that some problems are genuinely hard: there exist $L$-smooth functions for which any first-order algorithm requires $\Omega(1/\sqrt{\varepsilon})$ iterations.

### Linear convergence with strong convexity

If $f$ is also $\tau$-strongly convex, the mean value theorem gives $\nabla f(x) - \nabla f(x_*) = \nabla^2 f(w)(x - x_*)$ for some $w$. Setting $z = x - \alpha \nabla f(x)$:

$$z - x_* = (I - \alpha \nabla^2 f(w))(x - x_*), \qquad \|z - x_*\| \leq \|I - \alpha \nabla^2 f(w)\| \cdot \|x - x_*\|. \tag{6.4}$$

The contraction factor is $r = \|I - \alpha\nabla^2 f(w)\| = \max\{|1-\alpha\lambda_1|, |1-\alpha\lambda_n|\}$. Since $\tau \le \lambda_n \le \lambda_1 \le L$, the optimal choice $\alpha = 2/(L+\tau)$ gives

$$r = \frac{L-\tau}{L+\tau} = \frac{\kappa-1}{\kappa+1} < 1, \qquad \kappa = L/\tau. \tag{6.5}$$

Linear convergence, with rate governed by the condition number: the same story as for quadratics, but now for any $\tau$-strongly convex, $L$-smooth objective.

## 6.3 Inexact Line Search

Exact line search ($\alpha^* \in \arg\min_\alpha \varphi(\alpha)$) is rarely practical for general $f$. We need cheaper alternatives that still guarantee convergence.

### Armijo condition (avoiding Scylla: overshooting)

$$\varphi(\alpha) \le \varphi(0) + m_1 \alpha\, \varphi'(0), \qquad 0 < m_1 \ll 1. \tag{6.6}$$

This ensures *sufficient decrease*: the function drops by at least a fraction $m_1$ of what the linear model predicts. Small $\alpha$ always satisfies Armijo, but tiny steps cause stagnation.

### Wolfe condition (avoiding Charybdis: stagnation)

$$\varphi'(\alpha) \ge m_3\, \varphi'(0), \qquad m_1 < m_3 < 1. \tag{6.7}$$

The derivative at the accepted point should be less negative than at the start: we should have made "enough" progress along the ray. The *strong Wolfe condition* $|\varphi'(\alpha)| \le m_3\, |\varphi'(0)|$ additionally prevents $\varphi'(\alpha) \gg 0$, filtering out some local maxima.

**Theorem 6.1 (Existence of Armijo–Wolfe stepsizes).** *If $\varphi \in C^1$ and $\varphi(\alpha)$ is bounded below for $\alpha \ge 0$, then there exists $\alpha$ satisfying both Armijo and Wolfe conditions [2].*

Armijo plus Wolfe captures all local minima of $\varphi$ (unless $m_1 \approx 1$). Armijo plus strong Wolfe refines this by filtering maxima.

### Backtracking line search

The simplest practical approach: check only Armijo, and shrink $\alpha$ until it passes.

**Algorithm 8** Backtracking Line Search (BLS)

**Require:** $\varphi, \alpha, m_1, \tau$ with $\tau < 1$

```
procedure BLS(φ, α, m1, τ)
    while φ(α) > φ(0) + m1 α φ'(0) do
        α ← τ α
    end while
end procedure
```

Starting from $\alpha = 1$, the procedure generates $\alpha \ge \tau^h$ where $h$ is the number of shrinkage steps needed. If stepsizes stay bounded away from zero throughout the optimization, convergence is ensured.

### Convergence with Armijo–Wolfe

From $L$-smoothness and the Wolfe condition, a lower bound on the accepted stepsize follows: $\alpha' > (1 - m_3)/L$. If $f$ is also $\tau$-convex, convergence is linear with $r \approx 1 - \tau/L$, where the "$\approx$" depends on $m_1$ and $m_3$.

## 6.4 Efficiency for General Functions

**Theorem 6.2 (Local convergence rate).** *Let $f \in C^2$ with $x_*$ a local minimum satisfying $\nabla^2 f(x_*) \succ 0$. The exact-line-search gradient method converges linearly with*

$$r = \left(\frac{\lambda_1 - \lambda_n}{\lambda_1 + \lambda_n}\right)^2, \tag{6.8}$$

*where $\lambda_1, \lambda_n$ are the extreme eigenvalues of $\nabla^2 f(x_*)$.*

These properties need only hold in $\mathscr{B}(x_*, \delta)$. Inexact line search may slow convergence ($r \approx 1 - \lambda_n/\lambda_1$), but reduces function evaluations per iteration, a practical trade-off.

## 6.5 Twisted Gradient Methods and General Descent

The direction $d^i = -\nabla f(x^i)$ is not the only descent direction. Any $d^i$ with $\langle d^i, \nabla f(x^i)\rangle < 0$ works: the entire open half-space opposite to $\nabla f$ consists of descent directions.

**Definition 6.3 (Descent direction).** A direction $d^i$ is a *descent direction* if $\cos(\theta^i) = \langle d^i, \nabla f(x^i)\rangle / (\|d^i\|\|\nabla f(x^i)\|) < 0$.

A "twisted" gradient method uses $d^i = H^i(-\nabla f(x^i))$ where $H^i$ is a positive definite matrix. When $H^i \succ 0$, the direction is guaranteed to be descent. Newton's method sets $H^i = [\nabla^2 f(x^i)]^{-1}$; quasi-Newton methods approximate this.

**Theorem 6.4 (Zoutendijk's theorem).** *If $f \in C^1$ is $L$-smooth with $f^* > -\infty$, and stepsizes satisfy Armijo–Wolfe, then*

$$\sum_{i=1}^{\infty} \cos^2(\theta^i)\, \|\nabla f(x^i)\|^2 < \infty. \tag{6.9}$$

*If $\cos(\theta^i) \geq \bar{\theta} > 0$ for all $i$ (the angles stay bounded away from $90°$), then $\|\nabla f(x^i)\| \to 0$.*

Zoutendijk's theorem is the fundamental convergence guarantee for descent methods. It says: as long as you don't search in directions nearly orthogonal to the gradient, the gradient norm must eventually vanish.

## 6.6 Newton's Method

Newton's method replaces the linear model with a quadratic one, using the Hessian to capture local curvature. The payoff is quadratic convergence, but only locally.

**Definition 6.5 (Newton direction).** If $\nabla^2 f(x^i) \succ 0$, the *Newton direction* is $d^i = -[\nabla^2 f(x^i)]^{-1}\nabla f(x^i)$. It minimizes the second-order model $Q^i(z) = f(x^i) + \langle \nabla f(x^i), z - x^i\rangle + \frac{1}{2}(z-x^i)^T\nabla^2 f(x^i)(z-x^i)$.

When $\nabla^2 f(x^i) \succ 0$, the Newton direction is descent: $\langle \nabla f(x^i), d^i\rangle = -\nabla f(x^i)^T[\nabla^2 f(x^i)]^{-1}\nabla f(x^i) < 0$.

### Geometric interpretation

For a quadratic $f(x) = \frac{1}{2}x^TQx + q^Tx$ with $Q \succ 0$, Newton terminates in one step. The geometric reason: let $Q = RR^T$ and change variables to $z = Rx$. In $z$-space the Hessian is the identity, and steepest descent converges in one step. Newton in $x$-space *is* steepest descent in the space where the Hessian is the identity.

### Convergence

**Theorem 6.6 (Quadratic convergence).** *If $f \in C^3$, $\nabla f(x_*) = 0$, $\nabla^2 f(x_*) \succ 0$, then $\exists\, \delta > 0$ such that starting in $\mathscr{B}(x_*, \delta)$ gives quadratic convergence.*

**Theorem 6.7 (Global convergence with line search).** *If $f \in C^2$ is $L$-smooth and $\tau$-convex, then $\cos(\theta^i) \geq \tau/L$, and Zoutendijk's theorem guarantees global convergence. Near $x_*$, the Armijo condition with $m_1 \leq 1/2$ accepts the full Newton step ($\alpha = 1$) eventually, recovering quadratic convergence.*

### The nonconvex case

When $\nabla^2 f(x^i) \not\succ 0$, the Newton direction may not be descent. The fix: replace $\nabla^2 f(x^i)$ with a positive definite approximation $H^i$ satisfying $\tau I \preceq H^i \preceq LI$. A simple choice: $H^i = \nabla^2 f(x^i) + \epsilon^i I$ with $\epsilon^i = \max\{0, \delta - \lambda_{\min}(\nabla^2 f(x^i))\}$. If $\{x^i\} \to x_*$ with $\nabla^2 f(x_*) \succ 0$, then $\epsilon^i = 0$ eventually, recovering quadratic convergence. The cost is $\mathscr{O}(n^3)$ per iteration (Cholesky factorization), prohibitive for very large $n$.

## 6.7 Trust Region Methods

Trust region methods reverse the line search logic: first choose a radius $r$ (the "trust region"), then find the best direction within it.

**Definition 6.8 (Trust region subproblem).** Given a trust region $\mathscr{B}_2(x^i, r)$: $x^{i+1} \in \arg\min\{Q^i(z) : \|z - x^i\| \leq r\}$.

The KKT conditions for this subproblem give $[\nabla^2 f(x^i) + \lambda^i I]\, d^i = -\nabla f(x^i)$ with $\lambda^i \geq 0$. Three cases: if $\lambda^i > 0$, the constraint is active (similar to Hessian modification with $\epsilon^i = \lambda^i$); if $\|d^i\| < r$, then $\lambda^i = 0$ and we get the plain Newton step; as $\{x^i\} \to x_*$, eventually $\lambda^i = 0$ and quadratic convergence is recovered.

Trust region methods handle nonconvex Hessians naturally: negative-curvature directions are exploited within the trust region rather than suppressed.

## 6.8 Quasi-Newton Methods

Computing and inverting the Hessian costs $\mathcal{O}(n^3)$. Quasi-Newton methods approximate the Hessian (or its inverse) using only gradient differences, achieving superlinear convergence at $\mathcal{O}(n^2)$ per iteration.

**Definition 6.9 (Secant equation).** Define $s^i = x^{i+1} - x^i$ and $y^i = \nabla f(x^{i+1}) - \nabla f(x^i)$. The *secant equation* requires $H^{i+1}s^i = y^i$. For compatibility with $H^{i+1} \succ 0$, the *curvature condition* $\langle s^i, y^i \rangle > 0$ must hold, enforceable via Armijo–Wolfe line search.

Superlinear convergence holds whenever the approximation resembles the true Hessian along the search direction: $\lim_{i\to\infty} \|(H^i - \nabla^2 f(x^i))\, d^i\| / \|d^i\| = 0$.

### DFP formula

The Davidon–Fletcher–Powell update finds the closest positive-definite matrix satisfying the secant equation (in a weighted Frobenius norm sense):

$$H^{i+1} = (I - \rho^i y^i (s^i)^T)\, H^i\, (I - \rho^i s^i (y^i)^T) + \rho^i y^i (y^i)^T, \quad \rho^i = 1/\langle s^i, y^i \rangle. \tag{6.10}$$

### BFGS formula

The Broyden–Fletcher–Goldfarb–Shanno update works on both $H^i$ and its inverse $B^i = (H^i)^{-1}$:

$$\begin{aligned} H^{i+1} &= H^i + \rho^i y^i (y^i)^T - \frac{H^i s^i (s^i)^T H^i}{(s^i)^T H^i s^i}, \\ B^{i+1} &= (I - \rho^i s^i (y^i)^T)\, B^i\, (I - \rho^i y^i (s^i)^T) + \rho^i s^i (s^i)^T. \end{aligned}$$

Both are rank-two updates at $\mathcal{O}(n^2)$ cost. In practice, BFGS generally outperforms DFP. DFP and BFGS are the endpoints of the *Broyden family*: $H^{i+1} = \beta\, H^{i+1}_{\text{DFP}} + (1-\beta)\, H^{i+1}_{\text{BFGS}}$ for $\beta \in [0, 1]$.

### L-BFGS

For very large $n$, even $\mathcal{O}(n^2)$ is too much. L-BFGS stores only the last $k \ll n$ correction pairs $(s^i, y^i)$ and reconstructs the inverse Hessian on the fly. Memory and time per iteration: $\mathcal{O}(kn)$. Large $k$ approximates Newton; small $k$ degrades toward steepest descent. A common base matrix: $B^{i-k} = \gamma^i I$ with $\gamma^i = \langle s^{i-1}, y^{i-1} \rangle / \|y^{i-1}\|^2$.

## 6.9 Deflected Gradient Methods

Twisting ($d^i = H^i(-\nabla f(x^i))$) costs at least $\mathcal{O}(n^2)$. *Deflection* is cheaper, $\mathcal{O}(n)$ by design:

$$d^i = -\nabla f(x^i) + \beta^i d^{i-1}. \tag{6.11}$$

If $\beta^0 = 0$ (starting with steepest descent), the direction aggregates all past gradients. Ensuring descent ($\varphi'_{x^i, d^i}(0) < 0$) is nontrivial, unlike twisting where $H^i \succ 0$ suffices.

## 6.10 Nonlinear Conjugate Gradient

The conjugate gradient idea from Section 5.7 extends to general functions via deflection.

**Algorithm 9** Nonlinear Conjugate Gradient (NCG)

**Require:** $f, x, \varepsilon$

**procedure** NCG($f, x, \varepsilon$)
  $\nabla f^- \leftarrow 0$
  **while** $\|\nabla f(x)\| > \varepsilon$ **do**
    **if** $\nabla f^- = 0$ **then**
      $d \leftarrow -\nabla f(x)$ ▷ First step: steepest descent
    **else**
      $\beta \leftarrow$ chosen formula
      $d \leftarrow -\nabla f(x) + \beta\, d^-$
    **end if**
    $\alpha \leftarrow \mathrm{LS}(f, x, d)$; $x \leftarrow x + \alpha d$
    $d^- \leftarrow d$; $\nabla f^- \leftarrow \nabla f(x)$
  **end while**
**end procedure**

Four standard formulas for $\beta$:

- Fletcher–Reeves: $\beta^i_{\mathrm{FR}} = \|\nabla f(x^i)\|^2 / \|\nabla f(x^{i-1})\|^2$
- Polak–Ribière: $\beta^i_{\mathrm{PR}} = \langle \nabla f(x^i) - \nabla f(x^{i-1}),\, \nabla f(x^i) \rangle / \|\nabla f(x^{i-1})\|^2$
- Hestenes–Stiefel: $\beta^i_{\mathrm{HS}} = \langle \nabla f(x^i) - \nabla f(x^{i-1}),\, \nabla f(x^i) \rangle / \langle \nabla f(x^i) - \nabla f(x^{i-1}),\, d^{i-1} \rangle$
- Dai–Yuan: $\beta^i_{\mathrm{DY}} = \|\nabla f(x^i)\|^2 / \langle \nabla f(x^i) - \nabla f(x^{i-1}),\, d^{i-1} \rangle$

For quadratic $f$ with exact line search, all four reduce to the standard conjugate gradient and terminate in $n$ steps. For general $f$, they differ.

Each formula requires specific line search conditions. FR needs $m_1 < m_3 < 1/2$ (strong Wolfe). PR may produce non-descent directions; the fix is $\beta^i_{\mathrm{PR}+} = \max\{\beta^i_{\mathrm{PR}}, 0\}$. These "$\max\{0, \cdot\}$" modifications reset the direction to steepest descent when deflection goes wrong, a form of *restart*. Without restarts, FR can enter a failure mode: if $\|\nabla f(x^i)\| \ll \|d^i\|$, then $\cos(\theta^i) \approx 0$ and the steps barely move.

In practice, all variants perform similarly, with results that vary significantly across problems. On quadratics, $n$ conjugate gradient steps approximate one Newton step: $\|x^{i+n} - x_*\| \le r\, \|x^i - x_*\|^2$. For large $n$ this can be expensive; the method is powerful but not easy to tune.

## 6.11 Heavy Ball Method

The heavy ball method adds *momentum*: a fraction of the previous step.

$$x^{i+1} = x^i - \alpha^i \nabla f(x^i) + \beta^i (x^i - x^{i-1}). \tag{6.12}$$

The term $\beta^i(x^i - x^{i-1})$ maintains the previous direction, reducing zig-zagging; $-\nabla f(x^i)$ steers toward $x_*$. This is *not* an $f$-descent method ($f(x^{i+1})$ may increase), but with proper $\alpha^i, \beta^i$ it achieves $d$-descent: $\|x^{i+1} - x_*\| \le r\, \|x^i - x_*\|$.

The dynamics follow a two-term recurrence. Using the mean value theorem, the state update can be written as

$$\begin{bmatrix} x^{i+1} - x_* \\ x^i - x_* \end{bmatrix} = C^i \begin{bmatrix} x^i - x_* \\ x^{i-1} - x_* \end{bmatrix}, \qquad C^i = \begin{bmatrix} (1+\beta^i)I - \alpha^i \nabla^2 f(w^i) & -\beta^i I \\ I & 0 \end{bmatrix}. \tag{6.13}$$

The spectral radius $\rho(C^i)$ governs convergence. Minimizing over $\alpha, \beta$ for the class $\tau I \preceq \nabla^2 f \preceq LI$ gives the optimal parameters:

$$\alpha = \frac{4}{(\sqrt{L} + \sqrt{\tau})^2}, \qquad \beta = \left( \frac{\sqrt{L} - \sqrt{\tau}}{\sqrt{L} + \sqrt{\tau}} \right)^2, \qquad r = \frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1}. \tag{6.14}$$

This is the same $\sqrt{\kappa}$-dependence as conjugate gradient: a dramatic improvement over steepest descent's $\kappa$-dependence.

**Definition 6.10 (Gelfand's formula and quasi-linear convergence).** For any matrix $C$, $\rho(C) = \lim_{i\to\infty} \|C^i\|^{1/i}$. This means: for any $\varepsilon > 0$, $\exists\, h$ such that $\|C^i\| \leq (\rho(C) + \varepsilon)^i$ for all $i \geq h$. Convergence is linear if $\rho(C) + \varepsilon < 1$, but the method may not converge in the early iterates: it needs a "warm-up" phase. This is called *quasi-linear convergence*.

For non-convex $f$, the method converges if $\beta \in [0, 1)$ and $\alpha \in (0, 2(1-\beta)/L)$. Pushing $\beta \to 1$ forces $\alpha \to 0$: there is a tension between momentum and step size. If $\tau = 0$, the error decreases as $\mathcal{O}(1/k)$, same as vanilla gradient descent.

## 6.12 Accelerated Gradient Method

Nesterov's accelerated gradient achieves *optimal* worst-case rates among first-order methods.

**Algorithm 10** Accelerated Gradient (ACCG)

**procedure** ACCG($f, x, \varepsilon$)
  $x_- \leftarrow x$; $\gamma \leftarrow 1$
  **repeat**
    $\gamma_- \leftarrow \gamma$; $\gamma \leftarrow (\sqrt{4\gamma^2 + \gamma^4} - \gamma^2)/2$
    $\beta \leftarrow \gamma(\gamma_-^{-1} - 1)$
    $y \leftarrow x + \beta(x - x_-)$
    $g \leftarrow \nabla f(y)$
    $x_- \leftarrow x$; $x \leftarrow y - L^{-1} g$
  **until** $\|g\| \leq \varepsilon$
**end procedure**

For $L$-smooth convex functions, the method achieves $\mathcal{O}(LD^2/k^2)$ error, equivalently, $\mathcal{O}(1/\sqrt{\varepsilon})$ iterations. This matches the lower bound: no first-order method can do better in the worst case. If $f$ is also $\tau$-convex, convergence is linear with $r = (\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1)$.

The method evaluates $\nabla f$ at an *extrapolated* point $y$ rather than $x$: it "looks ahead" before stepping. It is designed to optimize worst-case behavior; in practice it can be slower than simpler methods on well-conditioned problems, but it achieves exactly what it is built for.

# 7 Nonsmooth Unconstrained Optimization

Smooth optimization has clean gradients, reliable descent, and convergence rates driven by curvature. Drop the smoothness assumption and most of that machinery breaks. Yet nonsmooth problems are unavoidable: $\ell_1$ regularization, hinge losses, max-type objectives, all introduce points where derivatives don't exist. This section first explains *why* nonsmoothness arises (fitting problems, regularization, classification), then develops the theory (subgradients, subdifferential calculus) and the algorithms (subgradient methods, smoothing, bundle methods) needed to handle it.

## 7.1 Fitting Problems and Stochastic Gradients

The problems at the heart of machine learning have a specific additive structure worth exploiting.

**Definition 7.1 (Fitting problem).** Given inputs $\mathscr{X} = [x^s \in \mathbb{R}^h]_{s\in S}$ and outputs $y = [y^s \in \mathbb{R}^k]_{s\in S}$ with $S = \{1, \dots, m\}$, define:

- *Predictor:* $\pi(x; w)\colon \mathbb{R}^h \to \mathbb{R}^k$, parametric in $w \in \mathbb{R}^n$.
- *Loss function:* $\mathscr{L}\colon \mathbb{R}^k \times \mathbb{R}^k \to \mathbb{R}$.
- *Objective:* $\min\Big\{f(w) = \sum_{s\in S} f_s(w) = \sum_{s\in S} \mathscr{L}(y^s, \pi(x^s; w)) : w \in \mathbb{R}^n\Big\}$.

The full gradient is $\nabla f(w) = \sum_{s\in S} \nabla f_s(w)$: a sum of $m$ individual gradients.

**Example 7.2 (Linear least squares).** With $\pi(x; w) = \langle x, w\rangle$ and $\mathscr{L}(y, z) = (y - z)^2/2$: $f_s(w) = (y^s - \langle x^s, w\rangle)^2/2$ and $\nabla f_s(w) = -x^s(y^s - \langle x^s, w\rangle)$.

When $m$ is large, computing the full gradient at every step is expensive.

**Definition 7.3 (Incremental gradient).** For a small subset $K \subset S$, $\nabla f^K(w) = \sum_{s\in K} \nabla f_s(w)$. This is much cheaper but is not guaranteed to be a descent direction for $f$.

**Definition 7.4 (Stochastic gradient).** Stochastic gradient is incremental gradient with $K$ chosen at random. Results are typically stated in terms of expectations and Cesàro averages $\bar{x}^k = \frac{1}{k+1}\sum_{j=0}^{k} x^j$. With $|K| = 1$ (single-sample minibatch) and $f$ $\tau$-convex [8]: $\mathbb{E}[f(\bar{x}^k) - f^*] \le \varepsilon$ for $k \ge \mathscr{O}(1/\varepsilon^2)$. Results improve as $|K|$ grows, but so does iteration cost. The stepsize $\alpha^k$ is chosen a priori; only $g^k$ is used, not $f(x^k)$.

First-order methods are "quite robust" to errors in the gradient. An inexact gradient that points roughly in the right direction can still drive convergence: a property that proves decisive when derivatives don't even exist.

## 7.2 Regularization and Feature Selection

A common strategy to control model complexity is to add a regularizer $\Omega(w)$:

$$\min\Big\{\sum_{s\in S} \mathscr{L}(y^s, \pi(x^s; w)) + \mu\,\Omega(w) : w \in \mathbb{R}^n\Big\}. \tag{7.1}$$

Two standard choices:

- *Ridge:* $\Omega(w) = \frac{1}{2}\|w\|_2^2 \in C^\infty$, with $\nabla\Omega(w) = w$. Smooth, strongly convex, well-behaved.
- *Lasso:* $\Omega(w) = \|w\|_1$: the best convex approximation of the sparsity-inducing $\|w\|_0$. Promotes sparsity in practice, but is *not* differentiable ($\notin C^1$).

An alternative is *feature selection*: directly restricting which components of $w$ can be nonzero. But $\|\cdot\|_1$ regularization achieves a similar effect while keeping the problem convex.

The Lasso is the canonical example of why nonsmooth optimization matters: the kink at $w_i = 0$ is precisely where sparsity is enforced, and smoothing it away would defeat the purpose.

### 7.3 Classification: SVM and SVR

Consider binary classification with $y^s \in \{-1, 1\}$. An affine hyperplane $H(w, b) = \{x : \langle w, x\rangle = b\}$ separates the two classes if $y^s(\langle w, x^s\rangle - b) \geq 1$ for all $s$. The margin between the two classes is $2/\|w\|$; maximizing it leads to minimizing $\|w\|^2$. When perfect separation is impossible, we introduce slack.

**Definition 7.5 (Support Vector Machine).**

$$\min_{w,b}\Big\{\|w\|^2 + C\sum_{s\in S}\max\{1 - y^s(\langle w, x^s\rangle - b),\ 0\}\Big\}. \tag{7.2}$$

The term $\|w\|^2$ is the regularizer (model complexity), $C$ weights the empirical loss, and $\max\{\cdot, 0\}$ is the *hinge loss*. The hinge loss makes the objective convex but non-differentiable: $[\max\{\cdot, 0\}]'(0)$ is undefined.

**Definition 7.6 (Support Vector Regression).** For regression ($y^s \in \mathbb{R}$), SVR uses the $\varepsilon$-insensitive loss:

$$\min_{w,b}\Big\{\|w\|^2 + C\sum_{s\in S}\max\{|\langle w, x^s\rangle - b - y^s| - \varepsilon,\ 0\}\Big\}. \tag{7.3}$$

Predictions within $\varepsilon$ of the target incur no penalty. The objective is again convex but non-differentiable.

Both SVM and SVR have rich dual formulations that we exploit in Section 8. For now, the key point: these are mainstream ML models with nonsmooth objectives. Smooth methods fail on them, and that failure is not a bug, it is the nature of the problem.

### 7.4 Why Smooth Methods Fail

When $f \notin C^1$, fixed-stepsize gradient descent breaks. The direction $-g$ for $g \in \partial f(x)$ (a *subgradient*, defined below) may not be a descent direction: there can exist points where $f(x + \alpha d) \geq f(x)$ for all $\alpha > 0$ and some $d = -g$. The norm $\|g\|$ is no longer a reliable proxy for $A(x)$: $\|g\|$ small implies proximity to $f^*$ only in one direction. And the stopping criterion $\|g\| \leq \varepsilon$ can be unreliable: the algorithm may stop far from the optimum.

The complexity picture is also worse.

| Aspect | $f \in C^1$ | $f \notin C^1$ |
|---|---|---|
| Direction | unique $d = -\nabla f(x)$ | multiple $d = -g,\ g \in \partial f(x)$ |
| Descent guarantee | small $\alpha$ ensures $f(x+\alpha d) < f(x)$ | possible $f(x+\alpha d) \geq f(x)\ \forall\, \alpha$ |
| $\|d\|$ as accuracy | $\|d\|$ small $\Leftrightarrow$ close to $f^*$ | $\|d\|$ small $\Rightarrow$ close, $\not\Leftarrow$ |
| Fixed stepsize | $\|x^{k+1} - x^k\| \to 0$ automatic | fails unless $\alpha^k \to 0$ |

**Table 7.1.** Smooth vs. nonsmooth: practical differences.

| Function class | Properties | Best achievable rate |
|---|---|---|
| $f \in C^1$ | $\tau$-convex, $L$-smooth | $\mathcal{O}(\log(1/\varepsilon))$ |
| $f \notin C^1$ | $\tau$-convex, $L$-Lipschitz | $\Omega(L^2/\varepsilon)$ |
| $f \in C^1$ | convex, $L$-smooth | $\mathcal{O}(1/\sqrt{\varepsilon})$ |
| $f \notin C^1$ | convex, $L$-Lipschitz | $\Omega(L^2/\varepsilon^2)$ |

**Table 7.2.** Convergence rates: smooth vs. nonsmooth. Losing $C^1$ costs one or two orders of magnitude in the exponent.

## 7.5 Subgradients and the Subdifferential

When $f$ isn't differentiable, we don't lack first-order information: we have *too much* of it.

**Definition 7.7 (Subgradient).** A vector $g \in \mathbb{R}^n$ is a *subgradient* of a convex function $f$ at $x$ if

$$f(z) \geq f(x) + \langle g, z - x \rangle \qquad \forall\, z \in \mathbb{R}^n. \tag{7.4}$$

Key properties: $g = 0$ implies $x$ is a global minimum (and conversely, $0 \in \partial f(x)$ iff $x$ minimizes $f$); if $f$ is differentiable at $x$, the unique subgradient is $\nabla f(x)$; at non-differentiable points, multiple subgradients exist, and not all are descent directions.

**Definition 7.8 (Subdifferential).** The *subdifferential* of $f$ at $x$ is $\partial f(x) = \{g \in \mathbb{R}^n : g \text{ is a subgradient of } f \text{ at } x\}$. For convex $f$, $\partial f(x)$ is a nonempty, compact, convex set at every $x$. The *steepest descent subgradient* is $g^* = \arg\min_{g \in \partial f(x)} \|g\|$, and $0 \in \partial f(x)$ iff $x$ is a global minimum.

Despite the multiple subgradients, every $g \in \partial f(x)$ satisfies $\langle g, x^* - x \rangle \leq f(x^*) - f(x) \leq 0$: the negative subgradient "points toward" $x^*$ in the sense of reducing $f$. This is enough to drive convergence, but not to guarantee descent at each step.

**Example 7.9 (Subdifferential of a pointwise maximum).** Let $f(x) = \max\{f_1(x), f_2(x)\}$ with $f_1(x) = (x-1)^2$ and $f_2(x) = (x+1)^2$. The two parabolas cross at $x = 0$, where $f_1(0) = f_2(0) = 1$. For $x > 0$: $f = f_1$, so $\partial f(x) = \{2(x-1)\}$. For $x < 0$: $f = f_2$, so $\partial f(x) = \{2(x+1)\}$. At $x = 0$: both are active, $\nabla f_1(0) = -2$, $\nabla f_2(0) = 2$, so $\partial f(0) = \text{conv}\{-2, 2\} = [-2, 2]$. Since $0 \in [-2, 2]$, the point $x = 0$ satisfies the optimality condition, but it is *not* the global minimum (which is at $x = \pm 1$ with $f = 0$). The minimum of $\|g\|$ over $g \in \partial f(0)$ is $g^* = 0$: the steepest descent subgradient gives no useful direction.

### Subdifferential calculus

Calculus rules parallel the smooth case, with set-valued additions:

- *Linearity:* $\partial[\alpha f + \beta g](x) = \alpha\,\partial f(x) + \beta\,\partial g(x)$ for $\alpha, \beta \geq 0$.
- *Affine pre-composition:* $\partial[f(Ax+b)] = A^T[\partial f](Ax+b)$.
- *Chain rule (monotone):* if $g\colon \mathbb{R} \to \mathbb{R}$ is increasing, $\partial[g(f(x))] = [\partial g](f(x)) \cdot [\partial f](x)$.
- *Maximum:* for $f(x) = \max\{f_1(x), \ldots, f_m(x)\}$ with active set $I(x) = \{i : f_i(x) = f(x)\}$: $\partial f(x) = \operatorname{conv}\big(\bigcup_{i \in I(x)} \partial f_i(x)\big)$.
- *Partial minimization:* if $f(x) = \inf\{g(x,y) : y \in \mathbb{R}^m\}$ and $y^*$ achieves the infimum, then $\partial f(x) = \{s : (s,0) \in \partial g(x,y^*)\}$.
- *Infimal convolution:* if $f(x) = \inf\{f_1(x_1) + f_2(x_2) : x_1 + x_2 = x\}$ with optimal split $x_1^* + x_2^* = x$, then $\partial f(x) = \partial f_1(x_1^*) \cap \partial f_2(x_2^*)$.

## 7.6 Subgradient Methods

Any subgradient $g^k \in \partial f(x^k)$ points toward $x^*$, so stepping along $-g^k$ should bring us closer. The iteration is $x^{k+1} = x^k - \alpha^k g^k$.

The fundamental relationship driving the analysis is:

$$\|x^{k+1} - x_*\|^2 = \|x^k - x_*\|^2 + \underbrace{2\alpha^k(f^* - f(x^k))}_{\leq 0} + (\alpha^k)^2\|g^k\|^2. \tag{7.5}$$

As $\alpha^k \to 0$, the negative middle term dominates the positive last term, pulling the iterates toward $x_*$.

### Diminishing and square-summable stepsize (DSS)

**Definition 7.10 (DSS condition).** $\sum_{k=0}^{\infty} \alpha^k = \infty$ and $\sum_{k=0}^{\infty} (\alpha^k)^2 < \infty$. A typical example is $\alpha^k = 1/(k+1)$.

If $\|g^k\| \leq L$ ($f$ is $L$-Lipschitz), summing the fundamental relationship over $k$ iterations shows that $f(x^k) - f^* \to 0$ along a subsequence: the first sum diverges and the second converges. But convergence is not monotonic and there is no control on individual stepsizes, only on the long-term average. The stepsize is chosen a priori; $f(x^k)$ is never used, only $g^k$.

### Polyak stepsize

If we knew $f^*$, we could be far more adaptive. The distance reduction $\|x^{k+1} - x_*\|^2 - \|x^k - x_*\|^2$ is a quadratic in $\alpha$, minimized at

$$\alpha_*^k = \frac{f(x^k) - f^*}{\|g^k\|^2}. \tag{7.6}$$

**Definition 7.11 (Polyak stepsize).** $\alpha^k \in (0, 2\alpha_*^k)$ ensures $\|x^{k+1} - x_*\| < \|x^k - x_*\|$ at every step. Setting $\alpha^k = \alpha_*^k$ and telescoping gives $\bar{f}^k - f^* \leq L\|x^0 - x_*\|/\sqrt{k} = \mathcal{O}(1/\sqrt{k})$, i.e., $\mathcal{O}(1/\varepsilon^2)$ iterations.

Reducing $\varepsilon$ from $10^{-3}$ to $10^{-4}$ takes 100 times more iterations: $\varepsilon < 10^{-4}$ is rarely practical. The Polyak stepsize would be optimal if $f^*$ were known; in practice, it must be estimated.

#### Target level stepsize

When $f^*$ is unknown, we estimate it and revise when progress stalls. A reference level $f_{\text{ref}}$ and threshold $\delta$ define a target approximating $f^*$. Significant improvement updates $f_{\text{ref}}$; stagnation shrinks $\delta$. The record sequence $\{\bar{f}^k\} \to f^*$, but no clean stopping criterion exists: typically one runs for a fixed budget. Many parameters are involved $(\rho, \beta, \delta_0, R)$, making tuning nontrivial.

#### Deflected subgradient

A momentum-like deflection smooths the search direction:

$$d^k = \gamma^k g^k + (1 - \gamma^k) d^{k-1}, \qquad x^{k+1} = x^k - \alpha^k d^k. \tag{7.7}$$

Two convergence strategies: *stepsize-restricted* (deflection grows ⇒ stepsize shrinks) or *deflection-restricted* (stepsize chosen first ⇒ deflection adapts) [9]. The deflection parameter can be chosen optimally: $\gamma^k = \arg\min_{\gamma \in [0,1]} \|\gamma g^k + (1-\gamma) d^{k-1}\|^2$.

### 7.7 Smoothed Gradient Methods

Can we modify $f$ slightly to make it smooth and then apply fast gradient methods? Consider a function of the form $f(x) = \max\{x^T A z : z \in Z\}$ where $Z$ is convex and compact. For a given $x$, any optimal $\bar{z}$ gives $A\bar{z} \in \partial f(x)$; if multiple optima exist, $f \notin C^1$.

The smoothed version adds a penalty:

$$f_\mu(x) = \max\Big\{x^T A z - \frac{\mu}{2}\|z\|^2 : z \in Z\Big\}. \tag{7.8}$$

This is differentiable ($f_\mu \in C^1$), with Lipschitz constant $L_\mu = \|A\|^2/\mu$.

**Theorem 7.12 (Smoothing bounds).** *Let $K = \max\{\|z\|^2/2 : z \in Z\}$. Then $f_\mu(x) \le f(x) \le f_\mu(x) + \mu K$, so $f_\mu \to f$ as $\mu \to 0$.*

Choosing $\mu = \varepsilon/(2K)$ and minimizing $f_\mu$ to accuracy $\varepsilon/2$ gives an $\varepsilon$-optimal solution to $f$. Using the accelerated gradient on $f_\mu$ (which has $L_\mu = 2\|A\|^2 K/\varepsilon$), the total complexity is $\mathcal{O}(1/\varepsilon)$: a big improvement over the $\mathcal{O}(1/\varepsilon^2)$ of subgradient methods [10].

In practice, the picture is less rosy. The smoothing parameter forces steps of size $1/L_\mu = \mathcal{O}(\varepsilon)$: far too short to make progress early on. Subgradient methods converge faster initially but stagnate around $10^{-4}$; the smoothed method can reach $10^{-6}$, but may require $\sim 10^6$ iterations. The method is "parameter-free" in theory (given $K$), but estimating $K$ is itself a nontrivial convex maximization problem.

## 7.8 Bundle Methods

Subgradient methods use one subgradient per step and throw it away. Bundle methods *accumulate* first-order information across iterations to build a better model: a kind of first-order Newton method, without second-order information.

Given a point $x$, an oracle returns $f(x)$ and $g \in \partial f(x)$, defining a global linear underestimator: $l_{x,f(x),g}(z) = f(x) + \langle g, z - x\rangle \leq f(z)$ for all $z$.

**Definition 7.13 (Cutting plane model).** The *bundle* at step $k$ collects oracle outputs: $\mathscr{B}^k = \{(x^j, f^j, g^j) : j < k\}$. The *cutting plane model* is

$$f^k_{\mathscr{B}}(x) = \max\{f^j + \langle g^j, x - x^j\rangle : (x^j, f^j, g^j) \in \mathscr{B}^k\} \leq f(x). \tag{7.9}$$

This is a piecewise-linear convex underestimate of $f$. Minimizing it reduces to a linear program.

**Theorem 7.14 (Convex functions as maxima of affine functions).** *Every convex $f$ satisfies $f(x) = \max\{f(z) + \langle g, x - z\rangle : z \in \mathbb{R}^n,\ g \in \partial f(z)\}$. So $f = f_{\mathscr{B}}$ for a sufficiently large (potentially infinite) bundle.*

### Cutting plane algorithm

Minimize $f_{\mathscr{B}}$, evaluate $f$ at the minimizer, add a new cut, repeat. Model values $v^k$ are nondecreasing; record values $\bar{f}^k = \min\{f(x^j) : j \leq k\}$ are nonincreasing. Both converge to $f^*$.

The method works well when $f$ is polyhedral and few facets are active at $x^*$. But it has serious drawbacks: iterates can jump erratically (no locality), the bundle grows without bound, the master problem becomes expensive, and worst-case complexity can reach $\mathscr{O}(1/\varepsilon^{n/2})$ [11].

### Proximal bundle method

To stabilize the cutting plane, add a proximity term. The *stabilized master problem* is

$$\min_x \Big\{ f_{\mathscr{B}}(x) + \frac{\mu}{2}\|x - \bar{x}\|^2 \Big\}, \tag{7.10}$$

where $\bar{x}$ is the *stability center* (best point so far) and $\mu > 0$ controls regularization. The quadratic term acts like a trust region: if $\mu$ is too large, the iterate stays close to $\bar{x}$; if too small, we recover the unstable cutting plane.

The algorithm alternates between *serious steps* (the candidate significantly improves $f$, so $\bar{x}$ is updated) and *null steps* (improvement is insufficient, only the bundle is enriched). The optimality condition for the stabilized subproblem gives $-\mu d^* \in \partial f_{\mathscr{B}}(\bar{x} + d^*)$, leading to $f_{\mathscr{B}}(\bar{x} + d^*) - f(\bar{x}) \leq -\mu\|d^*\|^2$. As $k \to \infty$, $\|d^*\| \to 0$, guaranteeing convergence.

Reducing the bundle size $|\mathscr{B}|$ lowers the master-problem cost but increases iterations: small $|\mathscr{B}|$ resembles a subgradient method. Keeping $\mathscr{B}$ large speeds convergence. The worst-case complexity is $\mathscr{O}(1/\varepsilon^2)$ (same as subgradient methods) but practical performance is typically much better thanks to the accumulated information and stabilization [11].

# 8 Constrained Optimality and Duality

Unconstrained optimization asks for stationary points: $\nabla f(x) = 0$. Add constraints and this simple condition breaks: a minimizer can sit on the boundary of the feasible region with $\nabla f(x) \neq 0$. This section develops the theory needed to characterize constrained optima: tangent cones, the KKT conditions, Lagrangian duality, and specialized dual forms for structured problems. It ends with two major applications of the theory: proximal bundle methods for Lagrangian duals, and the kernel trick for support vector machines.

## 8.1 Problem Formulation

**Definition 8.1 (Constrained optimization problem).** $(P)$ $f^* = \min\{f(x) : x \in X\}$, where $f\colon \mathbb{R}^n \to \mathbb{R}$ is the objective and $X \subseteq \mathbb{R}^n$ the feasible region.

**Definition 8.2 (Indicator function).** $\mathbb{I}_X(x) = 0$ if $x \in X$, $+\infty$ otherwise. This is convex iff $X$ is convex. Using it, $(P)$ becomes $\min\{f(x) + \mathbb{I}_X(x)\}$: the *essential objective*. But $\mathbb{I}_X \notin C^0$, making the problem harder.

It is sometimes useful to hide constraints in the objective: the *epigraph reformulation* $(P) \equiv \min\{v : v \geq f(x),\, x \in X\}$ turns a nonlinear objective into a linear one at the cost of an extra variable and a nonlinear constraint. If $X = \emptyset$, the problem is infeasible and $\nu(P) = +\infty$.

**Definition 8.3 (Interior, boundary, local optimum).** $x \in \operatorname{int}(X)$ if $\exists\, r > 0 : \mathscr{B}(x, r) \subseteq X$. $x \in \partial X$ if every ball around $x$ contains points both inside and outside $X$. If $x^* \in \operatorname{int}(X)$ and $x^*$ is a local minimum, then $\nabla f(x^*) = 0$: the unconstrained condition still holds. If $x^* \in \partial X$, this need not be true: the constraint "pushes" the optimum to the boundary.

**Definition 8.4 (Open, closed, full-dimensional sets).** $S$ is *open* if $S = \operatorname{int}(S)$; *closed* if it contains all its limit points ($x_i \to x$, $x_i \in S \Rightarrow x \in S$); *full-dimensional* if $\operatorname{int}(S) \neq \emptyset$. Finite intersections of open sets are open; finite unions of closed sets are closed.

## 8.2 Tangent Cone and First-Order Geometric Conditions

The key question: at a boundary point $x$, which directions lead into $X$? The answer is the tangent cone: obtained by "zooming in" on $x$ until the set looks like a cone.

**Definition 8.5 (Tangent cone).** $T_X(x) = \{d \in \mathbb{R}^n : \exists\, \{z_i \in X\} \to x,\ \{t_i\} \to 0,\ t_i > 0 : (z_i - x)/t_i \to d\}$.

**Definition 8.6 (Tangent cone condition (TCC)).** $\langle \nabla f(x), d \rangle \geq 0$ for all $d \in T_X(x)$.

**Theorem 8.7 (TCC is necessary for local optimality).** *If $x$ is a local optimum of $(P)$, then TCC holds.*

*Proof.* By contradiction. Suppose $\exists\, d \in T_X(x)$ with $\langle \nabla f(x), d \rangle < 0$. By definition of $T_X$, $\exists\, \{z_i\} \to x$ in $X$ and $\{t_i\} \to 0$ with $(z_i - x)/t_i \to d$. Taylor expansion: $f(z_i) - f(x) = \langle \nabla f(x), z_i -$

$x\rangle + R(z_i - x)$ where $R(h)/\|h\| \to 0$. Since $z_i - x \approx t_i d$, we get $\langle \nabla f(x), z_i - x\rangle \approx t_i \langle \nabla f(x), d\rangle < 0$. The remainder vanishes faster, so $f(z_i) < f(x)$ for large $i$, contradicting local optimality since $z_i \in X \cap \mathscr{B}(x, \varepsilon)$. □

If $X$ is convex, TCC is also *sufficient* for global optimality (since $X \subseteq x + T_X(x)$). At an interior point, $T_X(x) = \mathbb{R}^n$, so TCC reduces to $\nabla f(x) = 0$.

TCC generalizes the notion of stationary point to constrained problems: it is necessary for optimality and the only condition we can reasonably check in practice. But expressing $T_X(x)$ explicitly requires knowing $X$ algebraically.

## 8.3 Convexity of Sets

**Definition 8.8 (Convex set).** A set $C$ is convex if $\alpha x + (1-\alpha)z \in C$ for all $x, z \in C$ and $\alpha \in [0, 1]$. The *convex hull* $\mathrm{conv}(S)$ is the smallest convex set containing $S$; $C$ is convex iff $C = \mathrm{conv}(C)$.

Basic convex sets: polytopes (convex hull of finitely many points), affine sets (hyperplanes $\{x : \langle a, x\rangle = b\}$), half-spaces ($\{x : \langle a, x\rangle \le b\}$), ellipsoids ($\{z : (z-x)^T Q(z-x) \le r\}$ with $Q \succeq 0$), and $p$-norm balls $\mathscr{B}_p(x, r)$.

Convex cones: the conical hull $\mathrm{Cone}(d_1, \ldots, d_k) = \{\sum_i \mu_i d_i : \mu_i \ge 0\}$ is polyhedral. The Lorentz (ice-cream) cone $\mathscr{L} = \{x : x_n \ge \sqrt{\sum_{i<n} x_i^2}\}$ and the positive semidefinite cone $S^+ = \{Q : Q \succeq 0\}$ are important non-polyhedral examples.

### Convexity-preserving operations on sets

Finite intersection; image and pre-image under affine maps; Minkowski sum; slices and shadows of convex sets in higher dimensions; interior and closure of a convex set. These rules allow building complex feasible regions from simple convex pieces.

### Convex sets from convex functions

If all $g_i$ are convex, the sublevel sets $\{x : g_i(x) \le 0\}$ are convex. But $\{x : g_i(x) \ge 0\}$ is typically non-convex ("reverse convex"). Equality constraints $g_i(x) = 0$ yield convex sets only when $g_i$ is affine. So for $X$ to be convex: all inequality constraints must be convex, and all equality constraints must be affine.

## 8.4 Feasible Directions and Algebraic Formulation

**Definition 8.9 (Feasible direction cone).** $F_X(x) = \{d : \exists\, \bar\varepsilon > 0,\ x + \varepsilon d \in X\ \forall\, \varepsilon \in [0, \bar\varepsilon]\}$. If $X$ is convex, $F_X(x) \subseteq T_X(x)$ and they essentially agree (same closure), so $X \subseteq x + T_X(x)$.

**Definition 8.10 (Algebraic form).** With inequality constraints $g_i$ and equality constraints $h_j$: $X = \{x : g_i(x) \le 0,\ i \in I;\ h_j(x) = 0,\ j \in J\}$.

Reformulations are possible (equalities as pairs of inequalities, multiple inequalities as a single max), but they often destroy structure. A cautionary example: $\max\{g_1, g_2\} \notin C^1$ even if $g_1, g_2 \in C^1$.

### Linear constraints and polyhedra

Linear constraints $g_i(x) = \langle A_i, x\rangle + b_i \le 0$ define half-spaces; their intersection is a *polyhedron.* $k$ equalities $Ax = b$ define an affine subspace of dimension at most $n - k$. Polyhedra are tractable: optimizing a linear function over one is solvable in polynomial time.

For $(P)$ $\min\{f(x) : Ax = b\}$: every $x \in X$ lies on the boundary, and feasible directions satisfy $Ad = 0$. TCC forces $\nabla f(x) \perp \ker(A)$, i.e., $\nabla f(x) \in \mathrm{im}(A^T)$. This gives the *Poorman's KKT*: $x$ is optimal iff $Ax = b$ and $\exists\, \mu : \nabla f(x) = A^T\mu$ [12].

The *reduced problem* eliminates the equality constraints: partition $A = [A_B \mid A_N]$ with $A_B$ invertible, set $x_B = A_B^{-1}(b - A_N x_N)$, and optimize over $x_N$ alone. The reduced gradient $\nabla r(x_N) = D^T \nabla f(Dx_N + d)$ recovers the KKT conditions at its zero.

## 8.5 Nonlinear Constraints and KKT

**Definition 8.11 (Active constraints and first-order feasible direction cone).** $\mathscr{A}(x) = \{i \in I : g_i(x) = 0\}$ are the active constraints. $D_X(x) = \{d : \langle \nabla g_i(x), d\rangle \le 0 \ \forall i \in \mathscr{A}(x);\ \langle \nabla h_j(x), d\rangle = 0\ \forall j\}$. Always $D_X(x) \supseteq T_X(x)$, but the inclusion can be strict.

*Constraint qualifications* are conditions under which $T_X(x) = D_X(x)$:

- *Affine constraints (AffC):* all $g_i, h_j$ affine $\Rightarrow$ always holds.
- *Slater's condition (SlaC):* $g_i$ convex, $h_j$ affine, $\exists\, \bar{x} : g_i(\bar{x}) < 0\ \forall i$.
- *Linear independence (LinI):* gradients of active constraints at $\bar{x}$ are linearly independent.

**Definition 8.12 (Dual cone and Farkas' lemma).** For a polyhedral cone $C = \{d : Ad \le 0\}$, its dual cone is $C^* = \{\sum_i \lambda_i A_i : \lambda_i \ge 0\}$. *Farkas' lemma:* for any $c \in \mathbb{R}^n$, exactly one holds: (1) $\exists\, \lambda \ge 0$ with $c = \sum_i \lambda_i A_i$, or (2) $\exists\, d$ with $Ad \le 0$ and $\langle c, d\rangle > 0$. Equivalently, $C^\circ = C^*$ (polar equals dual) [5].

**Theorem 8.13 (Karush–Kuhn–Tucker conditions).** *If a constraint qualification holds and $x^*$ is a local optimum, then $\exists\, \lambda \in \mathbb{R}_+^{|I|}$, $\mu \in \mathbb{R}^{|J|}$ such that:*

- $g_i(x^*) \le 0\ \forall i$, $h_j(x^*) = 0\ \forall j$ *(KKT-F: feasibility)*
- $\nabla f(x^*) + \sum_i \lambda_i \nabla g_i(x^*) + \sum_j \mu_j \nabla h_j(x^*) = 0$ *(KKT-G: stationarity)*
- $\lambda_i g_i(x^*) = 0\ \forall i$ *(KKT-CS: complementary slackness)*

Two caveats. KKT can hold at saddle points or maxima of non-convex problems. And without constraint qualifications, KKT may fail even at the optimum. The safe case: $f$ convex on $X$ convex ($g_i$ convex, $h_j$ affine), where KKT is necessary and sufficient for global optimality.

## 8.6 Second-Order Conditions

When $(P)$ is not convex, KKT is necessary but not sufficient. Second-order conditions fill the gap.

**Definition 8.14 (Lagrangian and critical cone).** The *Lagrangian* is $L(x;\lambda,\mu) = f(x) + \sum_i \lambda_i g_i(x) + \sum_j \mu_j h_j(x)$. At a KKT point, $\nabla_x L = 0$. The *critical cone* restricts attention to directions consistent with the active constraints:

$$C(x;\lambda,\mu) = \{d : \langle \nabla g_i, d\rangle = 0 \text{ if } \lambda_i > 0;\ \langle \nabla g_i, d\rangle \le 0 \text{ if } \lambda_i = 0,\ i \in \mathscr{A};\ \langle \nabla h_j, d\rangle = 0\ \forall\, j\}. \quad (8.1)$$

**Theorem 8.15 (Second-order conditions).** *Under KKT and linear independence:*

- *Necessary:* $d^T \nabla_x^2 L(x;\lambda,\mu)\, d \ge 0$ *for all* $d \in C(x;\lambda,\mu)$.
- *Sufficient: strict inequality for all nonzero* $d \in C$ *implies* $x$ *is a strict local minimum.*

## 8.7 Lagrangian Duality

KKT stationarity says $x$ is stationary for $L(\cdot;\lambda,\mu)$ at the right multipliers. If we knew $\lambda, \mu$, we could find $x$ by solving an unconstrained problem. This idea drives duality.

**Definition 8.16 (Lagrangian relaxation and dual function).** $\psi(\lambda,\mu) = \min_x L(x;\lambda,\mu)$. For any $\lambda \ge 0$ and any $\mu$: $\psi(\lambda,\mu) \le \nu(P)$ (*weak duality*). $\psi$ is concave (even if $f, g_i, h_j$ are not convex), possibly $-\infty$, and often easy to compute. If the minimizer $\bar{x}$ of $L$ is unique, $\psi$ is differentiable with $\nabla\psi = [G(\bar{x}), H(\bar{x})]$. Otherwise $\psi$ may be nonsmooth: the methods of Section 7 apply directly.

**Definition 8.17 (Lagrangian dual).** $(D) \quad \max\{\psi(\lambda,\mu) : \lambda \ge 0,\ \mu \in \mathbb{R}^{|J|}\}$. This is a convex program (even if $(P)$ is not), providing a lower bound $\nu(D) \le \nu(P)$.

**Definition 8.18 (Strong duality).** If $\nu(D) = \nu(P)$, we have *strong duality*: the bound is tight and the dual solution recovers all the information of the primal.

Strong duality does not always hold. A simple counterexample: $\min\{-x^2 : 0 \le x \le 1\}$ has $\nu(P) = -1$, but $\psi(\lambda) = -\infty$ for all $\lambda \ge 0$, so $\nu(D) = -\infty$. The gap arises because $x^*$ is a maximizer (not minimizer) of $L$ for the KKT multipliers.

**Theorem 8.19 (Strong duality for convex problems).** *If* $(P)$ *is convex and a constraint qualification holds, then* $\nu(D) = \nu(P)$.

*Proof.* Since $(P)$ is convex with a CQ, KKT conditions hold at $x^*$: $\exists\, \lambda^* \ge 0$, $\mu^*$ satisfying stationarity ($\nabla_x L(x^*;\lambda^*,\mu^*) = 0$) and complementary slackness ($\lambda_i^* g_i(x^*) = 0$ for all $i$). By stationarity, $x^*$ minimizes $L(\cdot;\lambda^*,\mu^*)$ over $\mathbb{R}^n$ (the Lagrangian is convex in $x$). Therefore $\psi(\lambda^*,\mu^*) = L(x^*;\lambda^*,\mu^*)$. Expanding: $L(x^*;\lambda^*,\mu^*) = f(x^*) + \sum_i \lambda_i^* g_i(x^*) + \sum_j \mu_j^* h_j(x^*)$. By feasibility $h_j(x^*) = 0$, and by complementary slackness $\lambda_i^* g_i(x^*) = 0$. So $\psi(\lambda^*,\mu^*) = f(x^*) = \nu(P)$. Since $\psi(\lambda^*,\mu^*) \le \nu(D) \le \nu(P)$ (weak duality), equality follows. □

### Economic interpretation

The optimal multiplier $\lambda_i^*$ measures the *sensitivity* of $f^*$ to a relaxation of constraint $i$: loosening $g_i(x) \le 0$ to $g_i(x) \le \varepsilon$ changes $f^*$ by approximately $-\lambda_i^* \varepsilon$. Active constraints with large $\lambda_i^*$ are "expensive": removing them would significantly improve the objective.

## 8.8 Specialized Duals

### Linear programming

For $(P)$ $\min\{c^T x : Ax \ge b\}$: the dual is $(D)$ $\max\{\lambda^T b : A^T\lambda = c,\ \lambda \ge 0\}$. Strong duality holds almost always for LPs.

### Quadratic programming: equality constraints

For $(P)$ $\min\{\frac{1}{2}\|x\|^2 : Ax = b\}$: $\psi(\mu) = -\frac{1}{2}\mu^T AA^T\mu - \mu^T b$, unconstrained in $\mu$.

### Strictly convex QP

For $(P)$ $\min\{\frac{1}{2}x^T Qx + q^T x : Ax \ge b\}$ with $Q \succ 0$: $(D)$ $\max\{\lambda^T b - \frac{1}{2}v^T Q^{-1} v : \lambda^T A - v = q^T,\ \lambda \ge 0\}$. Strong duality holds.

## 8.9 Conic Programs

**Definition 8.20 (Conic program).** Given a pointed convex cone $K$: $(P)$ $\min\{c^T x : Ax \ge_K b\}$ where $x \ge_K y \Leftrightarrow x - y \in K$. Important cones: $K = \mathbb{R}^n_+$ (LP), $K = S^+$ (SDP), $K = \mathscr{Q}$ the second-order cone (SOCP). Every LP and convex QP is an SOCP; every SOCP is an SDP, but not vice versa.

**Definition 8.21 (Conic dual).** With dual cone $K_D = \{z : \langle z, v\rangle \ge 0\ \forall\, v \in K\}$ (self-dual for $\mathbb{R}^n_+$, $S^+$, $\mathscr{Q}$): $(D)$ $\max\{\lambda^T b : \lambda^T A = c^T,\ \lambda \ge_{K_D} 0\}$. Strong duality requires a Slater-type constraint qualification.

SOCP explicit form: $\min\{c^T x : \|D_i x - d_i\| \le p_i^T x - q_i,\ i = 1, \dots, m\}$.

SDP explicit form: $\min\{c^T x : \sum_i x_i A^i \succeq B\}$ with dual $\max\{\langle B, \Lambda\rangle : \langle A^i, \Lambda\rangle = c_i,\ \Lambda \succeq 0\}$, where $\langle A, B\rangle = \operatorname{tr}(A^T B)$ is the Frobenius inner product.

## 8.10 Proximal Bundle Method for Lagrangian Duals

When $(D)$ involves a nonsmooth $\psi$, the proximal bundle method from Section 7.8 applies directly. The bundle $\mathscr{B} = \{(x^h, f^h, g^h)\}_{h<i}$ builds a cutting-plane model $f^i_{\mathscr{B}} \le f$.

The stabilized master problem uses the *linearization error* $\alpha^h = f(\bar{x}) - f^h - \langle g^h, \bar{x} - x^h\rangle \ge 0$. The dual master problem is

$$\min\Big\{\frac{1}{2\mu}\Big\|\sum_h \theta_h g^h\Big\|^2 + \sum_h \alpha^h\theta_h : \theta \ge 0,\ \sum_h \theta_h = 1\Big\}, \tag{8.2}$$

giving $z^* = \sum_h \theta_h^* g^h$ and $d^* = -z^*/\mu$.

Key advantages: the dual master may be faster to solve; elements with $\theta_h^* = 0$ can be pruned; and the entire bundle can be replaced by the aggregate $(z^*, \sigma^*)$: a "poorman's bundle" that degrades to a subgradient method.

## 8.11 SVM and SVR: A Duality Application

The nonsmooth SVM and SVR formulations from Section 7.3 can be reformulated using slack variables and solved via their duals.

Introducing slacks $\xi_i \geq 0$, the SVM primal becomes

$$\min_{w,b,\xi} \Big\{ \tfrac{1}{2}\|w\|^2 + C \textstyle\sum_i \xi_i : y^i(\langle w, x^i \rangle - b) \geq 1 - \xi_i,\ \xi_i \geq 0 \Big\}. \tag{8.3}$$

The dual is a box-constrained QP:

$$\max_{\alpha} \Big\{ \textstyle\sum_i \alpha_i - \tfrac{1}{2} \sum_{i,j} \alpha_i y^i \langle x^i, x^j \rangle y^j \alpha_j : \sum_i y^i \alpha_i = 0,\ 0 \leq \alpha_i \leq C \Big\}. \tag{8.4}$$

The primal solution recovers as $w^* = \sum_i \alpha_i^* y^i x^i$; only *support vectors* ($\alpha_i^* > 0$) contribute.

### The kernel trick

In practice, linear separation is often impossible. The idea: embed data nonlinearly via $\varphi\colon \mathbb{R}^h \to \mathscr{F}$ into a (possibly infinite-dimensional) feature space where a linear model suffices. The dual depends on inner products $\langle x^i, x^j \rangle$ only, which we replace with a *kernel function* $\kappa(x^i, x^j) = \langle \varphi(x^i), \varphi(x^j) \rangle$, computed without forming $\varphi$ explicitly. Prediction for a new $\bar{x}$: $\sum_i \alpha_i^* y^i \kappa(x^i, \bar{x}) - b^*$.

A kernel must be positive semidefinite. Standard choices: polynomial $\kappa(x, z) = (\langle x, z \rangle + 1)^k$; Gaussian $\kappa(x, z) = \exp(-\|x - z\|^2/(2\sigma^2))$; sigmoid $\kappa(x, z) = \tanh(\sigma\langle x, z \rangle + \delta)$. SVR with a Gaussian kernel can approximate any continuous function on a bounded domain arbitrarily well, given enough data.

# 9 Constrained Optimization Algorithms

The previous section established *what* characterizes a constrained optimum: KKT conditions, duality, second-order tests. Finding one is the *how*, and the job of this section. The algorithmic landscape for constrained optimization is richer than the unconstrained case, because the feasible region $X$ introduces geometric structure that each method exploits differently.

The organizing principle: the structure of $X$ determines the algorithm. Linear inequality constraints lead to active-set and simplex-type methods; simple constraints (boxes, balls, simplices) favor projected gradient and Frank–Wolfe; general convex constraints are handled by barrier and interior point methods; and the dual perspective transforms complex constraints into simple ones at the cost of a nonsmooth objective.

## 9.1 Equality-Constrained Quadratic Programs

These serve as the foundation: every more complex method eventually reduces to solving a sequence of equality-constrained QPs.

The problem is $\min\{\frac{1}{2}x^TQx + q^Tx : Ax = b\}$ with $A \in \mathbb{R}^{m\times n}$, $\operatorname{rank}(A) = m < n$. The KKT conditions yield the symmetric indefinite system

$$\begin{bmatrix} Q & A^T \\ A & 0 \end{bmatrix} \begin{bmatrix} x \\ \mu \end{bmatrix} = \begin{bmatrix} -q \\ b \end{bmatrix}. \tag{9.1}$$

Three solution strategies exist, each with its own trade-offs.

### Schur complement (reduced KKT)

When $Q$ is nonsingular, eliminate $x$ to get the smaller system $[AQ^{-1}A^T]\mu = -b - AQ^{-1}q$. The Schur complement $M = AQ^{-1}A^T \in \mathbb{R}^{m\times m}$ is positive semidefinite when $Q \succeq 0$ and smaller than the original system, but can be dense even when $A$ and $Q$ are sparse. Preconditioned conjugate gradient on $M$ avoids forming it explicitly.

### Null space method

Partition $A = [A_B \mid A_N]$ with $A_B$ invertible. The constraint $Ax = b$ gives $x_B = A_B^{-1}(b - A_N x_N)$, so $x = Dx_N + d$ where $D = [-A_B^{-1}A_N;\ I]^T$. The *reduced Hessian* $\tilde{Q} = D^TQD \in \mathbb{R}^{(n-m)\times(n-m)}$ governs the reduced problem $\min_{x_N} \frac{1}{2}x_N^T\tilde{Q}\,x_N + (D^TQd + D^Tq)^Tx_N$. The method generalizes to any basis of $\ker(A)$ and handles singular $Q$.

### Cholesky on the KKT system

For moderate $n$, direct factorization of the KKT system at $\mathcal{O}(n^3)$ is often the best choice, providing both $x$ and $\mu$ simultaneously.

## 9.2 Active Set Methods

Active set methods solve inequality-constrained QPs by maintaining an estimate of the active constraints and solving a sequence of equality-constrained subproblems.

The key insight: if we knew $\mathcal{A}(x^*) = \{i : A_ix^* = b_i\}$, the problem would reduce to linear algebra. Since we don't, we estimate it iteratively, using dual information to guide updates.

Each iteration either moves to a strictly better feasible point or improves the dual solution. Since there are at most $2^m$ possible active sets and the objective decreases, the method terminates in finite time. Linear independence of $A_B$ is required for well-posedness.

### Box-constrained problems

For $\min\{\frac{1}{2}x^TQx + q^Tx : \underline{x} \le x \le \overline{x}\}$, the active set structure is particularly elegant. Each active constraint corresponds to a variable fixed at its bound. The reduced problem involves only the

**Algorithm 11** Active Set Method for Quadratic Programs (ASMQP)

**Require:** $Q \in \mathbb{R}^{n\times n}$, $q \in \mathbb{R}^n$, $A \in \mathbb{R}^{m\times n}$, $b \in \mathbb{R}^m$, feasible $x^0$
**Ensure:** Optimal solution to $(P)$ $\min\{\frac{1}{2}x^TQx + q^Tx : Ax \le b\}$

**procedure** ASMQP($Q, q, A, b, x^0$)
  $x \leftarrow x^0$; $B \leftarrow \mathscr{A}(x)$
  **while** true **do**
    Solve $(P_B)$ $\min\{f(x) : A_Bx = b_B\}$; let $(\bar{x}, \bar{\mu}_B)$ be the KKT pair
    **if** $A_i\bar{x} \le b_i$ for all $i \notin B$ **then** ▷ Candidate feasible?
      **if** $\bar{\mu}_B \ge 0$ **then return** $\bar{x}$ ▷ KKT satisfied: optimal
      **else**
        Remove $h = \arg\min_{i\in B}\{\bar{\mu}_i\}$ from $B$ ▷ Drop most violated dual
      **end if**
    **else**
      $d \leftarrow \bar{x} - x$; $\bar{\alpha} \leftarrow \min\{(b_i - A_ix)/(A_id) : A_id > 0,\ i \notin B\}$
      $x \leftarrow x + \bar{\alpha}\, d$; $B \leftarrow \mathscr{A}(x)$
    **end if**
  **end while**
**end procedure**

"free" variables: $\min\{\frac{1}{2}x_F^TQ_{FF}x_F + \tilde{q}_F^Tx_F\}$ where $\tilde{q}$ absorbs the contribution of fixed variables. Linear independence is automatic, initialization is trivial (any point in the box), and active set updates add or remove single bounds.

Active set methods extend to general nonlinear objectives by replacing the equality subproblem with an unconstrained optimization on the reduced space.

## 9.3 Projected Gradient Methods

Projected gradient methods maintain feasibility at every iteration while making progress toward optimality. When $-\nabla f(x)$ is infeasible (it points outside $X$), we project it onto the cone of feasible directions.

### Box-constrained projected gradient

For box constraints, the projection decomposes into independent scalar projections: simply zero out gradient components that would push a variable past its bound. The step size is bounded by the distance to the nearest violated bound.

### Goldstein's method

An alternative: take the gradient step first, then project onto $X$: $y^k = x^k - \alpha^k\nabla f(x^k)$, then $x^{k+1} = p_X(y^k)$. This is not a descent method, but works well when projection onto $X$ is cheap. For $L$-smooth convex functions with $\alpha = 1/L$: $\mathscr{O}(LD^2/\varepsilon)$ iterations; with $\tau$-strong convexity: $\mathscr{O}((L/\tau)\log(1/\varepsilon))$ linear convergence.

**Algorithm 12** Box-Constrained Projected Gradient (BCPGM)

**Require:** $f$, bounds $\underline{x}, \overline{x}$, initial feasible $x$, tolerance $\varepsilon$

**procedure** BCPGM($f, \underline{x}, \overline{x}, x, \varepsilon$)
  **while** true **do**
    $d \leftarrow -\nabla f(x)$; $\bar{\alpha} \leftarrow \infty$
    **for** $i = 1, \ldots, n$ **do**
      **if** $d_i < 0$ and $x_i = \underline{x}_i$, or $d_i > 0$ and $x_i = \overline{x}_i$ **then**
        $d_i \leftarrow 0$ ▷ Project out infeasible component
      **else if** $d_i \neq 0$ **then**
        $\bar{\alpha} \leftarrow \min(\bar{\alpha},$ distance to bound along $d_i)$
      **end if**
    **end for**
    **if** $\langle \nabla f(x), d \rangle \geq -\varepsilon$ **then return** $x$
    **end if**
    $\alpha \leftarrow$ line search$(f, x, d, \bar{\alpha})$; $x \leftarrow x + \alpha d$
  **end while**
**end procedure**

Projection cost determines practical efficiency: $\mathcal{O}(n)$ for boxes ($(p_X(y))_i = \max(\underline{x}_i, \min(y_i, \overline{x}_i))$), $\mathcal{O}(n)$ for balls, closed-form for simplices.

#### Rosen's method

For general linear constraints $Ax \leq b$, project onto the tangent space of active constraints. With $\bar{A} = A_{\mathscr{A}(x)}$ of full row rank: $d = (I - \bar{A}^T[\bar{A}\bar{A}^T]^{-1}\bar{A})(-\nabla f(x))$. If the projected gradient is zero, check dual feasibility (KKT multipliers $\lambda = -[\bar{A}\bar{A}^T]^{-1}\bar{A}\nabla f(x)$); if some $\lambda_h < 0$, drop constraint $h$ and re-project.

### 9.4 Frank–Wolfe (Conditional Gradient) Method

The Frank–Wolfe method replaces the projection step with a linear minimization over $X$. At each iterate, it linearizes $f$ and solves $\min\{\langle \nabla f(x), z \rangle : z \in X\}$, then moves toward the minimizer.

The optimality condition $\langle \nabla f(x), d \rangle = 0$ implies $x$ is optimal (for convex $f$, the quantity $f(x) + \langle \nabla f(x), d \rangle$ provides a computable lower bound, giving an optimality certificate).

Each iteration requires solving one LP over $X$. The method shines when $X$ has structure that makes linear optimization cheap: network flows, semidefinite constraints ($X = \{X : \operatorname{tr}(X) \leq 1, X \succeq 0\}$ reduces to a leading eigenvector computation), or norm balls (closed-form solutions).

#### Enhancements

A starting feasible point can be found by solving an LP with a dummy objective. Quadratic regularization (add $\frac{\gamma}{2}\|z - x\|^2$) or trust region constraints ($\|z - x\|_\infty \leq \tau$) improve convergence without complicating the subproblem much. Away steps (move away from vertices, not just toward new ones) help avoid stalling at non-optimal extreme points.

**Algorithm 13** Frank–Wolfe Method (FW)

**Require:** $f$, feasible region $X$, initial $x \in X$, tolerance $\varepsilon$

**procedure** FW($f, X, x, \varepsilon$)
  **while** true **do**
    $\bar{x} \leftarrow \arg\min\{\langle \nabla f(x), z\rangle : z \in X\}$ ▷ Linear subproblem
    $d \leftarrow \bar{x} - x$
    **if** $\langle \nabla f(x), d\rangle \geq -\varepsilon$ **then return** $x$
    **end if**
    $\alpha \leftarrow$ line search$(f, x, d, 1)$ with $\alpha \leq 1$
    $x \leftarrow x + \alpha d$
  **end while**
**end procedure**

### Extensions

For $f \notin C^1$: replace $\nabla f(x)$ with the cutting plane model $f_{\mathscr{B}}$, giving the constrained bundle method $\min\{f_{\mathscr{B}}(z) + \gamma\|z - \bar{x}\|^2 : z \in X\}$. For $f \in C^2$: incorporate second-order information via SQP (Sequential Quadratic Programming): $d^* = \arg\min\{\langle \nabla f(x), d\rangle + \frac{1}{2} d^T \nabla^2 f(x)\, d : x + d \in X\}$, requiring a constrained QP per iteration.

## 9.5 Dual Methods

Dual methods work in the space of Lagrange multipliers, transforming complex primal constraints into simple dual constraints.

For $(P)$ $\min\{\frac{1}{2} x^T Q x + q^T x : Ax \leq b\}$ with $Q \succ 0$, the dual function is

$$\psi(\lambda) = \lambda^T b - \frac{1}{2}(A^T\lambda - q)^T Q^{-1}(A^T\lambda - q), \tag{9.2}$$

continuously differentiable with $\nabla\psi(\lambda) = b - Ax(\lambda)$ where $x(\lambda) = Q^{-1}(A^T\lambda - q)$. The dual problem $(D)$ $\max\{\psi(\lambda) : \lambda \geq 0\}$ is a concave maximization with simple box constraints.

As $\lambda^k \to \lambda^*$, the primal recovery $x(\lambda^k) \to x^*$. The dual provides valid lower bounds throughout: $\psi(\lambda^k) \leq \nu(P)$. When $X$ has additional structure, a primal-feasible approximation $\hat{x}^k = p_X(x(\lambda^k))$ gives an upper bound, and the gap $f(\hat{x}^k) - \psi(\lambda^k)$ is a computable optimality certificate.

### Advantages

Step size issues (related to $\kappa = L/\tau$) are often less critical in the dual; simple constraints $\lambda \geq 0$ are easier than complex primal constraints; parallelization opportunities arise through decomposition.

### Decomposition

For problems with separable structure $(P)$ $\min\{f(x) : Ax \leq b,\, Ex \leq d\}$, relaxing only the "linking" constraints $Ax \leq b$ gives a Lagrangian relaxation $\psi(\lambda) = \min\{f(x) + \lambda^T(b - Ax) : Ex \leq d\}$ that decomposes into smaller, independent subproblems, enabling parallel solution.

## 9.6 Barrier Methods

Barrier methods transform the constrained problem into a sequence of unconstrained ones by penalizing proximity to the constraint boundary.

**Definition 9.1 (Logarithmic barrier).** For $X = \{x : Ax \leq b\}$, the barrier function is

$$f_\gamma(x) = f(x) - \gamma \sum_{i=1}^{m} \ln(b_i - A_i x), \tag{9.3}$$

where $\gamma > 0$ is the barrier parameter. $f_\gamma \in C^2$ when $f \in C^2$; $f_\gamma \to +\infty$ at the boundary of $X$ (keeping iterates strictly interior); $f_\gamma$ is strictly convex when $f$ is convex.

For each $\gamma > 0$, the unique minimizer $x_\gamma$ traces the *central path* $\mathscr{C} = \{x_\gamma : \gamma \in (0, \infty)\}$. As $\gamma \to 0$, $x_\gamma \to x^*$ (the constrained optimum); as $\gamma \to \infty$, $x_\gamma$ approaches the *analytical center* $x_\infty = \arg\max\{\prod_i (b_i - A_i x)\}$.

The *path-following* strategy: start near the analytical center and decrease $\gamma$ toward zero, tracking the central path with Newton steps. The logarithmic barrier is *self-concordant*, ensuring that Newton's method behaves well. If $x^k$ is close to $x_{\gamma^k}$, a few Newton steps yield $x^{k+1}$ close to $x_{\gamma^{k+1}}$ with $\gamma^{k+1} = \tau\gamma^k$ for $\tau < 1$.

Iteration complexity: $\mathscr{O}(\sqrt{m}\log(1/\varepsilon))$, independent of $n$ but depending on the number of constraints $m$. Each Newton step costs $\mathscr{O}(n^3)$, so the total is $\mathscr{O}(n^3\sqrt{m}\log(1/\varepsilon))$.

## 9.7 Primal-Dual Interior Point Methods

Primal-dual methods represent the most practically successful approach to barrier-based optimization, solving primal and dual problems simultaneously.

For the QP $\min\{\frac{1}{2}x^TQx + q^Tx : Ax \leq b\}$, introduce slacks $s \geq 0$ with $Ax + s = b$. The KKT conditions are: primal feasibility ($Ax + s = b$, $s \geq 0$), dual feasibility ($Qx + q - A^T\lambda = 0$, $\lambda \geq 0$), and complementary slackness ($\lambda_i s_i = 0$).

The barrier method relaxes complementary slackness to $\lambda_i s_i = \gamma$ for all $i$, giving a smooth system solvable by Newton's method. Using $\Lambda = \mathrm{diag}(\lambda)$ and $S = \mathrm{diag}(s)$, the Newton system for $(\Delta x, \Delta\lambda, \Delta s)$ is

$$\begin{bmatrix} Q & -A^T & 0 \\ A & 0 & I \\ 0 & S & \Lambda \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta\lambda \\ \Delta s \end{bmatrix} = \begin{bmatrix} r^D \\ r^P \\ \gamma u - \Lambda S u \end{bmatrix}, \tag{9.4}$$

where $r^P = b - Ax - s$ and $r^D = -(Qx + q - A^T\lambda)$ are residuals.

By eliminating $\Delta s$ and substituting, the system reduces to solving $(Q + A^T\Lambda S^{-1}A)\Delta x = \text{rhs}$. The matrix $M = Q + A^T\Lambda S^{-1}A$ is positive definite when $Q \succeq 0$ (since $\Lambda S^{-1} \succ 0$), ensuring a unique solution.

### Advanced techniques

The *predictor-corrector* approach solves the Newton system twice: first a "prediction" (affine scaling), then a "correction" that adds the nonlinear term $\Delta\Lambda\Delta S\, u$, reusing the matrix factoriza-

**Algorithm 14** Primal-Dual Interior Point Method (IPMQP)

**Require:** $Q, A, q, b$, tolerances $\varepsilon_P, \varepsilon_D, \varepsilon$, reduction factor $\rho$

**procedure** IPMQP($Q, A, q, b, \varepsilon_P, \varepsilon_D, \varepsilon, \rho$)
    Initialize $x$, $\lambda > 0$, $s > 0$
    $\gamma \leftarrow \langle \lambda, s \rangle / m$
    **while** $\|r^P\| > \varepsilon_P$ or $\|r^D\| > \varepsilon_D$ or $\gamma m > \varepsilon$ **do**
        Solve Newton system for $(\Delta x, \Delta\lambda, \Delta s)$
        $\alpha \leftarrow 0.995 \cdot \max\{\beta : \lambda + \beta\Delta\lambda \geq 0,\ s + \beta\Delta s \geq 0\}$
        $(x, \lambda, s) \leftarrow (x + \alpha\Delta x,\ \lambda + \alpha\Delta\lambda,\ s + \alpha\Delta s)$
        $\gamma \leftarrow \rho \cdot \langle \lambda, s \rangle / m$
    **end while**
**end procedure**

tion. Multiple centrality corrections further improve centering. Adaptive selection of $\rho$ based on Newton step quality keeps the iterates near the central path.

### Convergence

The method achieves polynomial complexity $\mathcal{O}(\sqrt{m}\log(1/\varepsilon))$, with each iteration costing $\mathcal{O}(n^3)$ for the matrix factorization. Superlinear convergence in the final phase is typical. Primal and dual solutions converge simultaneously, providing both an optimal $x^*$ and optimal multipliers $\lambda^*$. The framework extends directly to SOCP and SDP. Numerical issues may arise in ill-conditioned or degenerate problems, where small slack or multiplier values cause instability in the $\Lambda S^{-1}$ scaling.

# A Exercises

## A.1 Foundations and Quadratic Forms

**Fixing the relative gap** [Section 1.5]

The relative gap $R(x) = A(x)/|f^*|$ is ill-defined when $f^* = 0$. Propose at least two fixes and justify which one is more reliable in practice.

Solution. Two natural candidates:

- *Fix 1, threshold denominator:* replace $|f^*|$ with $\max\{|f^*|, 1\}$. This is simple to implement but introduces an arbitrary scale. When $f^*$ lies in the range $10^{-3}$, the denominator jumps from $10^{-3}$ to 1, making $R(x)$ appear much smaller than the true relative error. The threshold 1 has no intrinsic meaning: it just happens to be a convenient default.
- *Fix 2, symmetric denominator:* define $R(x) = A(x)/\max\{|f(x)|, |f^*|\}$. This is scale-invariant by construction: replacing $f$ by $\alpha f$ (for any $\alpha > 0$) leaves $R$ unchanged. When $f^* \approx 0$, the denominator is at least $|f(x)|$, which is positive unless $x$ is already optimal. The formula degrades gracefully and does not depend on an arbitrary constant.

Fix 2, or a close variant that divides by the incumbent $|f(x)|$ with a small additive constant, is the standard choice in optimization software (CPLEX, Gurobi). The key insight: a good relative measure should be dimensionless and invariant under positive rescaling of $f$.

**Locally but not globally Lipschitz** [Result 3.1]

Show that $f(x) = x^2$ is locally Lipschitz at every $x$ but not globally Lipschitz on $\mathbb{R}$.

Solution. *Local Lipschitz at any $x_0$.* Fix $x_0 \in \mathbb{R}$ and take $\delta > 0$. For any $x, z \in [x_0 - \delta, x_0 + \delta]$:

$$|f(x) - f(z)| = |x^2 - z^2| = |x + z| \cdot |x - z| \leq (2|x_0| + 2\delta) \cdot |x - z|. \tag{A.1}$$

So $f$ is Lipschitz on $[x_0 - \delta, x_0 + \delta]$ with constant $L = 2|x_0| + 2\delta$. This $L$ is finite for every fixed $x_0$ and $\delta$: the definition of locally Lipschitz.

*Not globally Lipschitz.* The constant $L$ depends on $x_0$ and grows without bound as $|x_0| \to \infty$. No single $L$ works for all of $\mathbb{R}$. An equivalent argument via derivatives: $f'(x) = 2x$ is unbounded on $\mathbb{R}$. If $f$ were globally $L$-Lipschitz, the mean value theorem would give $|f'(x)| \leq L$ for all $x$, which fails since $|f'(x)| = 2|x| \to \infty$. The relationship between Lipschitz constants and derivatives is developed in Section 3.4.

**Continuous but not Lipschitz** [Section 3.1, Section 3.4]

Find a function that is continuous on $[-1, 1]$ but not Lipschitz there. Why does this not contradict the extreme value theorem?

Solution. Take $f(x) = \sqrt{|x|}$. This function is continuous on $[-1, 1]$ (composition of continuous

functions). Near $x = 0$:

$$\frac{|f(x) - f(0)|}{|x - 0|} = \frac{\sqrt{|x|}}{|x|} = \frac{1}{\sqrt{|x|}} \to \infty \quad \text{as } x \to 0. \tag{A.2}$$

No finite $L$ bounds this ratio, so $f$ is not Lipschitz on any interval containing the origin.

*Why no contradiction with the extreme value theorem?* The extreme value theorem guarantees that $f$ attains its maximum and minimum on a closed bounded interval; it says nothing about the rate of change or the existence of a Lipschitz constant. In fact, $f \in C^0[-1, 1]$ but $f \notin C^1[-1, 1]$, because $|f'(x)| = 1/(2\sqrt{|x|}) \to \infty$ as $x \to 0$ (and $f'(0)$ does not exist).

The precise chain of implications is: $f \in C^1$ on a compact interval $\Rightarrow$ $f$ globally Lipschitz there (since $|f'|$ is bounded by the extreme value theorem). But $C^0 \not\Rightarrow$ Lipschitz, and Lipschitz $\not\Rightarrow C^1$.

**Full spectral analysis of a $2 \times 2$ matrix** [Section 2.8, Section 2.11, Result 5.3]

Let $Q = \left[\begin{smallmatrix} 5 & 3 \\ 3 & 5 \end{smallmatrix}\right]$. Find eigenvalues, eigenvectors, the decomposition $Q = H\Lambda H^T$, and the condition number $\kappa$. Sketch the level sets of $f(x) = \frac{1}{2}x^T Q x$.

Solution. *Eigenvalues.* The characteristic polynomial is $\det(Q - \lambda I) = (5 - \lambda)^2 - 9 = \lambda^2 - 10\lambda + 16 = 0$. The roots are $\lambda_1 = 8$ and $\lambda_2 = 2$.

*Eigenvectors.* For $\lambda_1 = 8$: $(Q - 8I)v = 0$ gives $-3v_1 + 3v_2 = 0$, so $v_1 = v_2$. Normalized: $H_1 = \frac{1}{\sqrt{2}}[1, 1]^T$. For $\lambda_2 = 2$: $(Q - 2I)v = 0$ gives $3v_1 + 3v_2 = 0$, so $v_1 = -v_2$. Normalized: $H_2 = \frac{1}{\sqrt{2}}[-1, 1]^T$.

*Decomposition.* $H = \frac{1}{\sqrt{2}}\left[\begin{smallmatrix} 1 & -1 \\ 1 & 1 \end{smallmatrix}\right]$, $\Lambda = \operatorname{diag}(8, 2)$, and $Q = H\Lambda H^T$.

*Condition number.* $\kappa = \lambda_1/\lambda_2 = 8/2 = 4$.

*Level sets.* The level set $L(f, v)$ is an ellipse centered at the origin with axes along $H_1$ and $H_2$ (rotated 45° from the coordinate axes). The semi-axis length in the $H_i$-direction is $\sqrt{2v/\lambda_i}$. Along $H_1$ (large eigenvalue $\lambda_1 = 8$): shorter axis $\sqrt{2v/8} = \sqrt{v/4}$. Along $H_2$ (small eigenvalue $\lambda_2 = 2$): longer axis $\sqrt{2v/2} = \sqrt{v}$. The ratio of semi-axes is $\sqrt{\lambda_1/\lambda_2} = \sqrt{4} = 2$: the ellipse is twice as long in the $H_2$-direction as in the $H_1$-direction.

**Characterizing linear functions** [Section 2.3]

A function $f\colon \mathbb{R}^n \to \mathbb{R}$ satisfies $f(\gamma x) = \gamma f(x)$ for all $\gamma > 0$ and $f(x + z) = f(x) + f(z)$ for all $x, z$. Prove that $f(x) = \langle b, x\rangle$ for some $b \in \mathbb{R}^n$. Does this extend to affine functions $f(x) = \langle b, x\rangle + c$?

Solution. *Construction of $b$.* Let $u^1, \ldots, u^n$ be the canonical basis vectors (Section 2.2). Define $b_i = f(u^i)$ for each $i$. Any $x \in \mathbb{R}^n$ decomposes as $x = \sum_{i=1}^n x_i u^i$. By additivity (applied $n - 1$ times): $f(x) = f(\sum_i x_i u^i) = \sum_{i=1}^n f(x_i u^i)$. By homogeneity: $f(x_i u^i) = x_i f(u^i) = x_i b_i$ (this requires extending homogeneity to all $\gamma \in \mathbb{R}$, which follows from additivity: $f(0) = f(0 + 0) = 2f(0)$ forces $f(0) = 0$, and $0 = f(x + (-x)) = f(x) + f(-x)$ gives $f(-x) = -f(x)$). Therefore $f(x) = \sum_i b_i x_i = \langle b, x\rangle$.

*Affine functions.* If $f(0) = c \neq 0$, homogeneity fails: $f(0) = f(\gamma \cdot 0) = \gamma f(0) = \gamma c$ for all $\gamma > 0$, which is impossible unless $c = 0$. So the characterization applies only to linear functions, not affine ones. An affine function $f(x) = \langle b, x \rangle + c$ with $c \neq 0$ satisfies additivity up to a constant ($f(x + z) = f(x) + f(z) - c$) but violates homogeneity.

> **Anatomy of a univariate quadratic** [Section 2.4]
>
> For $f(x) = ax^2 + bx + c$ with $a > 0$, express the center $\bar{x}$, the minimum value $f(\bar{x})$, and the roots of $f$ in terms of $a, b, c$. Discuss what happens when the discriminant $\Delta = b^2 - 4ac$ is positive, zero, or negative.

Solution. *Center and minimum.* By Equation (2.1): $\bar{x} = -b/(2a)$ and $f(\bar{x}) = c - b^2/(4a) = -\Delta/(4a)$.

*Roots.* Setting $f(x) = 0$ and applying the quadratic formula: $x = \bar{x} \pm \sqrt{\Delta}/(2a)$ when $\Delta \geq 0$.

*Three cases.*

- $\Delta > 0$: two distinct real roots, $f(\bar{x}) = -\Delta/(4a) < 0$. The parabola dips below zero between the roots.
- $\Delta = 0$: one double root at $\bar{x}$, $f(\bar{x}) = 0$. The parabola touches the $x$-axis exactly at its vertex.
- $\Delta < 0$: no real roots, $f(\bar{x}) = -\Delta/(4a) > 0$. The parabola stays strictly positive everywhere.

For optimization, the discriminant is irrelevant: regardless of $\Delta$, the minimum is always at $\bar{x}$ with value $-\Delta/(4a)$. The roots matter for feasibility questions (is $f(x) \leq 0$ achievable?) but not for finding the optimum.

## A.2 Univariate Optimization

> **Golden ratio vs. naive bisection** [Section 3.3]
>
> Derive the iteration count $k \geq \log(D/\delta)/\log(1/r)$ for GRS with $r = (\sqrt{5} - 1)/2 \approx 0.618$. Compare with naive bisection ($r = 0.5$, two evaluations per step) for $D/\delta = 10^6$.

Solution. *Derivation.* After $k$ iterations the interval has width $Dr^k$. We need $Dr^k \leq \delta$, i.e., $r^k \leq \delta/D$, i.e.,

$$k \geq \frac{\log(D/\delta)}{\log(1/r)} = \frac{\log(D/\delta)}{-\log r}. \tag{A.3}$$

*GRS.* Taking natural logs, $-\ln(0.618) \approx 0.481$, so $k \geq \ln(D/\delta)/0.481 \approx 2.078 \ln(D/\delta)$. For $D/\delta = 10^6$: $k \geq 2.078 \times \ln(10^6) = 2.078 \times 13.82 \approx 29$ iterations. Each iteration uses 1 function evaluation (after the initial 2), so total evaluations $\approx 31$.

*Naive bisection.* With $r = 0.5$: $k \geq \log(10^6)/\log(2) \approx 20$ iterations, but each requires 2 evaluations (neither test point carries over), giving 40 total evaluations.

*Comparison.* GRS wins: 31 vs. 40 evaluations. The reuse trick is why GRS beats bisection on evaluations: the golden ratio is the unique $r$ for which a test point carries over ($r^2 = 1 - r$),

cutting the cost to one evaluation per iteration. Optimality among methods that use only function values is the stronger, separate result cited in Section 3.3.

> **When exponential expansion fails** [Section 3.5]
>
> The bracket-finding procedure tries $x_+ = 2, 4, 8, 16, \ldots$ and stops when $f'(x_+) > 0$. Construct a $C^\infty$ function with infinitely many local minima such that the procedure never stops.

Solution. Take $f(x) = \sin(\pi x + 3\pi/4)$. Then $f'(x) = \pi\cos(\pi x + 3\pi/4)$. At the test points $x = 2^i$ for $i \geq 1$ (that is, $2, 4, 8, \ldots$, all even):

$$f'(2^i) = \pi\cos(\pi \cdot 2^i + 3\pi/4) = \pi\cos(2^i\pi + 3\pi/4) = \pi\cos(3\pi/4) = -\frac{\pi\sqrt{2}}{2} \approx -2.22, \tag{A.4}$$

where the second equality uses $\cos(\theta + 2k\pi) = \cos(\theta)$ for any integer $k$, and $2^i\pi$ (for $i \geq 1$) is always an even multiple of $\pi$. The derivative is negative at *every* test point, so the bracket is never found, even though $f$ has local minima at $x = 2k + 3/4$ for $k \in \mathbb{Z}$ (where $f = -1$).

The failure is fragile: shifting the phase constant by a small $\varepsilon$, or changing the doubling factor from 2 to 2.1, would break the counterexample. This illustrates why the exponential expansion "works in practice for all reasonable functions" but has no worst-case guarantee without a coercivity assumption on $f$.

> **The map is not the world** [Section 3.6]
>
> In quadratic interpolation the model predicts $x = x_- + 0.02(x_+ - x_-)$, nearly at the left boundary. With safeguard $\sigma = 0.1$, what point is actually used? If the model is consistently wrong, what convergence rate results?

Solution. *Clamping.* The safeguard from Equation (3.7) enforces $x \in [x_- + \sigma D,\ x_+ - \sigma D]$ where $D = x_+ - x_-$. Since the model's prediction $0.02D$ falls below the lower guard $0.1D$, the clamped point is $x_- + 0.1D$.

*Worst-case rate.* If the model is consistently wrong (always clamped), each iteration shrinks the interval by at most a factor $1 - \sigma = 0.9$. This gives linear convergence with rate $r = 1 - \sigma = 0.9$: slow (about 22 iterations per digit of accuracy), but guaranteed to converge.

*Best-case rate.* When the model is accurate, the interpolation estimate falls well inside $[\sigma D, (1 - \sigma)D]$ and the safeguard never activates. Convergence is then superlinear with order $p = (1 + \sqrt{5})/2 \approx 1.618$ (Result 3.8). The safeguard is a safety net, not a brake: it only activates when the quadratic model is unreliable.

## A.3 Optimality and Convexity

> **Convex implies quasi-convex, but not vice versa** [Section 4.5, Section 4.6]
>
> Prove the implication and give a counterexample for the reverse.

Solution. *Forward implication.* Let $f$ be convex. For any $x, z$ and $\alpha \in [0,1]$: $f(\alpha x+(1-\alpha)z) \leq \alpha f(x) + (1-\alpha)f(z)$. A convex combination of two real numbers is at most their maximum: $\alpha a+(1-\alpha)b \leq \max\{a,b\}$ for $\alpha \in [0,1]$ (since $\alpha a+(1-\alpha)b \leq \alpha \max\{a,b\}+(1-\alpha)\max\{a,b\} = \max\{a,b\}$). Chaining the two inequalities: $f(\alpha x+(1-\alpha)z) \leq \max\{f(x), f(z)\}$, which is exactly the quasi-convexity condition from Result 4.14.

*Counterexample for the reverse.* Take $f(x) = -e^{-x^2}$. This function is unimodal on $\mathbb{R}$ with a unique minimum at $x = 0$, so it is quasi-convex (its sublevel sets $\{x : f(x) \leq \ell\}$ are intervals, which are convex). But $f''(x) = (2-4x^2)e^{-x^2}$ is negative for $|x| > 1/\sqrt{2}$: at $x = 1$, for instance, $f''(1) = -2e^{-1} < 0$, so $f$ is concave in the tails. This violates the second-order characterization of convexity ($f \in C^2$ convex $\Leftrightarrow f''(x) \geq 0$ for all $x$; see Section 4.5). The function bends downward away from the origin while its sublevel sets stay intervals: the two notions genuinely differ.

**Epigraph characterization of convexity** [Result 4.13]

Prove: $\text{epi}(f) = \{(v,x) : v \geq f(x)\}$ is convex if and only if $f$ is convex.

Solution. *($\Rightarrow$) Convex epigraph implies convex function.* Take any $x_1, x_2 \in \mathbb{R}^n$ and $\alpha \in [0,1]$. The points $(f(x_1), x_1)$ and $(f(x_2), x_2)$ lie in $\text{epi}(f)$ (with equality $v = f(x)$). If $\text{epi}(f)$ is convex, their convex combination also lies in $\text{epi}(f)$: $(\alpha f(x_1) + (1-\alpha)f(x_2),\ \alpha x_1 + (1-\alpha)x_2) \in \text{epi}(f)$. By definition of the epigraph, this means $\alpha f(x_1) + (1-\alpha)f(x_2) \geq f(\alpha x_1 + (1-\alpha)x_2)$, which is the convexity condition for $f$.

*($\Leftarrow$) Convex function implies convex epigraph.* Take any $(v_1, x_1), (v_2, x_2) \in \text{epi}(f)$ and $\alpha \in [0,1]$, so $v_1 \geq f(x_1)$ and $v_2 \geq f(x_2)$. By convexity of $f$: $f(\alpha x_1 + (1-\alpha)x_2) \leq \alpha f(x_1) + (1-\alpha)f(x_2) \leq \alpha v_1 + (1-\alpha)v_2$. Therefore $(\alpha v_1 + (1-\alpha)v_2,\ \alpha x_1 + (1-\alpha)x_2) \in \text{epi}(f)$, confirming convexity of $\text{epi}(f)$.

**Gradient norm bounds the gap** [Result 4.15]

If $f$ is $\tau$-strongly convex, prove $\|\nabla f(x)\|^2 \geq 2\tau(f(x) - f^*)$. Why does this fail for merely convex functions?

Solution. *Proof.* The $C^1$ characterization of $\tau$-strong convexity (Result 4.15), evaluated at $z = x^*$, gives:

$$f(x^*) \geq f(x) + \langle \nabla f(x), x^* - x \rangle + \frac{\tau}{2}\|x^* - x\|^2. \tag{A.5}$$

Rearranging: $f(x) - f(x^*) \leq -\langle \nabla f(x), x^* - x\rangle - \frac{\tau}{2}\|x^* - x\|^2$. The right-hand side is a concave quadratic in $w = x^* - x$. To find the tightest bound, maximize over $w$: setting the gradient to zero gives $-\nabla f(x) - \tau w = 0$, i.e., $w = -\nabla f(x)/\tau$. Substituting back:

$$f(x) - f^* \leq \frac{\|\nabla f(x)\|^2}{\tau} - \frac{\tau}{2} \cdot \frac{\|\nabla f(x)\|^2}{\tau^2} = \frac{\|\nabla f(x)\|^2}{2\tau}. \tag{A.6}$$

Rearranging: $\|\nabla f(x)\|^2 \geq 2\tau(f(x) - f^*)$.

*Why this fails without strong convexity.* When $\tau = 0$, the bound degenerates to $f(x) - f^* \leq +\infty$, which is vacuous. A concrete example: $f(x) = |x|$ is convex with $\|g\| = 1$ for all $x \neq 0$ (every subgradient has unit norm), yet $f(x) - f^* = |x|$ can be arbitrarily large. The gradient norm is

bounded but the gap is unbounded: without the quadratic "bowl" guaranteed by strong convexity, the gradient says nothing about how far we are from the optimum.

> **Finite differences vs. autodiff** [Section 4.3, Section 3.4]
>
> Approximating $\nabla f(x) \in \mathbb{R}^n$ by forward finite differences costs $n+1$ evaluations of $f$. Automatic differentiation costs $\sim$ 3–5 times one evaluation. For what $n$ does the difference matter?

Solution. *Cost comparison.* Let $C_f$ denote the cost of one evaluation of $f$.

- *Forward finite differences:* approximate $\partial f/\partial x_i \approx (f(x+he_i) - f(x))/h$ for each $i$. Total: $(n+1)C_f$ (one base evaluation plus $n$ perturbed evaluations).
- *Reverse-mode autodiff:* propagate gradients backward through the computation graph. Total: $\sim 4C_f$, independent of $n$ (the constant depends on implementation overhead but not on the number of variables).

Break-even occurs at $n \approx 3$.

*Scaling.* For $n = 100$: finite differences cost $101C_f$, autodiff costs $\sim 4C_f$: a 25× speedup. For $n = 10^6$ (a moderate neural network): the ratio is 250,000×. Central differences ($(f(x+he_i) - f(x-he_i))/(2h)$, better accuracy) use $2n$ evaluations, making the gap even wider.

This is why every modern deep learning framework (PyTorch, JAX, TensorFlow) uses reverse-mode automatic differentiation. The $\mathcal{O}(1)$-in-$n$ cost of autodiff is what makes gradient-based optimization practical for problems with millions of parameters.

## A.4 Gradient Methods for Quadratics

> **Condition number in action** [Result 5.4, Result 5.12]
>
> For $\kappa(Q) = 1000$, compute the contraction rates and iteration counts for steepest descent and conjugate gradient to reduce the error by a factor of $10^{-6}$.

Solution. *Steepest descent.* By the Kantorovich inequality (Result 5.4): $r_{\text{SD}} = \big((\kappa-1)/(\kappa+1)\big)^2 = (999/1001)^2 \approx 0.99601$.

*Conjugate gradient.* By Result 5.12: $r_{\text{CG}} = (\sqrt{\kappa}-1)/(\sqrt{\kappa}+1) = (31.62-1)/(31.62+1) \approx 0.9387$.

*Iteration counts.* We need $r^k \le 10^{-6}$, i.e., $k \ge 6\ln 10/(-\ln r)$:

- SD: $k \ge 13.82/0.00400 \approx 3{,}455$ iterations.
- CG: $k \ge 13.82/0.0632 \approx 219$ iterations.

CG is roughly 16× faster in iterations, and the per-iteration cost is nearly the same ($\mathcal{O}(n^2)$, dominated by one matrix–vector product). For $\kappa = 10^6$, the gap widens to $\sim$ 500×: CG's $\sqrt{\kappa}$-dependence is dramatically better than SD's $\kappa$-dependence.

**CG exploits eigenvalue clustering** [Result 5.13]

Let $Q = \text{diag}(1, 1, 1, 1, 100)$: four eigenvalues equal to 1 and one equal to 100. How many CG iterations are needed for exact convergence (in exact arithmetic)? What if $Q = \text{diag}(1, 1.01, 0.99, 1.02, 100)$?

Solution. *Exact case.* $Q$ has only 2 distinct eigenvalues (1 and 100). By Result 5.13, CG terminates in at most 2 iterations, regardless of $n = 5$. The method "sees" only the number of distinct eigenvalues, not the matrix size.

*Perturbed case.* Now $Q$ has 5 distinct eigenvalues: $0.99, 1, 1.01, 1.02, 100$. The formal bound says at most 5 iterations. But the eigenvalue-dependent bound from Result 5.13 is much tighter. After $k = 2$ steps, the "effective" $\lambda_k$ is the second-largest eigenvalue $\lambda_2 = 1.02$, and $\lambda_n = 0.99$. The bound gives: $r \leq (\lambda_2 - \lambda_n)/(\lambda_2 + \lambda_n) = (1.02 - 0.99)/(1.02 + 0.99) = 0.03/2.01 \approx 0.015$. This is nearly zero: in practice, 2–3 iterations suffice because the cluster near 1 is effectively treated as a single eigenvalue.

This is why CG excels on matrices with a few outlier eigenvalues and the rest clustered: a common pattern in PDE discretizations (where a few modes dominate) and regularized problems (where the regularizer compresses most eigenvalues).

**Can we get $\varepsilon = 0$?** [Section 5.2, Section 5.7]

Does steepest descent converge to $x^*$ exactly in finite time? What about CG? What breaks in floating-point arithmetic?

Solution. *Steepest descent.* The linear rate $r < 1$ gives $A(x^k) \leq r^k A(x^0)$. Since $r^k > 0$ for all finite $k$, steepest descent never reaches $x^*$ exactly; it converges asymptotically. The error decreases geometrically but never hits zero.

*CG in exact arithmetic.* Finite termination in at most $n$ steps (Result 5.10): after $n$ $Q$-conjugate directions, the method has explored all of $\mathbb{R}^n$ and the residual is exactly zero.

*CG in floating point.* The $Q$-conjugacy condition $(p^i)^T Q p^j = 0$ for $i \neq j$ degrades: rounding errors accumulate and the directions gradually lose mutual orthogonality. After $n$ steps, the error is typically $\sim 10^{-12}$ rather than zero. For large $n$, the loss of orthogonality can be severe after far fewer than $n$ steps. The standard remedy is periodic restarts: reset $p \leftarrow -g$ every $n$ (or fewer) steps to "re-anchor" the conjugate directions. This sacrifices finite termination but maintains practical convergence quality.

## A.5 Smooth Optimization

**Fixed stepsize can diverge** [Section 6.2]

For $f(x) = x^4/4$, show that the fixed stepsize $\bar{\alpha} = 2$ causes oscillation from $x^0 = 1$. What is the correct stepsize near $x = 1$?

SOLUTION. *Oscillation.* $\nabla f(x) = x^3$, so the update is $x^{k+1} = x^k - 2(x^k)^3$. Starting at $x^0 = 1$: $x^1 = 1 - 2(1)^3 = -1$, then $x^2 = -1 - 2(-1)^3 = -1 + 2 = 1$, and the cycle $1 \to -1 \to 1 \to \cdots$ repeats indefinitely. The iterates oscillate without converging; $f(x^k) = 1/4$ for all $k$.

*Correct stepsize.* The Hessian is $f''(x) = 3x^2$, so the local smoothness constant at $x = 1$ is $L_{\text{local}} = f''(1) = 3$. The fixed-stepsize convergence guarantee (Section 6.2) requires $\bar{\alpha} < 2/L$, giving $\bar{\alpha} < 2/3$ near $x = 1$. With $\bar{\alpha} = 2$, we exceed this bound by a factor of 3.

*Why no global fixed stepsize exists.* $f''(x) = 3x^2 \to \infty$ as $|x| \to \infty$, so $f$ is not globally $L$-smooth: no single $L$ bounds the Hessian everywhere. A fixed stepsize that works near the origin fails far away, and vice versa. Line search is essential for functions without a global Lipschitz gradient.

> **Detecting unboundedness** [Section 6.1, Section 2.12.1]
>
> Can a gradient descent run detect that $f^* = -\infty$? For quadratics? For general $C^1$ functions?

SOLUTION. *Quadratics.* If $Q$ is indefinite, $\exists\, d$ with $d^TQd < 0$. Along this direction: $f(x + \alpha d) = \frac{1}{2}\alpha^2(d^TQd) + \alpha\langle q + Qx, d\rangle + f(x) \to -\infty$ as $\alpha \to +\infty$. Both Newton's method and Cholesky factorization (Section 5.9) can detect indefiniteness directly: a negative pivot during factorization is a constructive certificate that $Q \not\succeq 0$. The negative-curvature direction exposed by the factorization (the $d$ with $d^TQd < 0$ read off from the negative pivot) is exactly the direction of unbounded decrease.

*General $C^1$ functions.* No finite-time certificate exists. If $f(x^k) \to -\infty$ as $k \to \infty$, we can observe this empirically (function values keep decreasing without bound), but we cannot *prove* $f^* = -\infty$ from finitely many evaluations: the function might eventually turn around. In practice, a heuristic diagnostic (e.g., $f(x^k) < -M$ for some large threshold $M$, or $\|x^k\| \to \infty$ with $f$ still decreasing) signals likely unboundedness, but it remains an empirical observation, not a proof.

> **L-BFGS initialization** [Section 6.8]
>
> The initial approximation $B^0 = \gamma I$ with $\gamma = \langle s, y\rangle / \|y\|^2$ aims to capture the average curvature. Show that for a quadratic $f(x) = \frac{1}{2}x^TQx + q^Tx$, $\gamma = 1/\bar{\lambda}$ where $\bar{\lambda}$ is a weighted average of eigenvalues. What happens when $\gamma \to 0$ or $\gamma \to \infty$?

SOLUTION. *Derivation for quadratics.* For a quadratic, $y = \nabla f(x^{i+1}) - \nabla f(x^i) = Q(x^{i+1} - x^i) = Qs$. Therefore $\langle s, y\rangle = s^TQs$ and $\|y\|^2 = s^TQ^2s$. Expanding $s$ in the eigenbasis $s = \sum_j c_j H_j$:

$$\gamma = \frac{s^TQs}{s^TQ^2s} = \frac{\sum_j c_j^2\lambda_j}{\sum_j c_j^2\lambda_j^2}. \tag{A.7}$$

Equivalently $\gamma = 1/\bar{\lambda}$, where $\bar{\lambda} = (\sum_j c_j^2\lambda_j^2)/(\sum_j c_j^2\lambda_j)$ is a weighted arithmetic average of the eigenvalues with weights $c_j^2\lambda_j$. If $s$ is aligned with a single eigenvector $H_j$ (i.e., $c_j = 1$, all others zero), then $\bar{\lambda} = \lambda_j$ and $\gamma = 1/\lambda_j$ exactly. In general, $\gamma$ lies between $1/\lambda_1$ and $1/\lambda_n$, tilted toward the eigenvalues that $s$ "sees."

*Limiting behavior.*

- $\gamma \to 0$: the approximate inverse Hessian $B^0 = \gamma I$ shrinks to zero. The step $B^0 g = \gamma g$ becomes

vanishingly small: the method degrades to very tiny steepest-descent steps and effectively stalls.

- $\gamma \to \infty$: the step $B^0 g = \gamma g$ blows up, causing overshooting and potential divergence. The method trusts the gradient direction far too much.

The formula auto-calibrates: after the first gradient step, $\gamma$ picks up a scale commensurate with the local curvature, and subsequent L-BFGS corrections refine the approximation.

## A.6 Nonsmooth Optimization

> **Subdifferential of the absolute value** [Result 7.7, Result 7.8]
>
> Compute $\partial f(x)$ for $f(x) = |x|$ at every $x \in \mathbb{R}$. Verify that $0 \in \partial f(0)$ and interpret this geometrically.

Solution. *Away from the origin.* For $x > 0$: $f$ is differentiable with $f'(x) = 1$, so $\partial f(x) = \{1\}$. For $x < 0$: $f'(x) = -1$, so $\partial f(x) = \{-1\}$.

*At the origin.* We need all $g \in \mathbb{R}$ such that $|z| \geq |0| + g \cdot z = gz$ for all $z \in \mathbb{R}$. Testing $z > 0$: need $z \geq gz$, i.e., $g \leq 1$. Testing $z < 0$: need $-z \geq gz$, i.e., $g \geq -1$. So $\partial f(0) = [-1, 1]$.

*Optimality.* Since $0 \in [-1, 1] = \partial f(0)$, the optimality condition $0 \in \partial f(x)$ is satisfied at $x = 0$, confirming it as the global minimum.

*Geometric interpretation.* At $x = 0$, the graph of $|x|$ has a corner. A subgradient $g \in [-1, 1]$ defines a supporting line $\ell(z) = gz$ that touches the graph at $z = 0$ and lies entirely below it. The family of supporting lines sweeps from slope $-1$ to slope $+1$. The horizontal supporting line ($g = 0$) certifies that no direction decreases $f$: the point is optimal.

> **Smoothing equals the Huber function** [Section 7.7]
>
> Construct $f_\mu(x)$ for $f(x) = |x| = \max_{z \in [-1,1]} xz$. Show that the result is the Huber function. Compute $L_\mu$ and verify $f_\mu \to f$ as $\mu \to 0$.

Solution. *Construction.* Following the smoothing framework of Section 7.7,

$$f_\mu(x) = \max_{z \in [-1,1]} \left\{ xz - \tfrac{\mu}{2} z^2 \right\}.$$

The unconstrained maximizer of $xz - \frac{\mu}{2} z^2$ is $z^* = x/\mu$, from setting the derivative $x - \mu z = 0$. Projecting onto $[-1, 1]$ gives $z^* = \min\{1, \max\{-1, x/\mu\}\}$.

*Two cases.*

- $|x| \leq \mu$: the unconstrained maximizer $x/\mu$ lies in $[-1, 1]$. Substituting: $f_\mu(x) = x \cdot (x/\mu) - \frac{\mu}{2}(x/\mu)^2 = x^2/(2\mu)$.
- $|x| > \mu$: the maximizer saturates at $z^* = \operatorname{sign}(x)$. Substituting: $f_\mu(x) = |x| - \mu/2$.

Combining:

$$f_\mu(x) = \begin{cases} x^2/(2\mu) & \text{if } |x| \leq \mu, \\ |x| - \mu/2 & \text{if } |x| > \mu. \end{cases} \tag{A.8}$$

This is exactly the Huber function, widely used in robust statistics.

*Smoothness.* $\nabla f_\mu(x) = x/\mu$ for $|x| \le \mu$ and $\operatorname{sign}(x)$ for $|x| > \mu$. The gradient is continuous (both branches give $\pm 1$ at $|x| = \mu$) with Lipschitz constant $L_\mu = 1/\mu$.

*Convergence.* As $\mu \to 0$: $f_\mu(x) \to |x|$ pointwise (the quadratic region shrinks to a point), and $L_\mu = 1/\mu \to \infty$. The smooth approximation becomes arbitrarily close to $f$ but increasingly hard to optimize: this is the smoothing–accuracy trade-off from Section 7.7.

**Three ways to solve SVM** [Section 7.3, Section 8.11, Section 7.7]

The SVM objective $\frac{1}{2}\|w\|^2 + C\sum_i \max\{1 - y^i\langle w, x^i\rangle, 0\}$ is unconstrained but nonsmooth. Discuss three solution strategies and their trade-offs.

Solution. *(a) Subgradient method on the primal.* Apply a subgradient method directly to the nonsmooth objective.

- *Cost per iteration:* $\mathcal{O}(mn)$ for the full subgradient ($m$ samples, $n$ features); $\mathcal{O}(n)$ with stochastic variants (single-sample minibatch).
- *Convergence:* $\mathcal{O}(1/\varepsilon^2)$ (Table 7.2), which is slow for high accuracy.
- *Stopping criterion:* unreliable, since $\|g\|$ small does not guarantee proximity to $f^*$.
- *Verdict:* works well for low-accuracy solutions and scales to very large datasets via stochastic variants (SGD with hinge loss).

*(b) Smooth constrained dual.* Introduce slacks and dualize (Section 8.11): the dual is a smooth QP in $m$ variables with box constraints $0 \le \alpha_i \le C$.

- *Solver options:* projected gradient, active set, or interior point method (Section 9).
- *Accuracy:* fully controllable; duality gap provides a certificate.
- *Kernel trick:* the dual depends on inner products $\langle x^i, x^j\rangle$ only, enabling nonlinear models via kernel substitution without computing the feature map $\varphi$ explicitly.
- *Cost:* forming the kernel matrix is $\mathcal{O}(m^2 n)$; decomposition methods (e.g., libSVM's SMO) avoid this by working on small active subsets.
- *Verdict:* dominant approach for moderate $m$; libSVM-style decomposition is the industry standard.

*(c) Smoothing + accelerated gradient.* Replace the hinge loss $\max\{1 - t, 0\}$ with its Huber-smoothed version (Appendix A.6 applied to a shifted absolute value); apply the accelerated gradient method (Section 6.12).

- *Convergence:* $\mathcal{O}(1/\varepsilon)$, better than (a) by a factor of $1/\varepsilon$.
- *Caveat:* the smoothing parameter $\mu$ interacts with the regularization strength $C$. Small $\mu$ gives better approximation but larger $L_\mu$, forcing smaller steps.
- *Verdict:* theoretically appealing but less practical than (b) for moderate-size problems; useful when $m$ is very large and the dual QP is too expensive.

*Summary.* Approach (b) dominates for moderate $m$ via decomposition; approach (a) or stochastic variants dominate for $m > 10^6$ where even forming the kernel matrix is prohibitive. Approach

(c) occupies a middle ground with better theoretical rates than (a) but more tuning than (b).

## A.7 Constrained Optimization

> **A set that is neither open nor closed** [Result 8.4]
>
> Give examples in $\mathbb{R}$ and $\mathbb{R}^2$.

Solution. *In* $\mathbb{R}$. $S = [0, 1)$. Not open: $0 \in S$, but every open interval around 0 contains negative numbers not in $S$. Not closed: the sequence $x_k = 1 - 1/k$ lies in $S$ and converges to $1 \notin S$. The set contains part of its boundary (0) but not all of it (1 is missing).

*In* $\mathbb{R}^2$. $S = \{(x, y) : x^2 + y^2 < 1\} \cup \{(1, 0)\}$: the open unit disk plus one boundary point. Not open: the point $(1, 0) \in S$ has no open ball entirely inside $S$ (any ball around $(1, 0)$ contains points outside the closed disk). Not closed: the point $(0, 1)$ is a limit of the sequence $(0, 1 - 1/k) \in S$ but $(0, 1) \notin S$.

> **The tangent cone is a cone** [Result 8.5]
>
> Prove: if $d \in T_X(x)$ and $\alpha > 0$, then $\alpha d \in T_X(x)$.

Solution. By definition of the tangent cone, $d \in T_X(x)$ means there exist sequences $\{z_i\} \to x$ with $z_i \in X$ and $\{t_i\} \to 0$ with $t_i > 0$ such that $(z_i - x)/t_i \to d$.

Define $\tilde{t}_i = t_i/\alpha$. Then:

- $\tilde{t}_i > 0$ (since $t_i > 0$ and $\alpha > 0$).
- $\tilde{t}_i \to 0$ (since $t_i \to 0$ and $\alpha$ is fixed).
- $(z_i - x)/\tilde{t}_i = \alpha \cdot (z_i - x)/t_i \to \alpha d$.

The same sequence $\{z_i\}$ with the rescaled denominators $\{\tilde{t}_i\}$ witnesses $\alpha d \in T_X(x)$. Since $\alpha > 0$ was arbitrary, $T_X(x)$ is closed under positive scaling: it is a cone.

> **KKT by hand** [Result 8.13]
>
> Solve $\min\{2x_1^2 + x_2^2 : x_1 + x_2 = 1,\ x_1 \geq 0,\ x_2 \geq 0\}$. Identify active constraints, write KKT, and verify complementary slackness.

Solution. *Setup.* Rewrite the constraints in standard form: equality $h(x) = x_1 + x_2 - 1 = 0$; inequalities $g_1(x) = -x_1 \leq 0$ and $g_2(x) = -x_2 \leq 0$.

*KKT stationarity.* $\nabla f + \mu \nabla h + \lambda_1 \nabla g_1 + \lambda_2 \nabla g_2 = 0$, where $\nabla f = [4x_1, 2x_2]^T$, $\nabla h = [1, 1]^T$, $\nabla g_1 = [-1, 0]^T$, $\nabla g_2 = [0, -1]^T$. This gives the system: $4x_1 + \mu - \lambda_1 = 0$ and $2x_2 + \mu - \lambda_2 = 0$.

*Solving.* Guess that both bound constraints are inactive ($\lambda_1 = \lambda_2 = 0$). Then $\mu = -4x_1$ and $\mu = -2x_2$, so $4x_1 = 2x_2$, i.e., $x_2 = 2x_1$. Combined with $x_1 + x_2 = 1$: $3x_1 = 1$, giving $x_1 = 1/3$, $x_2 = 2/3$, $\mu = -4/3$.

*Verification.* Both $x_1 = 1/3 > 0$ and $x_2 = 2/3 > 0$: the bound constraints are indeed inactive, consistent with $\lambda_1 = \lambda_2 = 0$. Complementary slackness: $\lambda_1 \cdot g_1(x^*) = 0 \cdot (-1/3) = 0$ and

$\lambda_2 \cdot g_2(x^*) = 0 \cdot (-2/3) = 0$, both satisfied. The problem is convex ($f$ convex, constraints affine), so KKT is sufficient for global optimality. Optimal value: $f(1/3, 2/3) = 2/9 + 4/9 = 2/3$.

**When strong duality fails** [Section 8.7]

For $\min\{-x^2 : 0 \leq x \leq 1\}$, compute the dual explicitly and verify the infinite duality gap. Explain the failure in terms of convexity.

Solution. *Primal.* $f(x) = -x^2$ on $[0, 1]$. Since $f$ is concave, its minimum on a closed interval occurs at a vertex: $\nu(P) = \min\{f(0), f(1)\} = \min\{0, -1\} = -1$ at $x^* = 1$.

*Lagrangian.* With $g_1(x) = -x \leq 0$ and $g_2(x) = x - 1 \leq 0$: $L(x; \lambda_1, \lambda_2) = -x^2 - \lambda_1 x + \lambda_2(x - 1)$.

*Dual function.* $\psi(\lambda) = \inf_x L(x; \lambda)$. The stationary point is $\partial L/\partial x = -2x - \lambda_1 + \lambda_2 = 0$, giving $x = (\lambda_2 - \lambda_1)/2$. But $\partial^2 L/\partial x^2 = -2 < 0$: this stationary point is a *maximum*, not a minimum. The Lagrangian is concave in $x$ (since $f$ is concave and the constraint terms are linear), so $\inf_x L(x; \lambda) = -\infty$ for every $\lambda \geq 0$.

*Duality gap.* $\psi(\lambda) = -\infty$ for all $\lambda \geq 0$, so $\nu(D) = \sup_{\lambda \geq 0} \psi(\lambda) = -\infty$. The gap is $\nu(P) - \nu(D) = -1 - (-\infty) = +\infty$.

*Root cause.* Strong duality requires the Lagrangian to have a finite minimizer in $x$. When $f$ is convex and the constraints are affine, $L$ is convex in $x$, guaranteeing a finite minimum. Here $f$ is concave, so $L$ is concave in $x$ and has no finite infimum. This is precisely why Result 8.19 requires convexity of $(P)$.

**Farkas implies separation** [Result 8.12, Section 8.3]

Use Farkas' lemma to prove: if $C \subseteq \mathbb{R}^n$ is a closed convex set and $\bar{x} \notin C$, then $\exists\, a \in \mathbb{R}^n$ and $b \in \mathbb{R}$ such that $\langle a, \bar{x} \rangle > b \geq \langle a, x \rangle$ for all $x \in C$.

Solution. *Closest point.* Since $C$ is closed and convex, the projection $\hat{x} = \arg\min_{x \in C} \|x - \bar{x}\|$ exists and is unique (the squared distance is strictly convex, and closedness guarantees the infimum is attained).

*Separating hyperplane.* Set $a = \bar{x} - \hat{x}$ and $b = \langle a, \hat{x} \rangle$.

*Upper bound ($\langle a, \bar{x} \rangle > b$).* $\langle a, \bar{x} \rangle = \langle \bar{x} - \hat{x}, \bar{x} \rangle = \langle a, \hat{x} \rangle + \langle a, \bar{x} - \hat{x} \rangle = b + \|a\|^2$. Since $\bar{x} \neq \hat{x}$ (because $\bar{x} \notin C$ but $\hat{x} \in C$), we have $\|a\|^2 > 0$, so $\langle a, \bar{x} \rangle > b$.

*Lower bound ($\langle a, x \rangle \leq b$ for all $x \in C$).* Take any $x \in C$. The segment $\hat{x} + t(x - \hat{x}) \in C$ for $t \in [0, 1]$ (by convexity of $C$). The squared distance $\phi(t) = \|\bar{x} - \hat{x} - t(x - \hat{x})\|^2$ is minimized at $t = 0$ (by definition of $\hat{x}$ as the closest point). The derivative at $t = 0$ must be non-negative: $\phi'(0) = -2\langle \bar{x} - \hat{x},\, x - \hat{x} \rangle \geq 0$, i.e., $\langle a,\, x - \hat{x} \rangle \leq 0$, i.e., $\langle a, x \rangle \leq \langle a, \hat{x} \rangle = b$.

*Connection to Farkas.* This is the separating hyperplane theorem: strict separation of a point from a closed convex set. The proof via projection is essentially a geometric version of Farkas' lemma: either a point belongs to a convex set, or it can be separated from it by a hyperplane, exactly the "one of two alternatives" structure that Farkas captures algebraically.

## References


[1] P. Hansen and B. Jaumard. “Lipschitz Optimization”. In: *Handbook of Global Optimization*. Ed. by R. Horst and P. M. Pardalos. Nonconvex Optimization and Its Applications. Springer, 1995. Chap. 8, pp. 407–494.

[2] J. Nocedal and S. J. Wright. *Numerical Optimization*. 2nd. Series in Operations Research and Financial Engineering. Springer, 2006.

[3] E. de Klerk. “The complexity of optimizing over a simplex, hypercube or sphere: a short survey”. In: *Central European Journal of Operations Research* 16 (2008), pp. 111–125.

[4] L. De Giovanni and Di Serafino. “Optimizing Without Derivatives: What Does the No Free Lunch Theorem Actually Say?” In: *Notices of the AMS* 61.7 (2014), pp. 750–755.

[5] M. S. Bazaraa, H. D. Sherali, and C. M. Shetty. *Nonlinear Programming: Theory and Algorithms*. 3rd. John Wiley & Sons, 2006.

[6] W. Sun and Y.-X. Yuan. *Optimization Theory and Methods: Nonlinear Programming*. Optimization and Its Applications. Springer, 2006.

[7] S. Boyd and L. Vandenberghe. *Convex Optimization*. Cambridge University Press, 2008. URL: https://web.stanford.edu/~boyd/cvxbook.

[8] S. Bubeck. “Convex Optimization: Algorithms and Complexity”. In: *Foundations and Trends in Machine Learning* 8.3–4 (2015). arXiv:1405.4980v2, pp. 231–357. URL: https://arxiv.org/abs/1405.4980.

[9] G. d’Antonio and A. Frangioni. “Convergence Analysis of Deflected Conditional Approximate Subgradient Methods”. In: *SIAM Journal on Optimization* 20.1 (2009), pp. 357–386.

[10] Y. Nesterov. “Smooth minimization of non-smooth functions”. In: *Mathematical Programming* 103 (2005), pp. 127–152.

[11] A. Frangioni. “Standard Bundle Methods: Untrusted Models and Duality”. In: *Numerical Nonsmooth Optimization: State of the Art Algorithms*. Ed. by A. M. Bagirov et al. Springer, 2020, pp. 61–116.

[12] D. G. Luenberger and Y. Ye. *Linear and Nonlinear Programming*. 3rd. International Series in Operations Research & Management Science. Springer, 2008.